
Machine-Level Representation of Programs

- 3.1 A Historical Perspective 166
- 3.2 Program Encodings 169
- 3.3 Data Formats 177
- 3.4 Accessing Information 179
- 3.5 Arithmetic and Logical Operations 191
- 3.6 Control 200
- 3.7 Procedures 238
- 3.8 Array Allocation and Access 255
- 3.9 Heterogeneous Data Structures 265
- 3.10 Combining Control and Data in Machine-Level Programs 276
- 3.11 Floating-Point Code 293
- 3.12 Summary 309
- Bibliographic Notes 310
- Homework Problems 311
- Solutions to Practice Problems 325

Computers execute *machine code*, sequences of bytes encoding the low-level operations that manipulate data, manage memory, read and write data on storage devices, and communicate over networks. A compiler generates machine code through a series of stages, based on the rules of the programming language, the instruction set of the target machine, and the conventions followed by the operating system. The gcc C compiler generates its output in the form of *assembly code*, a textual representation of the machine code giving the individual instructions in the program. Gcc then invokes both an *assembler* and a *linker* to generate the executable machine code from the assembly code. In this chapter, we will take a close look at machine code and its human-readable representation as assembly code.

When programming in a high-level language such as C, and even more so in Java, we are shielded from the detailed machine-level implementation of our program. In contrast, when writing programs in assembly code (as was done in the early days of computing) a programmer must specify the low-level instructions the program uses to carry out a computation. Most of the time, it is much more productive and reliable to work at the higher level of abstraction provided by a high-level language. The type checking provided by a compiler helps detect many program errors and makes sure we reference and manipulate data in consistent ways. With modern optimizing compilers, the generated code is usually at least as efficient as what a skilled assembly-language programmer would write by hand. Best of all, a program written in a high-level language can be compiled and executed on a number of different machines, whereas assembly code is highly machine specific.

So why should we spend our time learning machine code? Even though compilers do most of the work in generating assembly code, being able to read and understand it is an important skill for serious programmers. By invoking the compiler with appropriate command-line parameters, the compiler will generate a file showing its output in assembly-code form. By reading this code, we can understand the optimization capabilities of the compiler and analyze the underlying inefficiencies in the code. As we will experience in Chapter 5, programmers seeking to maximize the performance of a critical section of code often try different variations of the source code, each time compiling and examining the generated assembly code to get a sense of how efficiently the program will run. Furthermore, there are times when the layer of abstraction provided by a high-level language hides information about the run-time behavior of a program that we need to understand. For example, when writing concurrent programs using a thread package, as covered in Chapter 12, it is important to understand how program data are shared or kept private by the different threads and precisely how and where shared data are accessed. Such information is visible at the machine-code level. As another example, many of the ways programs can be attacked, allowing malware to infect a system, involve nuances of the way programs store their run-time control information. Many attacks involve exploiting weaknesses in system programs to overwrite information and thereby take control of the system. Understanding how these vulnerabilities arise and how to guard against them requires a knowledge of the machine-level representation of programs. The need for programmers to learn

machine code has shifted over the years from one of being able to write programs directly in assembly code to one of being able to read and understand the code generated by compilers.

In this chapter, we will learn the details of one particular assembly language and see how C programs get compiled into this form of machine code. Reading the assembly code generated by a compiler involves a different set of skills than writing assembly code by hand. We must understand the transformations typical compilers make in converting the constructs of C into machine code. Relative to the computations expressed in the C code, optimizing compilers can rearrange execution order, eliminate unneeded computations, replace slow operations with faster ones, and even change recursive computations into iterative ones. Understanding the relation between source code and the generated assembly can often be a challenge—it's much like putting together a puzzle having a slightly different design than the picture on the box. It is a form of *reverse engineering*—trying to understand the process by which a system was created by studying the system and working backward. In this case, the system is a machine-generated assembly-language program, rather than something designed by a human. This simplifies the task of reverse engineering because the generated code follows fairly regular patterns and we can run experiments, having the compiler generate code for many different programs. In our presentation, we give many examples and provide a number of exercises illustrating different aspects of assembly language and compilers. This is a subject where mastering the details is a prerequisite to understanding the deeper and more fundamental concepts. Those who say “I understand the general principles, I don't want to bother learning the details” are deluding themselves. It is critical for you to spend time studying the examples, working through the exercises, and checking your solutions with those provided.

Our presentation is based on x86-64, the machine language for most of the processors found in today's laptop and desktop machines, as well as those that power very large data centers and supercomputers. This language has evolved over a long history, starting with Intel Corporation's first 16-bit processor in 1978, through to the expansion to 32 bits, and most recently to 64 bits. Along the way, features have been added to make better use of the available semiconductor technology, and to satisfy the demands of the marketplace. Much of the development has been driven by Intel, but its rival Advanced Micro Devices (AMD) has also made important contributions. The result is a rather peculiar design with features that make sense only when viewed from a historical perspective. It is also laden with features providing backward compatibility that are not used by modern compilers and operating systems. We will focus on the subset of the features used by gcc and Linux. This allows us to avoid much of the complexity and many of the arcane features of x86-64.

Our technical presentation starts with a quick tour to show the relation between C, assembly code, and machine code. We then proceed to the details of x86-64, starting with the representation and manipulation of data and the implementation of control. We see how control constructs in C, such as `if`, `while`, and `switch` statements, are implemented. We then cover the implementation of procedures, including how the program maintains a run-time stack to support the

Web Aside ASM:IA32 IA32 programming

IA32, the 32-bit predecessor to x86-64, was introduced by Intel in 1985. It served as the machine language of choice for several decades. Most x86 microprocessors sold today, and most operating systems installed on these machines, are designed to run x86-64. However, they can also execute IA32 programs in a backward compatibility mode. As a result, many application programs are still based on IA32. In addition, many existing systems cannot execute x86-64, due to limitations of their hardware or system software. IA32 continues to be an important machine language. You will find that having a background in x86-64 will enable you to learn the IA32 machine language quite readily.

passing of data and control between procedures, as well as storage for local variables. Next, we consider how data structures such as arrays, structures, and unions are implemented at the machine level. With this background in machine-level programming, we can examine the problems of out-of-bounds memory references and the vulnerability of systems to buffer overflow attacks. We finish this part of the presentation with some tips on using the GDB debugger for examining the run-time behavior of a machine-level program. The chapter concludes with a presentation on machine-program representations of code involving floating-point data and operations.

The computer industry has recently made the transition from 32-bit to 64-bit machines. A 32-bit machine can only make use of around 4 gigabytes (2^{32} bytes) of random access memory. With memory prices dropping at dramatic rates, and our computational demands and data sizes increasing, it has become both economically feasible and technically desirable to go beyond this limitation. Current 64-bit machines can use up to 256 terabytes (2^{48} bytes) of memory, and could readily be extended to use up to 16 exabytes (2^{64} bytes). Although it is hard to imagine having a machine with that much memory, keep in mind that 4 gigabytes seemed like an extreme amount of memory when 32-bit machines became commonplace in the 1970s and 1980s.

Our presentation focuses on the types of machine-level programs generated when compiling C and similar programming languages targeting modern operating systems. As a consequence, we make no attempt to describe many of the features of x86-64 that arise out of its legacy support for the styles of programs written in the early days of microprocessors, when much of the code was written manually and where programmers had to struggle with the limited range of addresses allowed by 16-bit machines.

3.1 A Historical Perspective

The Intel processor line, colloquially referred to as x86, has followed a long evolutionary development. It started with one of the first single-chip 16-bit microprocessors, where many compromises had to be made due to the limited capabilities of integrated circuit technology at the time. Since then, it has grown to take ad-

vantage of technology improvements as well as to satisfy the demands for higher performance and for supporting more advanced operating systems.

The list that follows shows some models of Intel processors and some of their key features, especially those affecting machine-level programming. We use the number of transistors required to implement the processors as an indication of how they have evolved in complexity. In this table, “K” denotes 1,000 (10^3), “M” denotes 1,000,000 (10^6), and “G” denotes 1,000,000,000 (10^9).

8086 (1978, 29 K transistors). One of the first single-chip, 16-bit microprocessors. The 8088, a variant of the 8086 with an 8-bit external bus, formed the heart of the original IBM personal computers. IBM contracted with then-tiny Microsoft to develop the MS-DOS operating system. The original models came with 32,768 bytes of memory and two floppy drives (no hard drive). Architecturally, the machines were limited to a 655,360-byte address space—addresses were only 20 bits long (1,048,576 bytes addressable), and the operating system reserved 393,216 bytes for its own use. In 1980, Intel introduced the 8087 floating-point coprocessor (45 K transistors) to operate alongside an 8086 or 8088 processor, executing the floating-point instructions. The 8087 established the floating-point model for the x86 line, often referred to as “x87.”

80286 (1982, 134 K transistors). Added more (and now obsolete) addressing modes. Formed the basis of the IBM PC-AT personal computer, the original platform for MS Windows.

i386 (1985, 275 K transistors). Expanded the architecture to 32 bits. Added the flat addressing model used by Linux and recent versions of the Windows operating system. This was the first machine in the series that could fully support a Unix operating system.

i486 (1989, 1.2 M transistors). Improved performance and integrated the floating-point unit onto the processor chip but did not significantly change the instruction set.

Pentium (1993, 3.1 M transistors). Improved performance but only added minor extensions to the instruction set.

PentiumPro (1995, 5.5 M transistors). Introduced a radically new processor design, internally known as the *P6* microarchitecture. Added a class of “conditional move” instructions to the instruction set.

Pentium/MMX (1997, 4.5 M transistors). Added new class of instructions to the Pentium processor for manipulating vectors of integers. Each datum can be 1, 2, or 4 bytes long. Each vector totals 64 bits.

Pentium II (1997, 7 M transistors). Continuation of the P6 microarchitecture.

Pentium III (1999, 8.2 M transistors). Introduced SSE, a class of instructions for manipulating vectors of integer or floating-point data. Each datum can be 1, 2, or 4 bytes, packed into vectors of 128 bits. Later versions of this chip

went up to 24 M transistors, due to the incorporation of the level-2 cache on chip.

Pentium 4 (2000, 42 M transistors). Extended SSE to SSE2, adding new data types (including double-precision floating point), along with 144 new instructions for these formats. With these extensions, compilers can use SSE instructions, rather than x87 instructions, to compile floating-point code.

Pentium 4E (2004, 125 M transistors). Added *hyperthreading*, a method to run two programs simultaneously on a single processor, as well as EM64T, Intel's implementation of a 64-bit extension to IA32 developed by Advanced Micro Devices (AMD), which we refer to as x86-64.

Core 2 (2006, 291 M transistors). Returned to a microarchitecture similar to P6. First *multi-core* Intel microprocessor, where multiple processors are implemented on a single chip. Did not support hyperthreading.

Core i7, Nehalem (2008, 781 M transistors). Incorporated both hyperthreading and multi-core, with the initial version supporting two executing programs on each core and up to four cores on each chip.

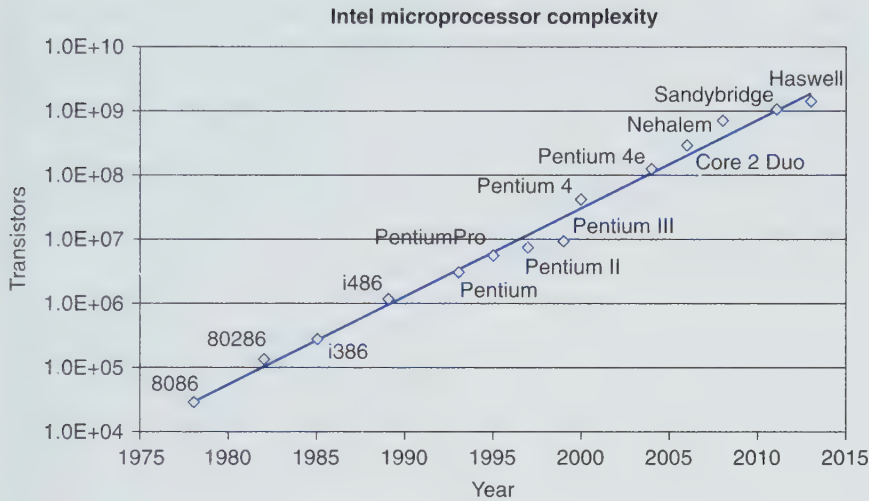
Core i7, Sandy Bridge (2011, 1.17 G transistors). Introduced AVX, an extension of the SSE to support data packed into 256-bit vectors.

Core i7, Haswell (2013, 1.4 G transistors). Extended AVX to AVX2, adding more instructions and instruction formats.

Each successive processor has been designed to be backward compatible—able to run code compiled for any earlier version. As we will see, there are many strange artifacts in the instruction set due to this evolutionary heritage. Intel has had several names for their processor line, including *IA32*, for “Intel Architecture 32-bit” and most recently *Intel64*, the 64-bit extension to IA32, which we will refer to as *x86-64*. We will refer to the overall line by the commonly used colloquial name “x86,” reflecting the processor naming conventions up through the i486.

Over the years, several companies have produced processors that are compatible with Intel processors, capable of running the exact same machine-level programs. Chief among these is Advanced Micro Devices (AMD). For years, AMD lagged just behind Intel in technology, forcing a marketing strategy where they produced processors that were less expensive although somewhat lower in performance. They became more competitive around 2002, being the first to break the 1-gigahertz clock-speed barrier for a commercially available microprocessor, and introducing x86-64, the widely adopted 64-bit extension to Intel's IA32. Although we will talk about Intel processors, our presentation holds just as well for the compatible processors produced by Intel's rivals.

Much of the complexity of x86 is not of concern to those interested in programs for the Linux operating system as generated by the gcc compiler. The memory model provided in the original 8086 and its extensions in the 80286 became obsolete with the i386. The original x87 floating-point instructions became obsolete

Aside Moore's Law

If we plot the number of transistors in the different Intel processors versus the year of introduction, and use a logarithmic scale for the y -axis, we can see that the growth has been phenomenal. Fitting a line through the data, we see that the number of transistors increases at an annual rate of approximately 37%, meaning that the number of transistors doubles about every 26 months. This growth has been sustained over the multiple-decade history of x86 microprocessors.

In 1965, Gordon Moore, a founder of Intel Corporation, extrapolated from the chip technology of the day (by which they could fabricate circuits with around 64 transistors on a single chip) to predict that the number of transistors per chip would double every year for the next 10 years. This prediction became known as *Moore's Law*. As it turns out, his prediction was just a little bit optimistic, but also too short-sighted. Over more than 50 years, the semiconductor industry has been able to double transistor counts on average every 18 months.

Similar exponential growth rates have occurred for other aspects of computer technology, including the storage capacities of magnetic disks and semiconductor memories. These remarkable growth rates have been the major driving forces of the computer revolution.

with the introduction of SSE2. Although we see vestiges of the historical evolution of x86 in x86-64 programs, many of the most arcane features of x86 do not appear.

3.2 Program Encodings

Suppose we write a C program as two files `p1.c` and `p2.c`. We can then compile this code using a Unix command line:

```
linux> gcc -Og -o p p1.c p2.c
```

The command `gcc` indicates the gcc C compiler. Since this is the default compiler on Linux, we could also invoke it as simply `cc`. The command-line option `-Og`¹ instructs the compiler to apply a level of optimization that yields machine code that follows the overall structure of the original C code. Invoking higher levels of optimization can generate code that is so heavily transformed that the relationship between the generated machine code and the original source code is difficult to understand. We will therefore use `-Og` optimization as a learning tool and then see what happens as we increase the level of optimization. In practice, higher levels of optimization (e.g., specified with the option `-O1` or `-O2`) are considered a better choice in terms of the resulting program performance.

The `gcc` command invokes an entire sequence of programs to turn the source code into executable code. First, the *C preprocessor* expands the source code to include any files specified with `#include` commands and to expand any macros, specified with `#define` declarations. Second, the *compiler* generates assembly-code versions of the two source files having names `p1.s` and `p2.s`. Next, the *assembler* converts the assembly code into binary *object-code* files `p1.o` and `p2.o`. Object code is one form of machine code—it contains binary representations of all of the instructions, but the addresses of global values are not yet filled in. Finally, the *linker* merges these two object-code files along with code implementing library functions (e.g., `printf`) and generates the final executable code file `p` (as specified by the command-line directive `-o p`). Executable code is the second form of machine code we will consider—it is the exact form of code that is executed by the processor. The relation between these different forms of machine code and the linking process is described in more detail in Chapter 7.

3.2.1 Machine-Level Code

As described in Section 1.9.3, computer systems employ several different forms of abstraction, hiding details of an implementation through the use of a simpler abstract model. Two of these are especially important for machine-level programming. First, the format and behavior of a machine-level program is defined by the *instruction set architecture*, or ISA, defining the processor state, the format of the instructions, and the effect each of these instructions will have on the state. Most ISAs, including x86-64, describe the behavior of a program as if each instruction is executed in sequence, with one instruction completing before the next one begins. The processor hardware is far more elaborate, executing many instructions concurrently, but it employs safeguards to ensure that the overall behavior matches the sequential operation dictated by the ISA. Second, the memory addresses used by a machine-level program are *virtual addresses*, providing a memory model that

1. This optimization level was introduced in gcc version 4.8. Earlier versions of gcc, as well as non-GNU compilers, will not recognize this option. For these, using optimization level one (specified with the command-line flag `-O1`) is probably the best choice for generating code that follows the original program structure.

appears to be a very large byte array. The actual implementation of the memory system involves a combination of multiple hardware memories and operating system software, as described in Chapter 9.

The compiler does most of the work in the overall compilation sequence, transforming programs expressed in the relatively abstract execution model provided by C into the very elementary instructions that the processor executes. The assembly-code representation is very close to machine code. Its main feature is that it is in a more readable textual format, as compared to the binary format of machine code. Being able to understand assembly code and how it relates to the original C code is a key step in understanding how computers execute programs.

The machine code for x86-64 differs greatly from the original C code. Parts of the processor state are visible that normally are hidden from the C programmer:

- The *program counter* (commonly referred to as the PC, and called `%rip` in x86-64) indicates the address in memory of the next instruction to be executed.
- The integer *register file* contains 16 named locations storing 64-bit values. These registers can hold addresses (corresponding to C pointers) or integer data. Some registers are used to keep track of critical parts of the program state, while others are used to hold temporary data, such as the arguments and local variables of a procedure, as well as the value to be returned by a function.
- The condition code registers hold status information about the most recently executed arithmetic or logical instruction. These are used to implement conditional changes in the control or data flow, such as is required to implement `if` and `while` statements.
- A set of vector registers can each hold one or more integer or floating-point values.

Whereas C provides a model in which objects of different data types can be declared and allocated in memory, machine code views the memory as simply a large byte-addressable array. Aggregate data types in C such as arrays and structures are represented in machine code as contiguous collections of bytes. Even for scalar data types, assembly code makes no distinctions between signed or unsigned integers, between different types of pointers, or even between pointers and integers.

The program memory contains the executable machine code for the program, some information required by the operating system, a run-time stack for managing procedure calls and returns, and blocks of memory allocated by the user (e.g., by using the `malloc` library function). As mentioned earlier, the program memory is addressed using virtual addresses. At any given time, only limited subranges of virtual addresses are considered valid. For example, x86-64 virtual addresses are represented by 64-bit words. In current implementations of these machines, the upper 16 bits must be set to zero, and so an address can potentially specify a byte over a range of 2^{48} , or 64 terabytes. More typical programs will only have access to a few megabytes, or perhaps several gigabytes. The operating system manages

Aside The ever-changing forms of generated code

In our presentation, we will show the code generated by a particular version of gcc with particular settings of the command-line options. If you compile code on your own machine, chances are you will be using a different compiler or a different version of gcc and hence will generate different code. The open-source community supporting gcc keeps changing the code generator, attempting to generate more efficient code according to changing code guidelines provided by the microprocessor manufacturers.

Our goal in studying the examples shown in our presentation is to demonstrate how to examine assembly code and map it back to the constructs found in high-level programming languages. You will need to adapt these techniques to the style of code generated by your particular compiler.

this virtual address space, translating virtual addresses into the physical addresses of values in the actual processor memory.

A single machine instruction performs only a very elementary operation. For example, it might add two numbers stored in registers, transfer data between memory and a register, or conditionally branch to a new instruction address. The compiler must generate sequences of such instructions to implement program constructs such as arithmetic expression evaluation, loops, or procedure calls and returns.

3.2.2 Code Examples

Suppose we write a C code file `mstore.c` containing the following function definition:

```
long mult2(long, long);

void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);
    *dest = t;
}
```

To see the assembly code generated by the C compiler, we can use the `-S` option on the command line:

```
linux> gcc -Og -S mstore.c
```

This will cause gcc to run the compiler, generating an assembly file `mstore.s`, and go no further. (Normally it would then invoke the assembler to generate an object-code file.)

The assembly-code file contains various declarations, including the following set of lines:

```
multstore:
    pushq    %rbx
```

Aside How do I display the byte representation of a program?

To display the binary object code for a program (say, `mstore`), we use a *disassembler* (described below) to determine that the code for the procedure is 14 bytes long. Then we run the GNU debugging tool GDB on file `mstore.o` and give it the command

```
(gdb) x/14xb multstore
```

telling it to display (abbreviated ‘x’) 14 hex-formatted (also ‘x’) bytes (‘b’) starting at the address where function `multstore` is located. You will find that GDB has many useful features for analyzing machine-level programs, as will be discussed in Section 3.10.2.

```
movq    %rdx, %rbx
call    mult2
movq    %rax, (%rbx)
popq    %rbx
ret
```

Each indented line in the code corresponds to a single machine instruction. For example, the `pushq` instruction indicates that the contents of register `%rbx` should be pushed onto the program stack. All information about local variable names or data types has been stripped away.

If we use the `-c` command-line option, `gcc` will both compile and assemble the code

```
linux> gcc -Og -c mstore.c
```

This will generate an object-code file `mstore.o` that is in binary format and hence cannot be viewed directly. Embedded within the 1,368 bytes of the file `mstore.o` is a 14-byte sequence with the hexadecimal representation

```
53 48 89 d3 e8 00 00 00 00 48 89 03 5b c3
```

This is the object code corresponding to the assembly instructions listed previously. A key lesson to learn from this is that the program executed by the machine is simply a sequence of bytes encoding a series of instructions. The machine has very little information about the source code from which these instructions were generated.

To inspect the contents of machine-code files, a class of programs known as *disassemblers* can be invaluable. These programs generate a format similar to assembly code from the machine code. With Linux systems, the program `OBJDUMP` (for “object dump”) can serve this role given the `-d` command-line flag:

```
linux> objdump -d mstore.o
```

The result (where we have added line numbers on the left and annotations in italicized text) is as follows:

Disassembly of function sum in binary file mstore.o

1	0000000000000000 <multstore>:		
	<i>Offset</i>	<i>Bytes</i>	<i>Equivalent assembly language</i>
2	0:	53	push %rbx
3	1:	48 89 d3	mov %rdx,%rbx
4	4:	e8 00 00 00 00	callq 9 <multstore+0x9>
5	9:	48 89 03	mov %rax,(%rbx)
6	c:	5b	pop %rbx
7	d:	c3	retq

On the left we see the 14 hexadecimal byte values, listed in the byte sequence shown earlier, partitioned into groups of 1 to 5 bytes each. Each of these groups is a single instruction, with the assembly-language equivalent shown on the right.

Several features about machine code and its disassembled representation are worth noting:

- x86-64 instructions can range in length from 1 to 15 bytes. The instruction encoding is designed so that commonly used instructions and those with fewer operands require a smaller number of bytes than do less common ones or ones with more operands.
- The instruction format is designed in such a way that from a given starting position, there is a unique decoding of the bytes into machine instructions. For example, only the instruction `pushq %rbx` can start with byte value 53.
- The disassembler determines the assembly code based purely on the byte sequences in the machine-code file. It does not require access to the source or assembly-code versions of the program.
- The disassembler uses a slightly different naming convention for the instructions than does the assembly code generated by gcc. In our example, it has omitted the suffix ‘q’ from many of the instructions. These suffixes are size designators and can be omitted in most cases. Conversely, the disassembler adds the suffix ‘q’ to the `call` and `ret` instructions. Again, these suffixes can safely be omitted.

Generating the actual executable code requires running a linker on the set of object-code files, one of which must contain a function `main`. Suppose in file `main.c` we had the following function:

```
#include <stdio.h>

void multstore(long, long, long *);

int main() {
    long d;
    multstore(2, 3, &d);
    printf("2 * 3 --> %ld\n", d);
    return 0;
}
```



```
long mult2(long a, long b) {
    long s = a * b;
    return s;
}
```

Then we could generate an executable program `prog` as follows:

```
linux> gcc -Og -o prog main.c mstore.c
```

The file `prog` has grown to 8,655 bytes, since it contains not just the machine code for the procedures we provided but also code used to start and terminate the program as well as to interact with the operating system.

We can disassemble the file `prog`:

```
linux> objdump -d prog
```

The disassembler will extract various code sequences, including the following:

```
Disassembly of function sum in binary file prog
1 0000000000400540 <multstore>:
2 400540: 53                push    %rbx
3 400541: 48 89 d3          mov     %rdx,%rbx
4 400544: e8 42 00 00 00    callq  40058b <mult2>
5 400549: 48 89 03          mov     %rax,(%rbx)
6 40054c: 5b              pop     %rbx
7 40054d: c3              retq
8 40054e: 90              nop
9 40054f: 90              nop
```

This code is almost identical to that generated by the disassembly of `mstore.c`. One important difference is that the addresses listed along the left are different—the linker has shifted the location of this code to a different range of addresses. A second difference is that the linker has filled in the address that the `callq` instruction should use in calling the function `mult2` (line 4 of the disassembly). One task for the linker is to match function calls with the locations of the executable code for those functions. A final difference is that we see two additional lines of code (lines 8–9). These instructions will have no effect on the program, since they occur after the return instruction (line 7). They have been inserted to grow the code for the function to 16 bytes, enabling a better placement of the next block of code in terms of memory system performance.

3.2.3 Notes on Formatting

The assembly code generated by `gcc` is difficult for a human to read. On one hand, it contains information with which we need not be concerned, while on the other hand, it does not provide any description of the program or how it works. For example, suppose we give the command

```
linux> gcc -Og -S mstore.c
```

to generate the file `mstore.s`. The full content of the file is as follows:

```
.file    "010-mstore.c"
.text
.globl   multstore
.type    multstore, @function
multstore:
    pushq    %rbx
    movq     %rdx, %rbx
    call     mult2
    movq     %rax, (%rbx)
    popq     %rbx
    ret
.size      multstore, .-multstore
.ident     "GCC: (Ubuntu 4.8.1-2ubuntu1~12.04) 4.8.1"
.section   .note.GNU-stack,"",@progbits
```

All of the lines beginning with ‘.’ are directives to guide the assembler and linker. We can generally ignore these. On the other hand, there are no explanatory remarks about what the instructions do or how they relate to the source code.

To provide a clearer presentation of assembly code, we will show it in a form that omits most of the directives, while including line numbers and explanatory annotations. For our example, an annotated version would appear as follows:

```
void multstore(long x, long y, long *dest)
x in %rdi, y in %rsi, dest in %rdx
1  multstore:
2      pushq    %rbx           Save %rbx
3      movq     %rdx, %rbx     Copy dest to %rbx
4      call     mult2          Call mult2(x, y)
5      movq     %rax, (%rbx)   Store result at *dest
6      popq     %rbx           Restore %rbx
7      ret                Return
```

We typically show only the lines of code relevant to the point being discussed. Each line is numbered on the left for reference and annotated on the right by a brief description of the effect of the instruction and how it relates to the computations of the original C code. This is a stylized version of the way assembly-language programmers format their code.

We also provide Web asides to cover material intended for dedicated machine-language enthusiasts. One Web aside describes IA32 machine code. Having a background in x86-64 makes learning IA32 fairly simple. Another Web aside gives a brief presentation of ways to incorporate assembly code into C programs. For some applications, the programmer must drop down to assembly code to access low-level features of the machine. One approach is to write entire functions in assembly code and combine them with C functions during the linking stage. A

Aside ATT versus Intel assembly-code formats

In our presentation, we show assembly code in ATT format (named after AT&T, the company that operated Bell Laboratories for many years), the default format for GCC, OBJDUMP, and the other tools we will consider. Other programming tools, including those from Microsoft as well as the documentation from Intel, show assembly code in *Intel* format. The two formats differ in a number of ways. As an example, GCC can generate code in Intel format for the `sum` function using the following command line:

```
linux> gcc -Og -S -masm=intel mstore.c
```

This gives the following assembly code:

```
multstore:
    push    rbx
    mov     rbx, rdx
    call    mult2
    mov     QWORD PTR [rbx], rax
    pop     rbx
    ret
```

We see that the Intel and ATT formats differ in the following ways:

- The Intel code omits the size designation suffixes. We see instruction `push` and `mov` instead of `pushq` and `movq`.
- The Intel code omits the ‘%’ character in front of register names, using `rbx` instead of `%rbx`.
- The Intel code has a different way of describing locations in memory—for example, `QWORD PTR [rbx]` rather than `(%rbx)`.
- Instructions with multiple operands list them in the reverse order. This can be very confusing when switching between the two formats.

Although we will not be using Intel format in our presentation, you will encounter it in documentation from Intel and Microsoft.

second is to use GCC’s support for embedding assembly code directly within C programs.

3.3 Data Formats

Due to its origins as a 16-bit architecture that expanded into a 32-bit one, Intel uses the term “word” to refer to a 16-bit data type. Based on this, they refer to 32-bit quantities as “double words,” and 64-bit quantities as “quad words.” Figure 3.1 shows the x86-64 representations used for the primitive data types of C. Standard `int` values are stored as double words (32 bits). Pointers (shown here as `char *`) are stored as 8-byte quad words, as would be expected in a 64-bit machine. With x86-64, data type `long` is implemented with 64 bits, allowing a very wide range of values. Most of our code examples in this chapter use pointers and long data

Web Aside ASM:EASM Combining assembly code with C programs

Although a C compiler does a good job of converting the computations expressed in a program into machine code, there are some features of a machine that cannot be accessed by a C program. For example, every time an x86-64 processor executes an arithmetic or logical operation, it sets a 1-bit *condition code* flag, named PF (for “parity flag”), to 1 when the lower 8 bits in the resulting computation have an even number of ones and to 0 otherwise. Computing this information in C requires at least seven shifting, masking, and EXCLUSIVE-OR operations (see Problem 2.65). Even though the hardware performs this computation as part of every arithmetic or logical operation, there is no way for a C program to determine the value of the PF condition code flag. This task can readily be performed by incorporating a small number of assembly-code instructions into the program.

There are two ways to incorporate assembly code into C programs. First, we can write an entire function as a separate assembly-code file and let the assembler and linker combine this with code we have written in C. Second, we can use the *inline assembly* feature of gcc, where brief sections of assembly code can be incorporated into a C program using the `asm` directive. This approach has the advantage that it minimizes the amount of machine-specific code.

Of course, including assembly code in a C program makes the code specific to a particular class of machines (such as x86-64), and so it should only be used when the desired feature can only be accessed in this way.

C declaration	Intel data type	Assembly-code suffix	Size (bytes)
<code>char</code>	Byte	<code>b</code>	1
<code>short</code>	Word	<code>w</code>	2
<code>int</code>	Double word	<code>l</code>	4
<code>long</code>	Quad word	<code>q</code>	8
<code>char *</code>	Quad word	<code>q</code>	8
<code>float</code>	Single precision	<code>s</code>	4
<code>double</code>	Double precision	<code>l</code>	8

Figure 3.1 Sizes of C data types in x86-64. With a 64-bit machine, pointers are 8 bytes long.

types, and so they will operate on quad words. The x86-64 instruction set includes a full complement of instructions for bytes, words, and double words as well.

Floating-point numbers come in two principal formats: single-precision (4-byte) values, corresponding to C data type `float`, and double-precision (8-byte) values, corresponding to C data type `double`. Microprocessors in the x86 family historically implemented all floating-point operations with a special 80-bit (10-byte) floating-point format (see Problem 2.86). This format can be specified in C programs using the declaration `long double`. We recommend against using this format, however. It is not portable to other classes of machines, and it is typically

not implemented with the same high-performance hardware as is the case for single- and double-precision arithmetic.

As the table of Figure 3.1 indicates, most assembly-code instructions generated by gcc have a single-character suffix denoting the size of the operand. For example, the data movement instruction has four variants: `movb` (move byte), `movw` (move word), `movl` (move double word), and `movq` (move quad word). The suffix ‘l’ is used for double words, since 32-bit quantities are considered to be “long words.” The assembly code uses the suffix ‘l’ to denote a 4-byte integer as well as an 8-byte double-precision floating-point number. This causes no ambiguity, since floating-point code involves an entirely different set of instructions and registers.

3.4 Accessing Information

An x86-64 central processing unit (CPU) contains a set of 16 *general-purpose registers* storing 64-bit values. These registers are used to store integer data as well as pointers. Figure 3.2 diagrams the 16 registers. Their names all begin with `%r`, but otherwise follow multiple different naming conventions, owing to the historical evolution of the instruction set. The original 8086 had eight 16-bit registers, shown in Figure 3.2 as registers `%ax` through `%bp`. Each had a specific purpose, and hence they were given names that reflected how they were to be used. With the extension to IA32, these registers were expanded to 32-bit registers, labeled `%eax` through `%ebp`. In the extension to x86-64, the original eight registers were expanded to 64 bits, labeled `%rax` through `%rbp`. In addition, eight new registers were added, and these were given labels according to a new naming convention: `%r8` through `%r15`.

As the nested boxes in Figure 3.2 indicate, instructions can operate on data of different sizes stored in the low-order bytes of the 16 registers. Byte-level operations can access the least significant byte, 16-bit operations can access the least significant 2 bytes, 32-bit operations can access the least significant 4 bytes, and 64-bit operations can access entire registers.

In later sections, we will present a number of instructions for copying and generating 1-, 2-, 4-, and 8-byte values. When these instructions have registers as destinations, two conventions arise for what happens to the remaining bytes in the register for instructions that generate less than 8 bytes: Those that generate 1- or 2-byte quantities leave the remaining bytes unchanged. Those that generate 4-byte quantities set the upper 4 bytes of the register to zero. The latter convention was adopted as part of the expansion from IA32 to x86-64.

As the annotations along the right-hand side of Figure 3.2 indicate, different registers serve different roles in typical programs. Most unique among them is the stack pointer, `%rsp`, used to indicate the end position in the run-time stack. Some instructions specifically read and write this register. The other 15 registers have more flexibility in their uses. A small number of instructions make specific use of certain registers. More importantly, a set of standard programming conventions governs how the registers are to be used for managing the stack, passing function

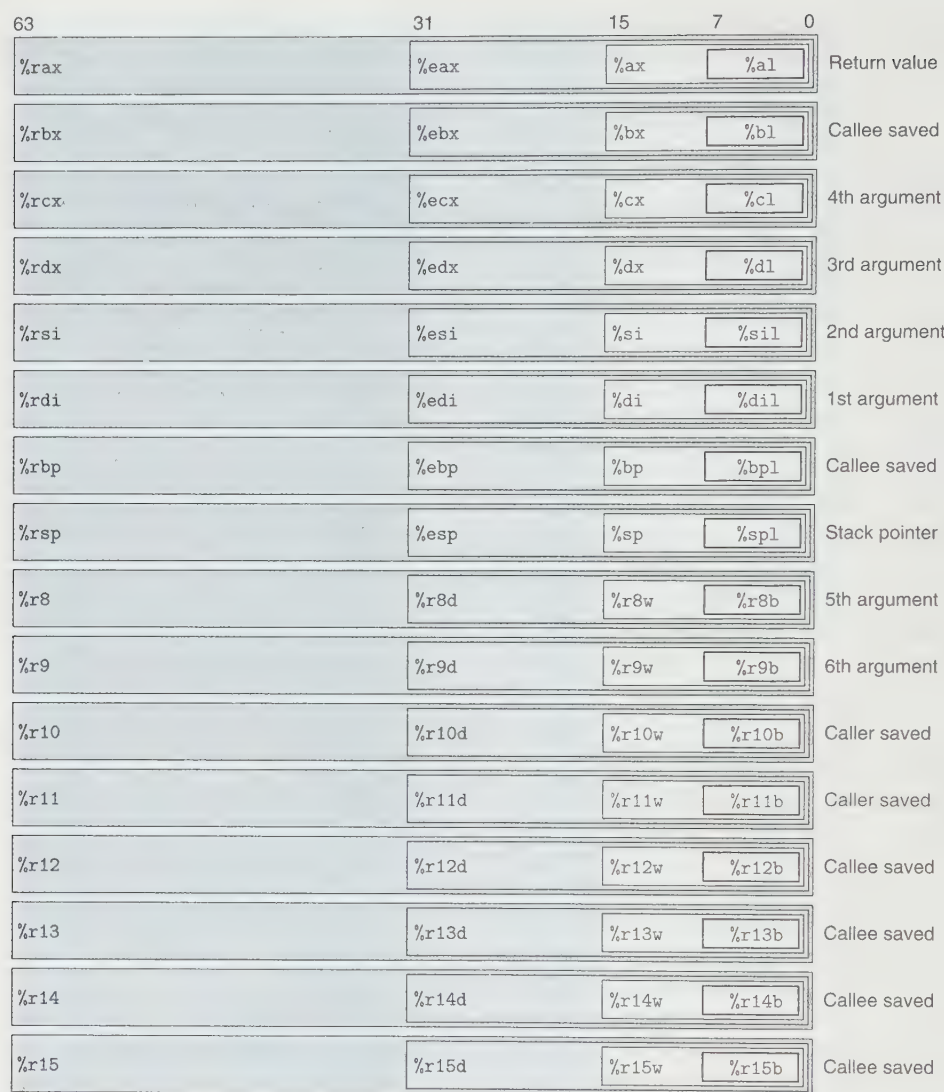


Figure 3.2 Integer registers. The low-order portions of all 16 registers can be accessed as byte, word (16-bit), double word (32-bit), and quad word (64-bit) quantities.

arguments, returning values from functions, and storing local and temporary data. We will cover these conventions in our presentation, especially in Section 3.7, where we describe the implementation of procedures.

3.4.1 Operand Specifiers

Most instructions have one or more *operands* specifying the source values to use in performing an operation and the destination location into which to place the

Type	Form	Operand value	Name
Immediate	$\$Imm$	Imm	Immediate
Register	r_a	$R[r_a]$	Register
Memory	Imm	$M[Imm]$	Absolute
Memory	(r_a)	$M[R[r_a]]$	Indirect
Memory	$Imm(r_b)$	$M[Imm + R[r_b]]$	Base + displacement
Memory	(r_b, r_i)	$M[R[r_b] + R[r_i]]$	Indexed
Memory	$Imm(r_b, r_i)$	$M[Imm + R[r_b] + R[r_i]]$	Indexed
Memory	$(, r_i, s)$	$M[R[r_i] \cdot s]$	Scaled indexed
Memory	$Imm(, r_i, s)$	$M[Imm + R[r_i] \cdot s]$	Scaled indexed
Memory	(r_b, r_i, s)	$M[R[r_b] + R[r_i] \cdot s]$	Scaled indexed
Memory	$Imm(r_b, r_i, s)$	$M[Imm + R[r_b] + R[r_i] \cdot s]$	Scaled indexed

Figure 3.3 Operand forms. Operands can denote immediate (constant) values, register values, or values from memory. The scaling factor s must be either 1, 2, 4, or 8.

result. x86-64 supports a number of operand forms (see Figure 3.3). Source values can be given as constants or read from registers or memory. Results can be stored in either registers or memory. Thus, the different operand possibilities can be classified into three types. The first type, *immediate*, is for constant values. In ATT-format assembly code, these are written with a ‘\$’ followed by an integer using standard C notation—for example, $\$-577$ or $\$0x1F$. Different instructions allow different ranges of immediate values; the assembler will automatically select the most compact way of encoding a value. The second type, *register*, denotes the contents of a register, one of the sixteen 8-, 4-, 2-, or 1-byte low-order portions of the registers for operands having 64, 32, 16, or 8 bits, respectively. In Figure 3.3, we use the notation r_a to denote an arbitrary register a and indicate its value with the reference $R[r_a]$, viewing the set of registers as an array R indexed by register identifiers.

The third type of operand is a *memory* reference, in which we access some memory location according to a computed address, often called the *effective address*. Since we view the memory as a large array of bytes, we use the notation $M_b[Addr]$ to denote a reference to the b -byte value stored in memory starting at address $Addr$. To simplify things, we will generally drop the subscript b .

As Figure 3.3 shows, there are many different *addressing modes* allowing different forms of memory references. The most general form is shown at the bottom of the table with syntax $Imm(r_b, r_i, s)$. Such a reference has four components: an immediate offset Imm , a base register r_b , an index register r_i , and a scale factor s , where s must be 1, 2, 4, or 8. Both the base and index must be 64-bit registers. The effective address is computed as $Imm + R[r_b] + R[r_i] \cdot s$. This general form is often seen when referencing elements of arrays. The other forms are simply special cases of this general form where some of the components are omitted. As we

will see, the more complex addressing modes are useful when referencing array and structure elements.

Practice Problem 3.1 (solution page 325)

Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Fill in the following table showing the values for the indicated operands:

Operand	Value
%rax	_____
0x104	_____
\$0x108	_____
(%rax)	_____
4(%rax)	_____
9(%rax,%rdx)	_____
260(%rcx,%rdx)	_____
0xFC(,%rcx,4)	_____
(%rax,%rdx,4)	_____

3.4.2 Data Movement Instructions

Among the most heavily used instructions are those that copy data from one location to another. The generality of the operand notation allows a simple data movement instruction to express a range of possibilities that in many machines would require a number of different instructions. We present a number of different data movement instructions, differing in their source and destination types, what conversions they perform, and other side effects they may have. In our presentation, we group the many different instructions into *instruction classes*, where the instructions in a class perform the same operation but with different operand sizes.

Figure 3.4 lists the simplest form of data movement instructions—*mov* class. These instructions copy data from a source location to a destination location, without any transformation. The class consists of four instructions: *movb*, *movw*, *movl*, and *movq*. All four of these instructions have similar effects; they differ primarily in that they operate on data of different sizes: 1, 2, 4, and 8 bytes, respectively.

Instruction		Effect	Description
MOV	<i>S, D</i>	$D \leftarrow S$	Move
movb			Move byte
movw			Move word
movl			Move double word
movq			Move quad word
movabsq	<i>I, R</i>	$R \leftarrow I$	Move absolute quad word

Figure 3.4 Simple data movement instructions.

The source operand designates a value that is immediate, stored in a register, or stored in memory. The destination operand designates a location that is either a register or a memory address. x86-64 imposes the restriction that a move instruction cannot have both operands refer to memory locations. Copying a value from one memory location to another requires two instructions—the first to load the source value into a register, and the second to write this register value to the destination. Referring to Figure 3.2, register operands for these instructions can be the labeled portions of any of the 16 registers, where the size of the register must match the size designated by the last character of the instruction (*'b'*, *'w'*, *'l'*, or *'q'*). For most cases, the `MOV` instructions will only update the specific register bytes or memory locations indicated by the destination operand. The only exception is that when `movl` has a register as the destination, it will also set the high-order 4 bytes of the register to 0. This exception arises from the convention, adopted in x86-64, that any instruction that generates a 32-bit value for a register also sets the high-order portion of the register to 0.

The following `MOV` instruction examples show the five possible combinations of source and destination types. Recall that the source operand comes first and the destination second.

1	<code>movl \$0x4050,%eax</code>	<i>Immediate--Register, 4 bytes</i>
2	<code>movw %bp,%sp</code>	<i>Register--Register, 2 bytes</i>
3	<code>movb (%rdi,%rcx),%al</code>	<i>Memory--Register, 1 byte</i>
4	<code>movb \$-17,(%esp)</code>	<i>Immediate--Memory, 1 byte</i>
5	<code>movq %rax,-12(%rbp)</code>	<i>Register--Memory, 8 bytes</i>

A final instruction documented in Figure 3.4 is for dealing with 64-bit immediate data. The regular `movq` instruction can only have immediate source operands that can be represented as 32-bit two's-complement numbers. This value is then sign extended to produce the 64-bit value for the destination. The `movabsq` instruction can have an arbitrary 64-bit immediate value as its source operand and can only have a register as a destination.

Figures 3.5 and 3.6 document two classes of data movement instructions for use when copying a smaller source value to a larger destination. All of these instructions copy data from a source, which can be either a register or stored

Aside Understanding how data movement changes a destination register

As described, there are two different conventions regarding whether and how data movement instructions modify the upper bytes of a destination register. This distinction is illustrated by the following code sequence:

```

1  movabsq $0x0011223344556677, %rax    %rax = 0011223344556677
2  movb    $-1, %al                     %rax = 00112233445566FF
3  movw    $-1, %ax                     %rax = 001122334455FFFF
4  movl    $-1, %eax                    %rax = 00000000FFFFFFFF
5  movq    $-1, %rax                    %rax = FFFFFFFFFFFFFFFF

```

In the following discussion, we use hexadecimal notation. In the example, the instruction on line 1 initializes register `%rax` to the pattern 0011223344556677. The remaining instructions have immediate value `-1` as their source values. Recall that the hexadecimal representation of `-1` is of the form `FF...F`, where the number of `F`'s is twice the number of bytes in the representation. The `movb` instruction (line 2) therefore sets the low-order byte of `%rax` to `FF`, while the `movw` instruction (line 3) sets the low-order 2 bytes to `FFFF`, with the remaining bytes unchanged. The `movl` instruction (line 4) sets the low-order 4 bytes to `FFFFFFFF`, but it also sets the high-order 4 bytes to `00000000`. Finally, the `movq` instruction (line 5) sets the complete register to `FFFFFFFFFFFFFFFF`.

Instruction	Effect	Description
<code>MOVZ S, R</code>	$R \leftarrow \text{ZeroExtend}(S)$	Move with zero extension
<code>movzwb</code>		Move zero-extended byte to word
<code>movzbl</code>		Move zero-extended byte to double word
<code>movzwl</code>		Move zero-extended word to double word
<code>movzbq</code>		Move zero-extended byte to quad word
<code>movzwq</code>		Move zero-extended word to quad word

Figure 3.5 Zero-extending data movement instructions. These instructions have a register or memory location as the source and a register as the destination.

in memory, to a register destination. Instructions in the `movz` class fill out the remaining bytes of the destination with zeros, while those in the `movs` class fill them out by sign extension, replicating copies of the most significant bit of the source operand. Observe that each instruction name has size designators as its final two characters—the first specifying the source size, and the second specifying the destination size. As can be seen, there are three instructions in each of these classes, covering all cases of 1- and 2-byte source sizes and 2- and 4-byte destination sizes, considering only cases where the destination is larger than the source, of course.

Instruction	Effect	Description
<code>movs S, R</code>	$R \leftarrow \text{SignExtend}(S)$	Move with sign extension
<code>movsbw</code>		Move sign-extended byte to word
<code>movsbl</code>		Move sign-extended byte to double word
<code>movswl</code>		Move sign-extended word to double word
<code>movsbq</code>		Move sign-extended byte to quad word
<code>movswq</code>		Move sign-extended word to quad word
<code>movslq</code>		Move sign-extended double word to quad word
<code>cmtq</code>	$\%rax \leftarrow \text{SignExtend}(\%eax)$	Sign-extend <code>%eax</code> to <code>%rax</code>

Figure 3.6 Sign-extending data movement instructions. The `movs` instructions have a register or memory location as the source and a register as the destination. The `cmtq` instruction is specific to registers `%eax` and `%rax`.

Note the absence of an explicit instruction to zero-extend a 4-byte source value to an 8-byte destination in Figure 3.5. Such an instruction would logically be named `movz1q`, but this instruction does not exist. Instead, this type of data movement can be implemented using a `movl` instruction having a register as the destination. This technique takes advantage of the property that an instruction generating a 4-byte value with a register as the destination will fill the upper 4 bytes with zeros. Otherwise, for 64-bit destinations, moving with sign extension is supported for all three source types, and moving with zero extension is supported for the two smaller source types.

Figure 3.6 also documents the `cmtq` instruction. This instruction has no operands—it always uses register `%eax` as its source and `%rax` as the destination for the sign-extended result. It therefore has the exact same effect as the instruction `movslq %eax, %rax`, but it has a more compact encoding.

Practice Problem 3.2 (solution page 325)

For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, `mov` can be rewritten as `movb`, `movw`, `movl`, or `movq`.)

```

mov__  %eax, (%rsp)
mov__  (%rax), %dx
mov__  $0xFF, %bl
mov__  (%rsp,%rdx,4), %dl
mov__  (%rdx), %rax
mov__  %dx, (%rax)

```

Aside Comparing byte movement instructions

The following example illustrates how different data movement instructions either do or do not change the high-order bytes of the destination. Observe that the three byte-movement instructions `movb`, `movsbq`, and `movzbq` differ from each other in subtle ways. Here is an example:

```

1      movabsq $0x0011223344556677, %rax      %rax = 0011223344556677
2      movb   $0xAA, %dl                     %dl  = AA
3      movb   %dl,%al                         %rax = 00112233445566AA
4      movsbq %dl,%rax                       %rax = FFFFFFFF000000AA
5      movzbq %dl,%rax                       %rax = 00000000000000AA

```

In the following discussion, we use hexadecimal notation for all of the values. The first two lines of the code initialize registers `%rax` and `%dl` to `0011223344556677` and `AA`, respectively. The remaining instructions all copy the low-order byte of `%rdx` to the low-order byte of `%rax`. The `movb` instruction (line 3) does not change the other bytes. The `movsbq` instruction (line 4) sets the other 7 bytes to either all ones or all zeros depending on the high-order bit of the source byte. Since hexadecimal `A` represents binary value `1010`, sign extension causes the higher-order bytes to each be set to `FF`. The `movzbq` instruction (line 5) always sets the other 7 bytes to zero.

Practice Problem 3.3 (solution page 326)

Each of the following lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```

movb $0xF, (%ebx)
movl %rax, (%rsp)
movw (%rax), 4(%rsp)
movb %al, %sl
movq %rax, $0x123
movl %eax, %rdx
movb %si, 8(%rbp)

```

3.4.3 Data Movement Example

As an example of code that uses data movement instructions, consider the data exchange routine shown in Figure 3.7, both as C code and as assembly code generated by gcc.

As Figure 3.7(b) shows, function `exchange` is implemented with just three instructions: two data movements (`movq`) plus an instruction to return back to the point from which the function was called (`ret`). We will cover the details of function call and return in Section 3.7. Until then, it suffices to say that arguments are passed to functions in registers. Our annotated assembly code documents these. A function returns a value by storing it in register `%rax`, or in one of the low-order portions of this register.

(a) C code

```

long exchange(long *xp, long y)
{
    long x = *xp;
    *xp = y;
    return x;
}

```

(b) Assembly code

```

    long exchange(long *xp, long y)
    xp in %rdi, y in %rsi
1   exchange:
2       movq    (%rdi), %rax    Get x at xp. Set as return value.
3       movq    %rsi, (%rdi)    Store y at xp.
4       ret                      Return.

```

Figure 3.7 C and assembly code for exchange routine. Registers %rdi and %rsi hold parameters xp and y, respectively.

When the procedure begins execution, procedure parameters xp and y are stored in registers %rdi and %rsi, respectively. Instruction 2 then reads x from memory and stores the value in register %rax, a direct implementation of the operation $x = *xp$ in the C program. Later, register %rax will be used to return a value from the function, and so the return value will be x. Instruction 3 writes y to the memory location designated by xp in register %rdi, a direct implementation of the operation $*xp = y$. This example illustrates how the mov instructions can be used to read from memory to a register (line 2), and to write from a register to memory (line 3).

Two features about this assembly code are worth noting. First, we see that what we call “pointers” in C are simply addresses. Dereferencing a pointer involves copying that pointer into a register, and then using this register in a memory reference. Second, local variables such as x are often kept in registers rather than stored in memory locations. Register access is much faster than memory access.

Practice Problem 3.4 (solution page 326)

Assume variables sp and dp are declared with types

```

src_t  *sp;
dest_t *dp;

```

where src_t and dest_t are data types declared with typedef. We wish to use the appropriate pair of data movement instructions to implement the operation

```
*dp = (dest_t) *sp;
```

New to C? Some examples of pointers

Function `exchange` (Figure 3.7(a)) provides a good illustration of the use of pointers in C. Argument `xp` is a pointer to a long integer, while `y` is a long integer itself. The statement

```
long x = *xp;
```

indicates that we should read the value stored in the location designated by `xp` and store it as a local variable named `x`. This read operation is known as pointer *dereferencing*. The C operator `*` performs pointer dereferencing.

The statement

```
*xp = y;
```

does the reverse—it writes the value of parameter `y` at the location designated by `xp`. This is also a form of pointer dereferencing (and hence the operator `*`), but it indicates a write operation since it is on the left-hand side of the assignment.

The following is an example of `exchange` in action:

```
long a = 4;
long b = exchange(&a, 3);
printf("a = %ld, b = %ld\\verb@\\n", a, b);
```

This code will print

```
a = 3, b = 4
```

The C operator `&` (called the “address of” operator) *creates* a pointer, in this case to the location holding local variable `a`. Function `exchange` overwrites the value stored in `a` with 3 but returns the previous value, 4, as the function value. Observe how by passing a pointer to `exchange`, it could modify data held at some remote location.

Assume that the values of `sp` and `dp` are stored in registers `%rdi` and `%rsi`, respectively. For each entry in the table, show the two instructions that implement the specified data movement. The first instruction in the sequence should read from memory, do the appropriate conversion, and set the appropriate portion of register `%rax`. The second instruction should then write the appropriate portion of `%rax` to memory. In both cases, the portions may be `%rax`, `%eax`, `%ax`, or `%al`, and they may differ from one another.

Recall that when performing a cast that involves both a size change and a change of “signedness” in C, the operation should change the size first (Section 2.2.6).

src_t	dest_t	Instruction
long	long	movq (%rdi), %rax movq %rax, (%rsi)
char	int	

char	unsigned
unsigned char	long
int	char
unsigned	unsigned char
char	short

Practice Problem 3.5 (solution page 327)

You are given the following information. A function with prototype

```
void decode1(long *xp, long *yp, long *zp);
```

is compiled into assembly code, yielding the following:

```
void decode1(long *xp, long *yp, long *zp)
  xp in %rdi, yp in %rsi, zp in %rdx
decode1:
  movq    (%rdi), %r8
  movq    (%rsi), %rcx
  movq    (%rdx), %rax
  movq    %r8, (%rsi)
  movq    %rcx, (%rdx)
  movq    %rax, (%rdi)
  ret
```

Parameters *xp*, *yp*, and *zp* are stored in registers *%rdi*, *%rsi*, and *%rdx*, respectively.

Write C code for *decode1* that will have an effect equivalent to the assembly code shown.

3.4.4 Pushing and Popping Stack Data

The final two data movement operations are used to push data onto and pop data from the program stack, as documented in Figure 3.8. As we will see, the stack plays a vital role in the handling of procedure calls. By way of background, a stack is a data structure where values can be added or deleted, but only according to a “last-in, first-out” discipline. We add data to a stack via a *push* operation and remove it via a *pop* operation, with the property that the value popped will always be the value that was most recently pushed and is still on the stack. A stack can be implemented as an array, where we always insert and remove elements from one

Instruction	Effect	Description
pushq <i>S</i>	$R[\%rsp] \leftarrow R[\%rsp] - 8;$ $M[R[\%rsp]] \leftarrow S$	Push quad word
popq <i>D</i>	$D \leftarrow M[R[\%rsp]];$ $R[\%rsp] \leftarrow R[\%rsp] + 8$	Pop quad word

Figure 3.8 Push and pop instructions.

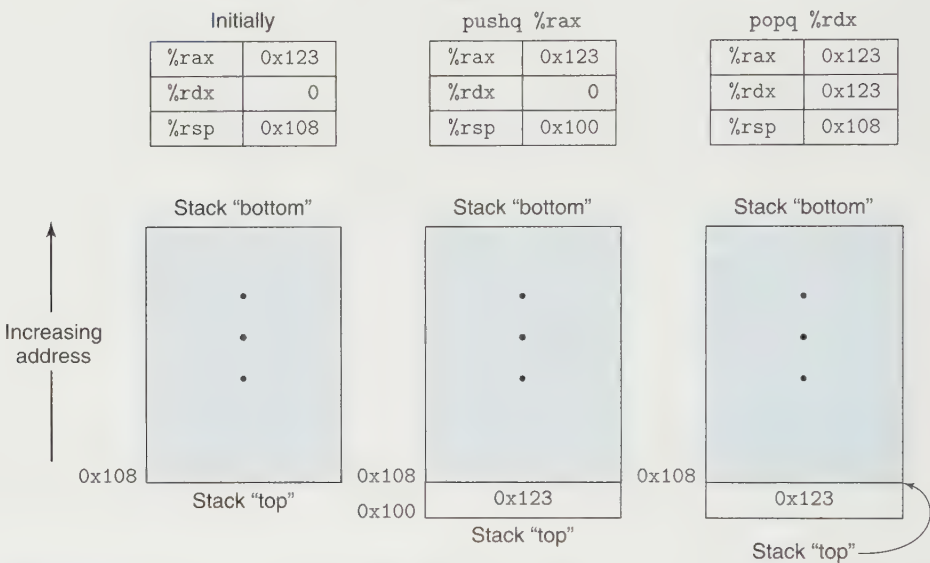


Figure 3.9 Illustration of stack operation. By convention, we draw stacks upside down, so that the “top” of the stack is shown at the bottom. With x86-64, stacks grow toward lower addresses, so pushing involves decrementing the stack pointer (register %rsp) and storing to memory, while popping involves reading from memory and incrementing the stack pointer.

end of the array. This end is called the *top* of the stack. With x86-64, the program stack is stored in some region of memory. As illustrated in Figure 3.9, the stack grows downward such that the top element of the stack has the lowest address of all stack elements. (By convention, we draw stacks upside down, with the stack “top” shown at the bottom of the figure.) The stack pointer %rsp holds the address of the top stack element.

The pushq instruction provides the ability to push data onto the stack, while the popq instruction pops it. Each of these instructions takes a single operand—the data source for pushing and the data destination for popping.

Pushing a quad word value onto the stack involves first decrementing the stack pointer by 8 and then writing the value at the new top-of-stack address.

Therefore, the behavior of the instruction `pushq %rbp` is equivalent to that of the pair of instructions

<code>subq \$8,%rsp</code>	<i>Decrement stack pointer</i>
<code>movq %rbp, (%rsp)</code>	<i>Store %rbp on stack</i>

except that the `pushq` instruction is encoded in the machine code as a single byte, whereas the pair of instructions shown above requires a total of 8 bytes. The first two columns in Figure 3.9 illustrate the effect of executing the instruction `pushq %rax` when `%rsp` is 0x108 and `%rax` is 0x123. First `%rsp` is decremented by 8, giving 0x100, and then 0x123 is stored at memory address 0x100.

Popping a quad word involves reading from the top-of-stack location and then incrementing the stack pointer by 8. Therefore, the instruction `popq %rax` is equivalent to the following pair of instructions:

<code>movq (%rsp), %rax</code>	<i>Read %rax from stack</i>
<code>addq \$8,%rsp</code>	<i>Increment stack pointer</i>

The third column of Figure 3.9 illustrates the effect of executing the instruction `popq %edx` immediately after executing the `pushq`. Value 0x123 is read from memory and written to register `%rdx`. Register `%rsp` is incremented back to 0x108. As shown in the figure, the value 0x123 remains at memory location 0x104 until it is overwritten (e.g., by another push operation). However, the stack top is always considered to be the address indicated by `%rsp`.

Since the stack is contained in the same memory as the program code and other forms of program data, programs can access arbitrary positions within the stack using the standard memory addressing methods. For example, assuming the topmost element of the stack is a quad word, the instruction `movq 8(%rsp), %rdx` will copy the second quad word from the stack to register `%rdx`.

3.5 Arithmetic and Logical Operations

Figure 3.10 lists some of the x86-64 integer and logic operations. Most of the operations are given as instruction classes, as they can have different variants with different operand sizes. (Only `leaq` has no other size variants.) For example, the instruction class `ADD` consists of four addition instructions: `addb`, `addw`, `addl`, and `addq`, adding bytes, words, double words, and quad words, respectively. Indeed, each of the instruction classes shown has instructions for operating on these four different sizes of data. The operations are divided into four groups: load effective address, unary, binary, and shifts. *Binary* operations have two operands, while *unary* operations have one operand. These operands are specified using the same notation as described in Section 3.4.

3.5.1 Load Effective Address

The *load effective address* instruction `leaq` is actually a variant of the `movq` instruction. It has the form of an instruction that reads from memory to a register,

Instruction		Effect	Description
leaq	S, D	$D \leftarrow \&S$	Load effective address
INC	D	$D \leftarrow D + 1$	Increment
DEC	D	$D \leftarrow D - 1$	Decrement
NEG	D	$D \leftarrow -D$	Negate
NOT	D	$D \leftarrow \sim D$	Complement
ADD	S, D	$D \leftarrow D + S$	Add
SUB	S, D	$D \leftarrow D - S$	Subtract
IMUL	S, D	$D \leftarrow D * S$	Multiply
XOR	S, D	$D \leftarrow D \wedge S$	Exclusive-or
OR	S, D	$D \leftarrow D \vee S$	Or
AND	S, D	$D \leftarrow D \& S$	And
SAL	k, D	$D \leftarrow D \ll k$	Left shift
SHL	k, D	$D \leftarrow D \ll k$	Left shift (same as SAL)
SAR	k, D	$D \leftarrow D \gg_A k$	Arithmetic right shift
SHR	k, D	$D \leftarrow D \gg_L k$	Logical right shift

Figure 3.10 Integer arithmetic operations. The load effective address (leaq) instruction is commonly used to perform simple arithmetic. The remaining ones are more standard unary or binary operations. We use the notation $\gg_A$ and $\gg_L$ to denote arithmetic and logical right shift, respectively. Note the nonintuitive ordering of the operands with ATT-format assembly code.

but it does not reference memory at all. Its first operand appears to be a memory reference, but instead of reading from the designated location, the instruction copies the effective address to the destination. We indicate this computation in Figure 3.10 using the C address operator $\&S$. This instruction can be used to generate pointers for later memory references. In addition, it can be used to compactly describe common arithmetic operations. For example, if register `%rdx` contains value x , then the instruction `leaq 7(%rdx,%rdx,4), %rax` will set register `%rax` to $5x + 7$. Compilers often find clever uses of `leaq` that have nothing to do with effective address computations. The destination operand must be a register.

Practice Problem 3.6 (solution page 327)

Suppose register `%rax` holds value x and `%rcx` holds value y . Fill in the table below with formulas indicating the value that will be stored in register `%rdx` for each of the given assembly-code instructions:

Instruction	Result
<code>leaq 6(%rax), %rdx</code>	_____
<code>leaq (%rax,%rcx), %rdx</code>	_____
<code>leaq (%rax,%rcx,4), %rdx</code>	_____
<code>leaq 7(%rax,%rax,8), %rdx</code>	_____

```
leaq 0xA(,%rcx,4), %rdx
leaq 9(%rax,%rcx,2), %rdx
```

As an illustration of the use of `leaq` in compiled code, consider the following C program:

```
long scale(long x, long y, long z) {
    long t = x + 4 * y + 12 * z;
    return t;
}
```

When compiled, the arithmetic operations of the function are implemented by a sequence of three `leaq` functions, as is documented by the comments on the right-hand side:

```
long scale(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
scale:
leaq    (%rdi,%rsi,4), %rax    x + 4*y
leaq    (%rdx,%rdx,2), %rdx    z + 2*z = 3*z
leaq    (%rax,%rdx,4), %rax    (x+4*y) + 4*(3*z) = x + 4*y + 12*z
ret
```

The ability of the `leaq` instruction to perform addition and limited forms of multiplication proves useful when compiling simple arithmetic expressions such as this example.

Practice Problem 3.7 (solution page 328)

Consider the following code, in which we have omitted the expression being computed:

```
long scale2(long x, long y, long z) {
    long t = _____;
    return t;
}
```

Compiling the actual function with gcc yields the following assembly code:

```
long scale2(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
scale2:
leaq    (%rdi,%rdi,4), %rax
leaq    (%rax,%rsi,2), %rax
leaq    (%rax,%rdx,8), %rax
ret
```

Fill in the missing expression in the C code.

3.5.2 Unary and Binary Operations

Operations in the second group are unary operations, with the single operand serving as both source and destination. This operand can be either a register or a memory location. For example, the instruction `incq (%rsp)` causes the 8-byte element on the top of the stack to be incremented. This syntax is reminiscent of the C increment (`++`) and decrement (`--`) operators.

The third group consists of binary operations, where the second operand is used as both a source and a destination. This syntax is reminiscent of the C assignment operators, such as `x -= y`. Observe, however, that the source operand is given first and the destination second. This looks peculiar for noncommutative operations. For example, the instruction `subq %rax,%rdx` decrements register `%rdx` by the value in `%rax`. (It helps to read the instruction as “Subtract `%rax` from `%rdx`.”) The first operand can be either an immediate value, a register, or a memory location. The second can be either a register or a memory location. As with the `mov` instructions, the two operands cannot both be memory locations. Note that when the second operand is a memory location, the processor must read the value from memory, perform the operation, and then write the result back to memory.

Practice Problem 3.8 (solution page 328)

Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	<code>%rax</code>	0x100
0x108	0xAB	<code>%rcx</code>	0x1
0x110	0x13	<code>%rdx</code>	0x3
0x118	0x11		

Fill in the following table showing the effects of the following instructions, in terms of both the register or memory location that will be updated and the resulting value:

Instruction	Destination	Value
<code>addq %rcx, (%rax)</code>		
<code>subq %rdx, 8(%rax)</code>		
<code>imulq \$16, (%rax,%rdx,8)</code>		
<code>incq 16(%rax)</code>		
<code>decq %rcx</code>		
<code>subq %rdx,%rax</code>		

3.5.3 Shift Operations

The final group consists of shift operations, where the shift amount is given first and the value to shift is given second. Both arithmetic and logical right shifts are

possible. The different shift instructions can specify the shift amount either as an immediate value or with the single-byte register %cl. (These instructions are unusual in only allowing this specific register as the operand.) In principle, having a 1-byte shift amount would make it possible to encode shift amounts ranging up to $2^8 - 1 = 255$. With x86-64, a shift instruction operating on data values that are w bits long determines the shift amount from the low-order m bits of register %cl, where $2^m = w$. The higher-order bits are ignored. So, for example, when register %cl has hexadecimal value 0xFF, then instruction salb would shift by 7, while salw would shift by 15, sall would shift by 31, and salq would shift by 63.

As Figure 3.10 indicates, there are two names for the left shift instruction: SAL and SHL. Both have the same effect, filling from the right with zeros. The right shift instructions differ in that SAR performs an arithmetic shift (fill with copies of the sign bit), whereas SHR performs a logical shift (fill with zeros). The destination operand of a shift operation can be either a register or a memory location. We denote the two different right shift operations in Figure 3.10 as $\gg_A$ (arithmetic) and $\gg_L$ (logical).

Practice Problem 3.9 (solution page 328)

Suppose we want to generate assembly code for the following C function:

```
long shift_left4_rightn(long x, long n)
{
    x <<= 4;
    x >>= n;
    return x;
}
```

The code that follows is a portion of the assembly code that performs the actual shifts and leaves the final value in register %rax. Two key instructions have been omitted. Parameters x and n are stored in registers %rdi and %rsi, respectively.

```
long shift_left4_rightn(long x, long n)
x in %rdi, n in %rsi
shift_left4_rightn:
    movq    %rdi, %rax    Get x
                        x <<= 4
    movl    %esi, %ecx    Get n (4 bytes)
                        x >>= n
```

Fill in the missing instructions, following the annotations on the right. The right shift should be performed arithmetically.

(a) C code

```

long arith(long x, long y, long z)
{
    long t1 = x ^ y;
    long t2 = z * 48;
    long t3 = t1 & 0x0F0F0F0F;
    long t4 = t2 - t3;
    return t4;
}

```

(b) Assembly code

```

    long arith(long x, long y, long z)
    x in %rdi, y in %rsi, z in %rdx
1   arith:
2   xorq    %rsi, %rdi           t1 = x ^ y
3   leaq    (%rdx,%rdx,2), %rax   3*z
4   salq    $4, %rax             t2 = 16 * (3*z) = 48*z
5   andl    $252645135, %edi     t3 = t1 & 0x0F0F0F0F
6   subq    %rdi, %rax           Return t2 - t3
7   ret

```

Figure 3.11 C and assembly code for arithmetic function.

3.5.4 Discussion

We see that most of the instructions shown in Figure 3.10 can be used for either unsigned or two's-complement arithmetic. Only right shifting requires instructions that differentiate between signed versus unsigned data. This is one of the features that makes two's-complement arithmetic the preferred way to implement signed integer arithmetic.

Figure 3.11 shows an example of a function that performs arithmetic operations and its translation into assembly code. Arguments *x*, *y*, and *z* are initially stored in registers *%rdi*, *%rsi*, and *%rdx*, respectively. The assembly-code instructions correspond closely with the lines of C source code. Line 2 computes the value of $x \wedge y$. Lines 3 and 4 compute the expression $z * 48$ by a combination of *leaq* and shift instructions. Line 5 computes the AND of *t1* and $0x0F0F0F0F$. The final subtraction is computed by line 6. Since the destination of the subtraction is register *%rax*, this will be the value returned by the function.

In the assembly code of Figure 3.11, the sequence of values in register *%rax* corresponds to program values $3 * z$, $z * 48$, and *t4* (as the return value). In general, compilers generate code that uses individual registers for multiple program values and moves program values among the registers.

Practice Problem 3.10 (solution page 329)

In the following variant of the function of Figure 3.11(a), the expressions have been replaced by blanks:

```

long arith2(long x, long y, long z)
{
    long t1 = _____;
    long t2 = _____;
    long t3 = _____;
    long t4 = _____;
    return t4;
}

```

The portion of the generated assembly code implementing these expressions is as follows:

```

long arith2(long x, long y, long z)
    x in %rdi, y in %rsi, z in %rdx
arith2:
    orq    %rsi, %rdi
    sarq   $3, %rdi
    notq   %rdi
    movq   %rdx, %rax
    subq   %rdi, %rax
    ret

```

Based on this assembly code, fill in the missing portions of the C code.

Practice Problem 3.11 (solution page 329)

It is common to find assembly-code lines of the form

```
xorq %rdx,%rdx
```

in code that was generated from C where no EXCLUSIVE-OR operations were present.

- A. Explain the effect of this particular EXCLUSIVE-OR instruction and what useful operation it implements.
 - B. What would be the more straightforward way to express this operation in assembly code?
 - C. Compare the number of bytes to encode these two different implementations of the same operation.
-

3.5.5 Special Arithmetic Operations

As we saw in Section 2.3, multiplying two 64-bit signed or unsigned integers can yield a product that requires 128 bits to represent. The x86-64 instruction set provides limited support for operations involving 128-bit (16-byte) numbers. Continuing with the naming convention of word (2 bytes), double word (4 bytes), and quad word (8 bytes), Intel refers to a 16-byte quantity as an *oct word*. Figure 3.12

Instruction		Effect	Description
imulq	<i>S</i>	$R[\%rdx]:R[\%rax] \leftarrow S \times R[\%rax]$	Signed full multiply
mulq	<i>S</i>	$R[\%rdx]:R[\%rax] \leftarrow S \times R[\%rax]$	Unsigned full multiply
cqto		$R[\%rdx]:R[\%rax] \leftarrow \text{SignExtend}(R[\%rax])$	Convert to oct word
idivq	<i>S</i>	$R[\%rdx] \leftarrow R[\%rdx]:R[\%rax] \bmod S;$ $R[\%rax] \leftarrow R[\%rdx]:R[\%rax] \div S$	Signed divide
divq	<i>S</i>	$R[\%rdx] \leftarrow R[\%rdx]:R[\%rax] \bmod S;$ $R[\%rax] \leftarrow R[\%rdx]:R[\%rax] \div S$	Unsigned divide

Figure 3.12 Special arithmetic operations. These operations provide full 128-bit multiplication and division, for both signed and unsigned numbers. The pair of registers `%rdx` and `%rax` are viewed as forming a single 128-bit oct word.

describes instructions that support generating the full 128-bit product of two 64-bit numbers, as well as integer division.

The `imulq` instruction has two different forms. One form, shown in Figure 3.10, is as a member of the `IMUL` instruction class. In this form, it serves as a “two-operand” multiply instruction, generating a 64-bit product from two 64-bit operands. It implements the operations $\ast_{64}^u$ and $\ast_{64}^l$ described in Sections 2.3.4 and 2.3.5. (Recall that when truncating the product to 64 bits, both unsigned multiply and two’s-complement multiply have the same bit-level behavior.)

Additionally, the x86-64 instruction set includes two different “one-operand” multiply instructions to compute the full 128-bit product of two 64-bit values—one for unsigned (`mulq`) and one for two’s-complement (`imulq`) multiplication. For both of these instructions, one argument must be in register `%rax`, and the other is given as the instruction source operand. The product is then stored in registers `%rdx` (high-order 64 bits) and `%rax` (low-order 64 bits). Although the name `imulq` is used for two distinct multiplication operations, the assembler can tell which one is intended by counting the number of operands.

As an example, the following C code demonstrates the generation of a 128-bit product of two unsigned 64-bit numbers `x` and `y`:

```
#include <inttypes.h>

typedef unsigned __int128 uint128_t;

void store_uprod(uint128_t *dest, uint64_t x, uint64_t y) {
    *dest = x * (uint128_t) y;
}
```

In this program, we explicitly declare `x` and `y` to be 64-bit numbers, using definitions declared in the file `inttypes.h`, as part of an extension of the C standard. Unfortunately, this standard does not make provisions for 128-bit values. Instead,

we rely on support provided by gcc for 128-bit integers, declared using the name `__int128`. Our code uses a typedef declaration to define data type `uint128_t`, following the naming pattern for other data types found in `inttypes.h`. The code specifies that the resulting product should be stored at the 16 bytes designated by pointer `dest`.

The assembly code generated by gcc for this function is as follows:

```
void store_uprod(uint128_t *dest, uint64_t x, uint64_t y)
dest in %rdi, x in %rsi, y in %rdx
1  store_uprod:
2      movq    %rsi, %rax          Copy x to multiplicand
3      mulq    %rdx                Multiply by y
4      movq    %rax, (%rdi)        Store lower 8 bytes at dest
5      movq    %rdx, 8(%rdi)       Store upper 8 bytes at dest+8
6      ret
```

Observe that storing the product requires two `movq` instructions: one for the low-order 8 bytes (line 4), and one for the high-order 8 bytes (line 5). Since the code is generated for a little-endian machine, the high-order bytes are stored at higher addresses, as indicated by the address specification `8(%rdi)`.

Our earlier table of arithmetic operations (Figure 3.10) does not list any division or modulus operations. These operations are provided by the single-operand divide instructions similar to the single-operand multiply instructions. The signed division instruction `idivl` takes as its dividend the 128-bit quantity in registers `%rdx` (high-order 64 bits) and `%rax` (low-order 64 bits). The divisor is given as the instruction operand. The instruction stores the quotient in register `%rax` and the remainder in register `%rdx`.

For most applications of 64-bit addition, the dividend is given as a 64-bit value. This value should be stored in register `%rax`. The bits of `%rdx` should then be set to either all zeros (unsigned arithmetic) or the sign bit of `%rax` (signed arithmetic). The latter operation can be performed using the instruction `cqto`.² This instruction takes no operands—it implicitly reads the sign bit from `%rax` and copies it across all of `%rdx`.

As an illustration of the implementation of division with x86-64, the following C function computes the quotient and remainder of two 64-bit, signed numbers:

```
void remdiv(long x, long y,
            long *qp, long *rp) {
    long q = x/y;
    long r = x%y;
    *qp = q;
    *rp = r;
}
```

2. This instruction is called `cqo` in the Intel documentation, one of the few cases where the ATT-format name for an instruction does not match the Intel name.

This compiles to the following assembly code:

```

void remdiv(long x, long y, long *qp, long *rp)
  x in %rdi, y in %rsi, qp in %rdx, rp in %rcx
1  remdiv:
2      movq    %rdx, %r8          Copy qp
3      movq    %rdi, %rax         Move x to lower 8 bytes of dividend
4      cqto                                Sign-extend to upper 8 bytes of dividend
5      idivq   %rsi               Divide by y
6      movq    %rax, (%r8)        Store quotient at qp
7      movq    %rdx, (%rcx)       Store remainder at rp
8      ret

```

In this code, argument `rp` must first be saved in a different register (line 2), since argument register `%rdx` is required for the division operation. Lines 3–4 then prepare the dividend by copying and sign-extending `x`. Following the division, the quotient in register `%rax` gets stored at `qp` (line 6), while the remainder in register `%rdx` gets stored at `rp` (line 7).

Unsigned division makes use of the `divq` instruction. Typically, register `%rdx` is set to zero beforehand.

Practice Problem 3.12 (solution page 329)

Consider the following function for computing the quotient and remainder of two unsigned 64-bit numbers:

```

void uremdiv(unsigned long x, unsigned long y,
             unsigned long *qp, unsigned long *rp) {
    unsigned long q = x/y;
    unsigned long r = x%y;
    *qp = q;
    *rp = r;
}

```

Modify the assembly code shown for signed division to implement this function.

3.6 Control

So far, we have only considered the behavior of *straight-line* code, where instructions follow one another in sequence. Some constructs in C, such as conditionals, loops, and switches, require conditional execution, where the sequence of operations that get performed depends on the outcomes of tests applied to the data. Machine code provides two basic low-level mechanisms for implementing conditional behavior: it tests data values and then alters either the control flow or the data flow based on the results of these tests.

Data-dependent control flow is the more general and more common approach for implementing conditional behavior, and so we will examine this first. Normally,

both statements in C and instructions in machine code are executed *sequentially*, in the order they appear in the program. The execution order of a set of machine-code instructions can be altered with a *jump* instruction, indicating that control should pass to some other part of the program, possibly contingent on the result of some test. The compiler must generate instruction sequences that build upon this low-level mechanism to implement the control constructs of C.

In our presentation, we first cover the two ways of implementing conditional operations. We then describe methods for presenting loops and switch statements.

3.6.1 Condition Codes

In addition to the integer registers, the CPU maintains a set of single-bit *condition code* registers describing attributes of the most recent arithmetic or logical operation. These registers can then be tested to perform conditional branches. These condition codes are the most useful:

CF: Carry flag. The most recent operation generated a carry out of the most significant bit. Used to detect overflow for unsigned operations.

ZF: Zero flag. The most recent operation yielded zero.

SF: Sign flag. The most recent operation yielded a negative value.

OF: Overflow flag. The most recent operation caused a two's-complement overflow—either negative or positive.

For example, suppose we used one of the ADD instructions to perform the equivalent of the C assignment `t = a+b`, where variables `a`, `b`, and `t` are integers. Then the condition codes would be set according to the following C expressions:

CF	<code>(unsigned) t < (unsigned) a</code>	Unsigned overflow
ZF	<code>(t == 0)</code>	Zero
SF	<code>(t < 0)</code>	Negative
OF	<code>(a < 0 == b < 0) && (t < 0 != a < 0)</code>	Signed overflow

The `leaq` instruction does not alter any condition codes, since it is intended to be used in address computations. Otherwise, all of the instructions listed in Figure 3.10 cause the condition codes to be set. For the logical operations, such as `XOR`, the carry and overflow flags are set to zero. For the shift operations, the carry flag is set to the last bit shifted out, while the overflow flag is set to zero. For reasons that we will not delve into, the `INC` and `DEC` instructions set the overflow and zero flags, but they leave the carry flag unchanged.

In addition to the setting of condition codes by the instructions of Figure 3.10, there are two instruction classes (having 8-, 16-, 32-, and 64-bit forms) that set condition codes without altering any other registers; these are listed in Figure 3.13. The `CMP` instructions set the condition codes according to the differences of their two operands. They behave in the same way as the `SUB` instructions, except that they set the condition codes without updating their destinations. With AT&T format,

Instruction		Based on	Description
CMP	S_1, S_2	$S_2 - S_1$	Compare
cmpb			Compare byte
cmpw			Compare word
cmpl			Compare double word
cmpq			Compare quad word
TEST	S_1, S_2	$S_1 \& S_2$	Test
testb			Test byte
testw			Test word
testl			Test double word
testq			Test quad word

Figure 3.13 Comparison and test instructions. These instructions set the condition codes without updating any other registers.

the operands are listed in reverse order, making the code difficult to read. These instructions set the zero flag if the two operands are equal. The other flags can be used to determine ordering relations between the two operands. The `TEST` instructions behave in the same manner as the `AND` instructions, except that they set the condition codes without altering their destinations. Typically, the same operand is repeated (e.g., `testq %rax, %rax` to see whether `%rax` is negative, zero, or positive), or one of the operands is a mask indicating which bits should be tested.

3.6.2 Accessing the Condition Codes

Rather than reading the condition codes directly, there are three common ways of using the condition codes: (1) we can set a single byte to 0 or 1 depending on some combination of the condition codes, (2) we can conditionally jump to some other part of the program, or (3) we can conditionally transfer data. For the first case, the instructions described in Figure 3.14 set a single byte to 0 or to 1 depending on some combination of the condition codes. We refer to this entire class of instructions as the `SET` instructions; they differ from one another based on which combinations of condition codes they consider, as indicated by the different suffixes for the instruction names. It is important to recognize that the suffixes for these instructions denote different conditions and not different operand sizes. For example, instructions `setl` and `setb` denote “set less” and “set below,” not “set long word” or “set byte.”

A `SET` instruction has either one of the low-order single-byte register elements (Figure 3.2) or a single-byte memory location as its destination, setting this byte to either 0 or 1. To generate a 32-bit or 64-bit result, we must also clear the high-order bits. A typical instruction sequence to compute the C expression `a < b`, where `a` and `b` are both of type `long`, proceeds as follows:

Instruction	Synonym	Effect	Set condition
sete <i>D</i>	setz	$D \leftarrow ZF$	Equal / zero
setne <i>D</i>	setnz	$D \leftarrow \sim ZF$	Not equal / not zero
sets <i>D</i>		$D \leftarrow SF$	Negative
setns <i>D</i>		$D \leftarrow \sim SF$	Nonnegative
setg <i>D</i>	setnle	$D \leftarrow \sim (SF \wedge OF) \& \sim ZF$	Greater (signed >)
setge <i>D</i>	setnl	$D \leftarrow \sim (SF \wedge OF)$	Greater or equal (signed >=)
setl <i>D</i>	setnge	$D \leftarrow SF \wedge OF$	Less (signed <)
setle <i>D</i>	setng	$D \leftarrow (SF \wedge OF) \mid ZF$	Less or equal (signed <=)
seta <i>D</i>	setnbe	$D \leftarrow \sim CF \& \sim ZF$	Above (unsigned >)
setae <i>D</i>	setnb	$D \leftarrow \sim CF$	Above or equal (unsigned >=)
setb <i>D</i>	setnae	$D \leftarrow CF$	Below (unsigned <)
setbe <i>D</i>	setna	$D \leftarrow CF \mid ZF$	Below or equal (unsigned <=)

Figure 3.14 The SET instructions. Each instruction sets a single byte to 0 or 1 based on some combination of the condition codes. Some instructions have “synonyms,” that is, alternate names for the same machine instruction.

```

int comp(data_t a, data_t b)
a in %rdi, b in %rsi
1  comp:
2      cmpq    %rsi, %rdi    Compare a:b
3      setl    %al           Set low-order byte of %eax to 0 or 1
4      movzbl  %al, %eax     Clear rest of %eax (and rest of %rax)
5      ret

```

Note the comparison order of the `cmpq` instruction (line 2). Although the arguments are listed in the order `%rsi` (b), then `%rdi` (a), the comparison is really between a and b. Recall also, as discussed in Section 3.4.2, that the `movzbl` instruction (line 4) clears not just the high-order 3 bytes of `%eax`, but the upper 4 bytes of the entire register, `%rax`, as well.

For some of the underlying machine instructions, there are multiple possible names, which we list as “synonyms.” For example, both `setg` (for “set greater”) and `setnle` (for “set not less or equal”) refer to the same machine instruction. Compilers and disassemblers make arbitrary choices of which names to use.

Although all arithmetic and logical operations set the condition codes, the descriptions of the different SET instructions apply to the case where a comparison instruction has been executed, setting the condition codes according to the computation $t = a - b$. More specifically, let a , b , and t be the integers represented in two’s-complement form by variables a , b , and t , respectively, and so $t = a -_w b$, where w depends on the sizes associated with a and b .

Consider the `sete`, or “set when equal,” instruction. When $a = b$, we will have $t = 0$, and hence the zero flag indicates equality. Similarly, consider testing for signed comparison with the `setl`, or “set when less,” instruction. When no overflow occurs (indicated by having `OF` set to 0), we will have $a < b$ when $a -_w^t b < 0$, indicated by having `SF` set to 1, and $a \geq b$ when $a -_w^t b \geq 0$, indicated by having `SF` set to 0. On the other hand, when overflow occurs, we will have $a < b$ when $a -_w^t b > 0$ (negative overflow) and $a > b$ when $a -_w^t b < 0$ (positive overflow). We cannot have overflow when $a = b$. Thus, when `OF` is set to 1, we will have $a < b$ if and only if `SF` is set to 0. Combining these cases, the `EXCLUSIVE-OR` of the overflow and sign bits provides a test for whether $a < b$. The other signed comparison tests are based on other combinations of `SF` $\wedge$ `OF` and `ZF`.

For the testing of unsigned comparisons, we now let a and b be the integers represented in unsigned form by variables `a` and `b`. In performing the computation $t = a - b$, the carry flag will be set by the `cmp` instruction when $a - b < 0$, and so the unsigned comparisons use combinations of the carry and zero flags.

It is important to note how machine code does or does not distinguish between signed and unsigned values. Unlike in C, it does not associate a data type with each program value. Instead, it mostly uses the same instructions for the two cases, because many arithmetic operations have the same bit-level behavior for unsigned and two’s-complement arithmetic. Some circumstances require different instructions to handle signed and unsigned operations, such as using different versions of right shifts, division and multiplication instructions, and different combinations of condition codes.

Practice Problem 3.13 (solution page 330)

The C code

```
int comp(data_t a, data_t b) {
    return a COMP b;
}
```

shows a general comparison between arguments `a` and `b`, where `data_t`, the data type of the arguments, is defined (via `typedef`) to be one of the integer data types listed in Figure 3.1 and either signed or unsigned. The comparison `COMP` is defined via `#define`.

Suppose `a` is in some portion of `%rdx` while `b` is in some portion of `%rsi`. For each of the following instruction sequences, determine which data types `data_t` and which comparisons `COMP` could cause the compiler to generate this code. (There can be multiple correct answers; you should list them all.)

- A. `cmpl    %esi, %edi`
 `setl    %al`

- B. `cmpw    %si, %di`
 `setge   %al`

- C. `cmpb    %sil, %dil`
 `setbe    %al`
- D. `cmpq    %rsi, %rdi`
 `setne    %a`
-

Practice Problem 3.14 (solution page 330)

The C code

```
int test(data_t a) {
    return a TEST 0;
}
```

shows a general comparison between argument `a` and 0, where we can set the data type of the argument by declaring `data_t` with a `typedef`, and the nature of the comparison by declaring `TEST` with a `#define` declaration. The following instruction sequences implement the comparison, where `a` is held in some portion of register `%rdi`. For each sequence, determine which data types `data_t` and which comparisons `TEST` could cause the compiler to generate this code. (There can be multiple correct answers; list all correct ones.)

- A. `testq    %rdi, %rdi`
 `setge    %al`
- B. `testw    %di, %di`
 `sete    %al`
- C. `testb    %dil, %dil`
 `seta    %al`
- D. `testl    %edi, %edi`
 `setle    %al`
-

3.6.3 Jump Instructions

Under normal execution, instructions follow each other in the order they are listed. A *jump* instruction can cause the execution to switch to a completely new position in the program. These jump destinations are generally indicated in assembly code by a *label*. Consider the following (very contrived) assembly-code sequence:

<code>movq \$0,%rax</code>	<i>Set %rax to 0</i>
<code>jmp .L1</code>	<i>Goto .L1</i>
<code>movq (%rax),%rdx</code>	<i>Null pointer dereference (skipped)</i>
<code>.L1:</code>	
<code>popq %rdx</code>	<i>Jump target</i>

Instruction	Synonym	Jump condition	Description
<code>jmp Label</code>		1	Direct jump
<code>jmp *Operand</code>		1	Indirect jump
<code>jz Label</code>	<code>jz</code>	ZF	Equal / zero
<code>jne Label</code>	<code>jnz</code>	$(\sim \text{ZF})$	Not equal / not zero
<code>js Label</code>		SF	Negative
<code>jns Label</code>		$\sim \text{SF}$	Nonnegative
<code>jg Label</code>	<code>jnl</code>	$\sim (\text{SF} \wedge \text{OF}) \ \& \ \sim \text{ZF}$	Greater (signed >)
<code>jge Label</code>	<code>jnl</code>	$\sim (\text{SF} \wedge \text{OF})$	Greater or equal (signed >=)
<code>jl Label</code>	<code>jnge</code>	$\text{SF} \wedge \text{OF}$	Less (signed <)
<code>jle Label</code>	<code>jng</code>	$(\text{SF} \wedge \text{OF}) \mid \text{ZF}$	Less or equal (signed <=)
<code>ja Label</code>	<code>jnb</code>	$\sim \text{CF} \ \& \ \sim \text{ZF}$	Above (unsigned >)
<code>jae Label</code>	<code>jnb</code>	$\sim \text{CF}$	Above or equal (unsigned >=)
<code>jb Label</code>	<code>jnae</code>	CF	Below (unsigned <)
<code>jbe Label</code>	<code>jna</code>	$\text{CF} \mid \text{ZF}$	Below or equal (unsigned <=)

Figure 3.15 The jump instructions. These instructions jump to a labeled destination when the jump condition holds. Some instructions have “synonyms,” alternate names for the same machine instruction.

The instruction `jmp .L1` will cause the program to skip over the `movq` instruction and instead resume execution with the `popq` instruction. In generating the object-code file, the assembler determines the addresses of all labeled instructions and encodes the *jump targets* (the addresses of the destination instructions) as part of the jump instructions.

Figure 3.15 shows the different jump instructions. The `jmp` instruction jumps unconditionally. It can be either a *direct* jump, where the jump target is encoded as part of the instruction, or an *indirect* jump, where the jump target is read from a register or a memory location. Direct jumps are written in assembly code by giving a label as the jump target, for example, the label `.L1` in the code shown. Indirect jumps are written using ‘*’ followed by an operand specifier using one of the memory operand formats described in Figure 3.3. As examples, the instruction

```
jmp %rax
```

uses the value in register `%rax` as the jump target, and the instruction

```
jmp *(%rax)
```

reads the jump target from memory, using the value in `%rax` as the read address.

The remaining jump instructions in the table are *conditional*—they either jump or continue executing at the next instruction in the code sequence, depending on some combination of the condition codes. The names of these instructions

and the conditions under which they jump match those of the `SET` instructions (see Figure 3.14). As with the `SET` instructions, some of the underlying machine instructions have multiple names. Conditional jumps can only be direct.

3.6.4 Jump Instruction Encodings

For the most part, we will not concern ourselves with the detailed format of machine code. On the other hand, understanding how the targets of jump instructions are encoded will become important when we study linking in Chapter 7. In addition, it helps when interpreting the output of a disassembler. In assembly code, jump targets are written using symbolic labels. The assembler, and later the linker, generate the proper encodings of the jump targets. There are several different encodings for jumps, but some of the most commonly used ones are *PC relative*. That is, they encode the difference between the address of the target instruction and the address of the instruction immediately following the jump. These offsets can be encoded using 1, 2, or 4 bytes. A second encoding method is to give an “absolute” address, using 4 bytes to directly specify the target. The assembler and linker select the appropriate encodings of the jump destinations.

As an example of PC-relative addressing, the following assembly code for a function was generated by compiling a file `branch.c`. It contains two jumps: the `jmp` instruction on line 2 jumps forward to a higher address, while the `jg` instruction on line 7 jumps back to a lower one.

```

1    movq    %rdi, %rax
2    jmp     .L2
3    .L3:
4    sarq    %rax
5    .L2:
6    testq   %rax, %rax
7    jg      .L3
8    rep; ret
```

The disassembled version of the `.o` format generated by the assembler is as follows:

1	0:	48 89 f8	mov	%rdi,%rax
2	3:	eb 03	jmp	8 <loop+0x8>
3	5:	48 d1 f8	sar	%rax
4	8:	48 85 c0	test	%rax,%rax
5	b:	7f f8	jg	5 <loop+0x5>
6	d:	f3 c3	repz retq	

In the annotations on the right generated by the disassembler, the jump targets are indicated as `0x8` for the jump instruction on line 2 and `0x5` for the jump instruction on line 5 (the disassembler lists all numbers in hexadecimal). Looking at the byte encodings of the instructions, however, we see that the target of the first jump instruction is encoded (in the second byte) as `0x03`. Adding this to `0x5`, the

Aside What do the instructions `rep` and `repz` do?

Line 8 of the assembly code shown on page 207 contains the instruction combination `rep; ret`. These are rendered in the disassembled code (line 6) as `repz retq`. One can infer that `repz` is a synonym for `rep`, just as `retq` is a synonym for `ret`. Looking at the Intel and AMD documentation for the `rep` instruction, we find that it is normally used to implement a repeating string operation [3, 51]. It seems completely inappropriate here. The answer to this puzzle can be seen in AMD's guidelines to compiler writers [1]. They recommend using the combination of `rep` followed by `ret` to avoid making the `ret` instruction the destination of a conditional jump instruction. Without the `rep` instruction, the `jg` instruction (line 7 of the assembly code) would proceed to the `ret` instruction when the branch is not taken. According to AMD, their processors cannot properly predict the destination of a `ret` instruction when it is reached from a jump instruction. The `rep` instruction serves as a form of no-operation here, and so inserting it as the jump destination does not change behavior of the code, except to make it faster on AMD processors. We can safely ignore any `rep` or `repz` instruction we see in the rest of the code presented in this book.

address of the following instruction, we get jump target address `0x8`, the address of the instruction on line 4.

Similarly, the target of the second jump instruction is encoded as `0xf8` (decimal -8) using a single-byte two's-complement representation. Adding this to `0xd` (decimal 13), the address of the instruction on line 6, we get `0x5`, the address of the instruction on line 3.

As these examples illustrate, the value of the program counter when performing PC-relative addressing is the address of the instruction following the jump, not that of the jump itself. This convention dates back to early implementations, when the processor would update the program counter as its first step in executing an instruction.

The following shows the disassembled version of the program after linking:

1	4004d0:	48 89 f8	<code>mov %rdi,%rax</code>
2	4004d3:	eb 03	<code>jmp 4004d8 <loop+0x8></code>
3	4004d5:	48 d1 f8	<code>sar %rax</code>
4	4004d8:	48 85 c0	<code>test %rax,%rax</code>
5	4004db:	7f f8	<code>jg 4004d5 <loop+0x5></code>
6	4004dd:	f3 c3	<code>repz retq</code>

The instructions have been relocated to different addresses, but the encodings of the jump targets in lines 2 and 5 remain unchanged. By using a PC-relative encoding of the jump targets, the instructions can be compactly encoded (requiring just 2 bytes), and the object code can be shifted to different positions in memory without alteration.

Practice Problem 3.15 (solution page 330)

In the following excerpts from a disassembled binary, some of the information has been replaced by X's. Answer the following questions about these instructions.

- A. What is the target of the `je` instruction below? (You do not need to know anything about the `callq` instruction here.)

```
4003fa: 74 02          je      XXXXXX
4003fc: ff d0          callq   *%rax
```

- B. What is the target of the `je` instruction below?

```
40042f: 74 f4          je      XXXXXX
400431: 5d             pop     %rbp
```

- C. What is the address of the `ja` and `pop` instructions?

```
XXXXXX: 77 02          ja      400547
XXXXXX: 5d             pop     %rbp
```

- D. In the code that follows, the jump target is encoded in PC-relative form as a 4-byte two's-complement number. The bytes are listed from least significant to most, reflecting the little-endian byte ordering of x86-64. What is the address of the jump target?

```
4005e8: e9 73 ff ff    jmpq   XXXXXXXX
4005ed: 90             nop
```

The jump instructions provide a means to implement conditional execution (`if`), as well as several different loop constructs.

3.6.5 Implementing Conditional Branches with Conditional Control

The most general way to translate conditional expressions and statements from C into machine code is to use combinations of conditional and unconditional jumps. (As an alternative, we will see in Section 3.6.6 that some conditionals can be implemented by conditional transfers of data rather than control.) For example, Figure 3.16(a) shows the C code for a function that computes the absolute value of the difference of two numbers.³ The function also has a side effect of incrementing one of two counters, encoded as global variables `lt_cnt` and `ge_cnt`. Gcc generates the assembly code shown as Figure 3.16(c). Our rendition of the machine code into C is shown as the function `gotodiff_se` (Figure 3.16(b)). It uses the `goto` statement in C, which is similar to the unconditional jump of

3. Actually, it can return a negative value if one of the subtractions overflows. Our interest here is to demonstrate machine code, not to implement robust code.

(a) Original C code	(b) Equivalent goto version
<code>long lt_cnt = 0;</code>	1 <code>long gotodiff_se(long x, long y)</code>
<code>long ge_cnt = 0;</code>	2 <code>{</code>
<code>long absdiff_se(long x, long y)</code>	3 <code> long result;</code>
<code>{</code>	4 <code> if (x >= y)</code>
<code>long result;</code>	5 <code> goto x_ge_y;</code>
<code>if (x < y) {</code>	6 <code> lt_cnt++;</code>
<code>lt_cnt++;</code>	7 <code> result = y - x;</code>
<code>result = y - x;</code>	8 <code> return result;</code>
<code>}</code>	9 <code> x_ge_y:</code>
<code>else {</code>	10 <code> ge_cnt++;</code>
<code>ge_cnt++;</code>	11 <code> result = x - y;</code>
<code>result = x - y;</code>	12 <code> return result;</code>
<code>}</code>	13 <code>}</code>
<code>return result;</code>	
<code>}</code>	
 (c) Generated assembly code	
<code>long absdiff_se(long x, long y)</code>	
<code>x in %rdi, y in %rsi</code>	
1 <code>absdiff_se:</code>	
2 <code> cmpq %rsi, %rdi</code>	<i>Compare x:y</i>
3 <code> jge .L2</code>	<i>If >= goto x_ge_y</i>
4 <code> addq \$1, lt_cnt(%rip)</code>	<i>lt_cnt++</i>
5 <code> movq %rsi, %rax</code>	
6 <code> subq %rdi, %rax</code>	<i>result = y - x</i>
7 <code> ret</code>	<i>Return</i>
8 <code> .L2:</code>	<i>x_ge_y:</i>
9 <code> addq \$1, ge_cnt(%rip)</code>	<i>ge_cnt++</i>
10 <code> movq %rdi, %rax</code>	
11 <code> subq %rsi, %rax</code>	<i>result = x - y</i>
12 <code> ret</code>	<i>Return</i>

Figure 3.16 Compilation of conditional statements. (a) C procedure `absdiff_se` contains an if-else statement. The generated assembly code is shown (c), along with (b) a C procedure `gotodiff_se` that mimics the control flow of the assembly code.

assembly code. Using goto statements is generally considered a bad programming style, since their use can make code very difficult to read and debug. We use them in our presentation as a way to construct C programs that describe the control flow of machine code. We call this style of programming “goto code.”

In the goto code (Figure 3.16(b)), the statement `goto x_ge_y` on line 5 causes a jump to the label `x_ge_y` (since it occurs when $x \geq y$) on line 9. Continuing the

Aside Describing machine code with C code

Figure 3.16 shows an example of how we will demonstrate the translation of C language control constructs into machine code. The figure contains an example C function (a) and an annotated version of the assembly code generated by gcc (c). It also contains a version in C that closely matches the structure of the assembly code (b). Although these versions were generated in the sequence (a), (c), and (b), we recommend that you read them in the order (a), (b), and then (c). That is, the C rendition of the machine code will help you understand the key points, and this can guide you in understanding the actual assembly code.

execution from this point, it completes the computations specified by the `else` portion of function `absdiff_se` and returns. On the other hand, if the test `x >= y` fails, the program procedure will carry out the steps specified by the `if` portion of `absdiff_se` and return.

The assembly-code implementation (Figure 3.16(c)) first compares the two operands (line 2), setting the condition codes. If the comparison result indicates that `x` is greater than or equal to `y`, it then jumps to a block of code starting at line 8 that increments global variable `ge_cnt`, computes `x-y` as the return value, and returns. Otherwise, it continues with the execution of code beginning at line 4 that increments global variable `lt_cnt`, computes `y-x` as the return value, and returns. We can see, then, that the control flow of the assembly code generated for `absdiff_se` closely follows the `goto` code of `gotodiff_se`.

The general form of an if-else statement in C is given by the template

```
if (test-expr)
    then-statement
else
    else-statement
```

where *test-expr* is an integer expression that evaluates either to zero (interpreted as meaning “false”) or to a nonzero value (interpreted as meaning “true”). Only one of the two branch statements (*then-statement* or *else-statement*) is executed.

For this general form, the assembly implementation typically adheres to the following form, where we use C syntax to describe the control flow:

```
t = test-expr;
if (!t)
    goto false;
then-statement
goto done;
false:
    else-statement
done:
```

That is, the compiler generates separate blocks of code for *then-statement* and *else-statement*. It inserts conditional and unconditional branches to make sure the correct block is executed.

Practice Problem 3.16 (solution page 331)

When given the C code

```
void cond(long a, long *p)
{
    if (p && a > *p)
        *p = a;
}
```

gcc generates the following assembly code:

```
void cond(long a, long *p)
a in %rdi, p in %rsi
cond:
    testq    %rsi, %rsi
    je       .L1
    cmpq     %rdi, (%rsi)
    jge      .L1
    movq     %rdi, (%rsi)
.L1:
    rep; ret
```

- A. Write a goto version in C that performs the same computation and mimics the control flow of the assembly code, in the style shown in Figure 3.16(b). You might find it helpful to first annotate the assembly code as we have done in our examples.
- B. Explain why the assembly code contains two conditional branches, even though the C code has only one if statement.

Practice Problem 3.17 (solution page 331)

An alternate rule for translating if statements into goto code is as follows:

```
t = test-expr;
if (t)
    goto true;
else-statement
goto done;
true:
    then-statement
done:
```

- A. Rewrite the goto version of `absdiff_se` based on this alternate rule.
- B. Can you think of any reasons for choosing one rule over the other?
-

Practice Problem 3.18 (solution page 332)

Starting with C code of the form

```
long test(long x, long y, long z) {
    long val = _____;
    if (_____) {
        if (_____)
            val = _____;
        else
            val = _____;
    } else if (_____)
        val = _____;
    return val;
}
```

gcc generates the following assembly code:

```
long test(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
test:
    leaq    (%rdi,%rsi), %rax
    addq    %rdx, %rax
    cmpq    $-3, %rdi
    jge     .L2
    cmpq    %rdx, %rsi
    jge     .L3
    movq    %rdi, %rax
    imulq    %rsi, %rax
    ret
.L3:
    movq    %rsi, %rax
    imulq    %rdx, %rax
    ret
.L2:
    cmpq    $2, %rdi
    jle     .L4
    movq    %rdi, %rax
    imulq    %rdx, %rax
.L4:
    rep; ret
```

Fill in the missing expressions in the C code.

3.6.6 Implementing Conditional Branches with Conditional Moves

The conventional way to implement conditional operations is through a conditional transfer of *control*, where the program follows one execution path when a condition holds and another when it does not. This mechanism is simple and general, but it can be very inefficient on modern processors.

An alternate strategy is through a conditional transfer of *data*. This approach computes both outcomes of a conditional operation and then selects one based on whether or not the condition holds. This strategy makes sense only in restricted cases, but it can then be implemented by a simple *conditional move* instruction that is better matched to the performance characteristics of modern processors. Here, we examine this strategy and its implementation with x86-64.

Figure 3.17(a) shows an example of code that can be compiled using a conditional move. The function computes the absolute value of its arguments *x* and *y*, as did our earlier example (Figure 3.16). Whereas the earlier example had side effects in the branches, modifying the value of either *lt_cnt* or *ge_cnt*, this version simply computes the value to be returned by the function.

(a) Original C code

```
long absdiff(long x, long y)
{
    long result;
    if (x < y)
        result = y - x;
    else
        result = x - y;
    return result;
}
```

(b) Implementation using conditional assignment

```
1  long cmovdiff(long x, long y)
2  {
3      long rval = y-x;
4      long eval = x-y;
5      long ntest = x >= y;
6      /* Line below requires
7         single instruction: */
8      if (ntest) rval = eval;
9      return rval;
10 }
```

(c) Generated assembly code

```
long absdiff(long x, long y)
x in %rdi, y in %rsi
1  absdiff:
2      movq    %rsi, %rax
3      subq    %rdi, %rax      rval = y-x
4      movq    %rdi, %rdx
5      subq    %rsi, %rdx      eval = x-y
6      cmpq    %rsi, %rdi      Compare x:y
7      cmovge  %rdx, %rax      If >=, rval = eval
8      ret                                Return tval
```

Figure 3.17 Compilation of conditional statements using conditional assignment. (a) C function *absdiff* contains a conditional expression. The generated assembly code is shown (c), along with (b) a C function *cmovdiff* that mimics the operation of the assembly code.

For this function, gcc generates the assembly code shown in Figure 3.17(c), having an approximate form shown by the C function `cmovdiff` shown in Figure 3.17(b). Studying the C version, we can see that it computes both $y-x$ and $x-y$, naming these `rval` and `eval`, respectively. It then tests whether x is greater than or equal to y , and if so, copies `eval` to `rval` before returning `rval`. The assembly code in Figure 3.17(c) follows the same logic. The key is that the single `cmovge` instruction (line 7) of the assembly code implements the conditional assignment (line 8) of `cmovdiff`. It will transfer the data from the source register to the destination, only if the `cmpq` instruction of line 6 indicates that one value is greater than or equal to the other (as indicated by the suffix `ge`).

To understand why code based on conditional data transfers can outperform code based on conditional control transfers (as in Figure 3.16), we must understand something about how modern processors operate. As we will see in Chapters 4 and 5, processors achieve high performance through *pipelining*, where an instruction is processed via a sequence of stages, each performing one small portion of the required operations (e.g., fetching the instruction from memory, determining the instruction type, reading from memory, performing an arithmetic operation, writing to memory, and updating the program counter). This approach achieves high performance by overlapping the steps of the successive instructions, such as fetching one instruction while performing the arithmetic operations for a previous instruction. To do this requires being able to determine the sequence of instructions to be executed well ahead of time in order to keep the pipeline full of instructions to be executed. When the machine encounters a conditional jump (referred to as a “branch”), it cannot determine which way the branch will go until it has evaluated the branch condition. Processors employ sophisticated *branch prediction logic* to try to guess whether or not each jump instruction will be followed. As long as it can guess reliably (modern microprocessor designs try to achieve success rates on the order of 90%), the instruction pipeline will be kept full of instructions. Mispredicting a jump, on the other hand, requires that the processor discard much of the work it has already done on future instructions and then begin filling the pipeline with instructions starting at the correct location. As we will see, such a misprediction can incur a serious penalty, say, 15–30 clock cycles of wasted effort, causing a serious degradation of program performance.

As an example, we ran timings of the `absdiff` function on an Intel Haswell processor using both methods of implementing the conditional operation. In a typical application, the outcome of the test $x < y$ is highly unpredictable, and so even the most sophisticated branch prediction hardware will guess correctly only around 50% of the time. In addition, the computations performed in each of the two code sequences require only a single clock cycle. As a consequence, the branch misprediction penalty dominates the performance of this function. For x86-64 code with conditional jumps, we found that the function requires around 8 clock cycles per call when the branching pattern is easily predictable, and around 17.50 clock cycles per call when the branching pattern is random. From this, we can infer that the branch misprediction penalty is around 19 clock cycles. That means time required by the function ranges between around 8 and 27 cycles, depending on whether or not the branch is predicted correctly.

Aside How did you determine this penalty?

Assume the probability of misprediction is p , the time to execute the code without misprediction is T_{OK} , and the misprediction penalty is T_{MP} . Then the average time to execute the code as a function of p is $T_{avg}(p) = (1 - p)T_{OK} + p(T_{OK} + T_{MP}) = T_{OK} + pT_{MP}$. We are given T_{OK} and T_{ran} , the average time when $p = 0.5$, and we want to determine T_{MP} . Substituting into the equation, we get $T_{ran} = T_{avg}(0.5) = T_{OK} + 0.5T_{MP}$, and therefore $T_{MP} = 2(T_{ran} - T_{OK})$. So, for $T_{OK} = 8$ and $T_{ran} = 17.5$, we get $T_{MP} = 19$.

On the other hand, the code compiled using conditional moves requires around 8 clock cycles regardless of the data being tested. The flow of control does not depend on data, and this makes it easier for the processor to keep its pipeline full.

Practice Problem 3.19 (solution page 332)

Running on an older processor model, our code required around 16 cycles when the branching pattern was highly predictable, and around 31 cycles when the pattern was random.

- A. What is the approximate miss penalty?
- B. How many cycles would the function require when the branch is mispredicted?

Figure 3.18 illustrates some of the conditional move instructions available with x86-64. Each of these instructions has two operands: a source register or memory location S , and a destination register R . As with the different `SET` (Section 3.6.2) and `jump` (Section 3.6.3) instructions, the outcome of these instructions depends on the values of the condition codes. The source value is read from either memory or the source register, but it is copied to the destination only if the specified condition holds.

The source and destination values can be 16, 32, or 64 bits long. Single-byte conditional moves are not supported. Unlike the unconditional instructions, where the operand length is explicitly encoded in the instruction name (e.g., `movw` and `movl`), the assembler can infer the operand length of a conditional move instruction from the name of the destination register, and so the same instruction name can be used for all operand lengths.

Unlike conditional jumps, the processor can execute conditional move instructions without having to predict the outcome of the test. The processor simply reads the source value (possibly from memory), checks the condition code, and then either updates the destination register or keeps it the same. We will explore the implementation of conditional moves in Chapter 4.

To understand how conditional operations can be implemented via conditional data transfers, consider the following general form of conditional expression and assignment:

Instruction	Synonym	Move condition	Description	
cmovz	S, R	cmovz	ZF	Equal / zero
cmovne	S, R	cmovnz	~ZF	Not equal / not zero
cmovs	S, R		SF	Negative
cmovns	S, R		~SF	Nonnegative
cmovg	S, R	cmovnle	~(SF ^ OF) & ~ZF	Greater (signed >)
cmovge	S, R	cmovnl	~(SF ^ OF)	Greater or equal (signed >=)
cmovl	S, R	cmovnge	SF ^ OF	Less (signed <)
cmovle	S, R	cmovng	(SF ^ OF) ZF	Less or equal (signed <=)
cmova	S, R	cmovnbe	~CF & ~ZF	Above (unsigned >)
cmovae	S, R	cmovnb	~CF	Above or equal (Unsigned >=)
cmovb	S, R	cmovnae	CF	Below (unsigned <)
cmovbe	S, R	cmovna	CF ZF	Below or equal (unsigned <=)

Figure 3.18 The conditional move instructions. These instructions copy the source value *S* to its destination *R* when the move condition holds. Some instructions have “synonyms,” alternate names for the same machine instruction.

$v = \text{test-expr} ? \text{then-expr} : \text{else-expr};$

The standard way to compile this expression using conditional control transfer would have the following form:

```

    if (!test-expr)
        goto false;
    v = then-expr;
    goto done;
false:
    v = else-expr;
done:

```

This code contains two code sequences—one evaluating *then-expr* and one evaluating *else-expr*. A combination of conditional and unconditional jumps is used to ensure that just one of the sequences is evaluated.

For the code based on a conditional move, both the *then-expr* and the *else-expr* are evaluated, with the final value chosen based on the evaluation *test-expr*. This can be described by the following abstract code:

```

v  = then-expr;
ve = else-expr;
t  = test-expr;
if (!t) v = ve;

```

The final statement in this sequence is implemented with a conditional move—value *ve* is copied to *v* only if test condition *t* does not hold.

Not all conditional expressions can be compiled using conditional moves. Most significantly, the abstract code we have shown evaluates both *then-expr* and *else-expr* regardless of the test outcome. If one of those two expressions could possibly generate an error condition or a side effect, this could lead to invalid behavior. Such is the case for our earlier example (Figure 3.16). Indeed, we put the side effects into this example specifically to force gcc to implement this function using conditional transfers.

As a second illustration, consider the following C function:

```
long cread(long *xp) {
    return (xp ? *xp : 0);
}
```

At first, this seems like a good candidate to compile using a conditional move to set the result to zero when the pointer is null, as shown in the following assembly code:

```
long cread(long *xp)
Invalid implementation of function cread
xp in register %rdi
1  cread:
2      movq    (%rdi), %rax    v = *xp
3      testq   %rdi, %rdi     Test x
4      movl    $0, %edx       Set ve = 0
5      cmovbe  %rdx, %rax     If x==0, v = ve
6      ret                    Return v
```

This implementation is invalid, however, since the dereferencing of *xp* by the *movq* instruction (line 2) occurs even when the test fails, causing a null pointer dereferencing error. Instead, this code must be compiled using branching code.

Using conditional moves also does not always improve code efficiency. For example, if either the *then-expr* or the *else-expr* evaluation requires a significant computation, then this effort is wasted when the corresponding condition does not hold. Compilers must take into account the relative performance of wasted computation versus the potential for performance penalty due to branch misprediction. In truth, they do not really have enough information to make this decision reliably; for example, they do not know how well the branches will follow predictable patterns. Our experiments with gcc indicate that it only uses conditional moves when the two expressions can be computed very easily, for example, with single add instructions. In our experience, gcc uses conditional control transfers even in many cases where the cost of branch misprediction would exceed even more complex computations.

Overall, then, we see that conditional data transfers offer an alternative strategy to conditional control transfers for implementing conditional operations. They can only be used in restricted cases, but these cases are fairly common and provide a much better match to the operation of modern processors.

Practice Problem 3.20 (solution page 333)

In the following C function, we have left the definition of operation OP incomplete:

```
#define OP _____ /* Unknown operator */

long arith(long x) {
    return x OP 8;
}
```

When compiled, GCC generates the following assembly code:

```
long arith(long x)
x in %rdi
arith:
    leaq    7(%rdi), %rax
    testq   %rdi, %rdi
    cmovns  %rdi, %rax
    sarq    $3, %rax
    ret
```

- A. What operation is OP?
 - B. Annotate the code to explain how it works.
-

Practice Problem 3.21 (solution page 333)

Starting with C code of the form

```
long test(long x, long y) {
    long val = _____;
    if (_____) {
        if (_____)
            val = _____;
        else
            val = _____;
    } else if (_____)
        val = _____;
    return val;
}
```

GCC generates the following assembly code:

```
long test(long x, long y)
x in %rdi, y in %rsi
test:
    leaq    0(,%rdi,8), %rax
    testq   %rsi, %rsi
    jle     .L2
```



```

    movq    %rsi, %rax
    subq    %rdi, %rax
    movq    %rdi, %rdx
    andq    %rsi, %rdx
    cmpq    %rsi, %rdi
    cmovge  %rdx, %rax
    ret
.L2:
    addq    %rsi, %rdi
    cmpq    $-2, %rsi
    cmovle  %rdi, %rax
    ret

```

Fill in the missing expressions in the C code.

3.6.7 Loops

C provides several looping constructs—namely, `do-while`, `while`, and `for`. No corresponding instructions exist in machine code. Instead, combinations of conditional tests and jumps are used to implement the effect of loops. Gcc and other compilers generate loop code based on the two basic loop patterns. We will study the translation of loops as a progression, starting with `do-while` and then working toward ones with more complex implementations, covering both patterns.

Do-While Loops

The general form of a `do-while` statement is as follows:

```

do
    body-statement
while (test-expr);

```

The effect of the loop is to repeatedly execute *body-statement*, evaluate *test-expr*, and continue the loop if the evaluation result is nonzero. Observe that *body-statement* is executed at least once.

This general form can be translated into conditionals and `goto` statements as follows:

```

loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;

```

That is, on each iteration the program evaluates the body statement and then the test expression. If the test succeeds, the program goes back for another iteration.

(a) C code

```
long fact_do(long n)
{
    long result = 1;
    do {
        result *= n;
        n = n-1;
    } while (n > 1);
    return result;
}
```

(b) Equivalent goto version

```
long fact_do_goto(long n)
{
    long result = 1;
loop:
    result *= n;
    n = n-1;
    if (n > 1)
        goto loop;
    return result;
}
```

(c) Corresponding assembly-language code

```
long fact_do(long n)
n in %rdi
1 fact_do:
2     movl    $1, %eax      Set result = 1
3     .L2:      loop:
4     imulq   %rdi, %rax     Compute result *= n
5     subq    $1, %rdi       Decrement n
6     cmpq    $1, %rdi       Compare n:1
7     jg      .L2            If >, goto loop
8     rep; ret              Return
```

Figure 3.19 Code for do-while version of factorial program. A conditional jump causes the program to loop.

As an example, Figure 3.19(a) shows an implementation of a routine to compute the factorial of its argument, written $n!$, with a do-while loop. This function only computes the proper value for $n > 0$.

Practice Problem 3.22 (solution page 333)

- A. What is the maximum value of n for which we can represent $n!$ with a 32-bit int?
- B. What about for a 64-bit long?

The goto code shown in Figure 3.19(b) shows how the loop gets turned into a lower-level combination of tests and conditional jumps. Following the initialization of `result`, the program begins looping. First it executes the body of the loop, consisting here of updates to variables `result` and `n`. It then tests whether $n > 1$, and, if so, it jumps back to the beginning of the loop. Figure 3.19(c) shows

Aside Reverse engineering loops

A key to understanding how the generated assembly code relates to the original source code is to find a mapping between program values and registers. This task was simple enough for the loop of Figure 3.19, but it can be much more challenging for more complex programs. The C compiler will often rearrange the computations, so that some variables in the C code have no counterpart in the machine code, and new values are introduced into the machine code that do not exist in the source code. Moreover, it will often try to minimize register usage by mapping multiple program values onto a single register.

The process we described for `fact_do` works as a general strategy for reverse engineering loops. Look at how registers are initialized before the loop, updated and tested within the loop, and used after the loop. Each of these provides a clue that can be combined to solve a puzzle. Be prepared for surprising transformations, some of which are clearly cases where the compiler was able to optimize the code, and others where it is hard to explain why the compiler chose that particular strategy.

the assembly code from which the `goto` code was generated. The conditional jump instruction `jg` (line 7) is the key instruction in implementing a loop. It determines whether to continue iterating or to exit the loop.

Reverse engineering assembly code, such as that of Figure 3.19(c), requires determining which registers are used for which program values. In this case, the mapping is fairly simple to determine: We know that n will be passed to the function in register `%rdi`. We can see register `%rax` getting initialized to 1 (line 2). (Recall that, although the instruction has `%eax` as its destination, it will also set the upper 4 bytes of `%rax` to 0.) We can see that this register is also updated by multiplication on line 4. Furthermore, since `%rax` is used to return the function value, it is often chosen to hold program values that are returned. We therefore conclude that `%rax` corresponds to program value `result`.

Practice Problem 3.23 (solution page 334)

For the C code

```
long dw_loop(long x) {
    long y = x*x;
    long *p = &x;
    long n = 2*x;
    do {
        x += y;
        (*p)++;
        n--;
    } while (n > 0);
    return x;
}
```

gcc generates the following assembly code:

```

long dw_loop(long x)
x initially in %rdi
1  dw_loop:
2      movq    %rdi, %rax
3      movq    %rdi, %rcx
4      imulq   %rdi, %rcx
5      leaq    (%rdi,%rdi), %rdx
6  .L2:
7      leaq    1(%rcx,%rax), %rax
8      subq    $1, %rdx
9      testq   %rdx, %rdx
10     jg      .L2
11     rep; ret

```

- A. Which registers are used to hold program values x , y , and n ?
 - B. How has the compiler eliminated the need for pointer variable p and the pointer dereferencing implied by the expression $(*p)++$?
 - C. Add annotations to the assembly code describing the operation of the program, similar to those shown in Figure 3.19(c).
-

While Loops

The general form of a while statement is as follows:

```

while (test-expr)
    body-statement

```

It differs from do-while in that *test-expr* is evaluated and the loop is potentially terminated before the first execution of *body-statement*. There are a number of ways to translate a while loop into machine code, two of which are used in code generated by gcc. Both use the same loop structure as we saw for do-while loops but differ in how to implement the initial test.

The first translation method, which we refer to as *jump to middle*, performs the initial test by performing an unconditional jump to the test at the end of the loop. It can be expressed by the following template for translating from the general while loop form to goto code:

```

goto test;
loop:
    body-statement
test:
     $t = \text{test-expr};$ 
    if (t)
        goto loop;

```

As an example, Figure 3.20(a) shows an implementation of the factorial function using a while loop. This function correctly computes $0! = 1$. The adjacent

(a) C code

```
long fact_while(long n)
{
    long result = 1;
    while (n > 1) {
        result *= n;
        n = n-1;
    }
    return result;
}
```

(b) Equivalent goto version

```
long fact_while_jm_goto(long n)
{
    long result = 1;
    goto test;
loop:
    result *= n;
    n = n-1;
test:
    if (n > 1)
        goto loop;
    return result;
}
```

(c) Corresponding assembly-language code

```
long fact_while(long n)
n in %rdi
fact_while:
    movl    $1, %eax        Set result = 1
    jmp     .L5              Goto test
.L6:
    imulq   %rdi, %rax       Compute result *= n
    subq    $1, %rdi         Decrement n
.L5:
    cmpq    $1, %rdi         Compare n:1
    jg      .L6              If >, goto loop
    rep; ret                 Return
```

Figure 3.20 C and assembly code for while version of factorial using jump-to-middle translation. The C function `fact_while_jm_goto` illustrates the operation of the assembly-code version.

function `fact_while_jm_goto` (Figure 3.20(b)) is a C rendition of the assembly code generated by gcc when optimization is specified with the command-line option `-Og`. Comparing the goto code generated for `fact_while` (Figure 3.20(b)) to that for `fact_do` (Figure 3.19(b)), we see that they are very similar, except that the statement `goto test` before the loop causes the program to first perform the test of `n` before modifying the values of `result` or `n`. The bottom portion of the figure (Figure 3.20(c)) shows the actual assembly code generated.

Practice Problem 3.24 (solution page 335)

For C code having the general form

```
long loop_while(long a, long b)
{
```



```

long result = _____;
while (_____) {
    result = _____;
    a = _____;
}
return result;
}

```

gcc, run with command-line option `-Og`, produces the following code:

```

long loop_while(long a, long b)
a in %rdi, b in %rsi
1  loop_while:
2      movl    $1, %eax
3      jmp     .L2
4  .L3:
5      leaq    (%rdi,%rsi), %rdx
6      imulq   %rdx, %rax
7      addq    $1, %rdi
8  .L2:
9      cmpq    %rsi, %rdi
10     jl      .L3
11     rep; ret

```

We can see that the compiler used a jump-to-middle translation, using the `jmp` instruction on line 3 to jump to the test starting with label `.L2`. Fill in the missing parts of the C code.

The second translation method, which we refer to as *guarded do*, first transforms the code into a do-while loop by using a conditional branch to skip over the loop if the initial test fails. Gcc follows this strategy when compiling with higher levels of optimization, for example, with command-line option `-O1`. This method can be expressed by the following template for translating from the general while loop form to a do-while loop:

```

t = test-expr;
if (!t)
    goto done;
do
    body-statement
    while (test-expr);
done:

```

This, in turn, can be transformed into `goto` code as

```

t = test-expr;
if (!t)
    goto done;

```

```

loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
done:

```

Using this implementation strategy, the compiler can often optimize the initial test, for example, determining that the test condition will always hold.

As an example, Figure 3.21 shows the same C code for a factorial function as in Figure 3.20, but demonstrates the compilation that occurs when gcc is given command-line option `-O1`. Figure 3.21(c) shows the actual assembly code generated, while Figure 3.21(b) renders this assembly code in a more readable C representation. Referring to this goto code, we see that the loop will be skipped if $n \leq 1$, for the initial value of n . The loop itself has the same general structure as that generated for the do-while version of the function (Figure 3.19). One interesting feature, however, is that the loop test (line 9 of the assembly code) has been changed from $n > 1$ in the original C code to $n \neq 1$. The compiler has determined that the loop can only be entered when $n > 1$, and that decrementing n will result in either $n > 1$ or $n = 1$. Therefore, the test $n \neq 1$ will be equivalent to the test $n \leq 1$.

Practice Problem 3.25 (solution page 335)

For C code having the general form

```

long loop_while2(long a, long b)
{
    long result = _____;
    while (_____) {
        result = _____;
        b = _____;
    }
    return result;
}

```

gcc, run with command-line option `-O1`, produces the following code:

```

a in %rdi, b in %rsi.
1  loop_while2:
2      testq    %rsi, %rsi
3      jle      .L8
4      movq     %rsi, %rax
5  .L7:
6      imulq    %rdi, %rax
7      subq     %rdi, %rsi
8      testq    %rsi, %rsi

```

(a) C code

```

long fact_while(long n)
{
    long result = 1;
    while (n > 1) {
        result *= n;
        n = n-1;
    }
    return result;
}

```

(b) Equivalent goto version

```

long fact_while_gd_goto(long n)
{
    long result = 1;
    if (n <= 1)
        goto done;
loop:
    result *= n;
    n = n-1;
    if (n != 1)
        goto loop;
done:
    return result;
}

```

(c) Corresponding assembly-language code

```

    long fact_while(long n)
    n in %rdi
1   fact_while:
2       cmpq    $1, %rdi        Compare n:1
3       jle     .L7             If <=, goto done
4       movl    $1, %eax        Set result = 1
5   .L6:                        loop:
6       imulq   %rdi, %rax       Compute result *= n
7       subq    $1, %rdi        Decrement n
8       cmpq    $1, %rdi        Compare n:1
9       jne     .L6             If !=, goto loop
10      rep; ret                Return
11   .L7:                        done:
12      movl    $1, %eax        Compute result = 1
13      ret                    Return

```

Figure 3.21 C and assembly code for while version of factorial using guarded-do translation. The fact_while_gd_goto function illustrates the operation of the assembly-code version.

```

9       jg      .L7
10      rep; ret
11   .L8:
12      movq    %rsi, %rax
13      ret

```

We can see that the compiler used a guarded-do translation, using the `jle` instruction on line 3 to skip over the loop code when the initial test fails. Fill in the missing parts of the C code. Note that the control structure in the assembly

code does not exactly match what would be obtained by a direct translation of the C code according to our translation rules. In particular, it has two different `ret` instructions (lines 10 and 13). However, you can fill out the missing portions of the C code in a way that it will have equivalent behavior to the assembly code.

Practice Problem 3.26 (solution page 336)

A function `fun_a` has the following overall structure:

```
long fun_a(unsigned long x) {
    long val = 0;
    while ( ... ) {
        :
        :
    }
    return ...;
}
```

The gcc C compiler generates the following assembly code:

```

    long fun_a(unsigned long x)
    x in %rdi
1   fun_a:
2       movl    $0, %eax
3       jmp     .L5
4   .L6:
5       xorq    %rdi, %rax
6       shrq    %rdi           Shift right by 1
7   .L5:
8       testq   %rdi, %rdi
9       jne     .L6
10      andl    $1, %eax
11      ret
```

Reverse engineer the operation of this code and then do the following:

- A. Determine what loop translation method was used.
 - B. Use the assembly-code version to fill in the missing parts of the C code.
 - C. Describe in English what this function computes.
-

For Loops

The general form of a `for` loop is as follows:

```
for (init-expr; test-expr; update-expr)
    body-statement
```

The C language standard states (with one exception, highlighted in Problem 3.29) that the behavior of such a loop is identical to the following code using a `while` loop:

```
init-expr;
while (test-expr) {
    body-statement
    update-expr;
}
```

The program first evaluates the initialization expression *init-expr*. It enters a loop where it first evaluates the test condition *test-expr*, exiting if the test fails, then executes the body of the loop *body-statement*, and finally evaluates the update expression *update-expr*.

The code generated by gcc for a `for` loop then follows one of our two translation strategies for `while` loops, depending on the optimization level. That is, the jump-to-middle strategy yields the `goto` code

```
init-expr;
goto test;
loop:
    body-statement
    update-expr;
test:
    t = test-expr;
    if (t)
        goto loop;
```

while the guarded-do strategy yields

```
init-expr;
t = test-expr;
if (!t)
    goto done;
loop:
    body-statement
    update-expr;
    t = test-expr;
    if (t)
        goto loop;
done:
```

As examples, consider a factorial function written with a `for` loop:

```
long fact_for(long n)
{
    long i;
    long result = 1;
```



```

    for (i = 2; i <= n; i++)
        result *= i;
    return result;
}

```

As shown, the natural way of writing a factorial function with a `for` loop is to multiply factors from 2 up to n , and so this function is quite different from the code we showed using either a `while` or a `do-while` loop.

We can identify the different components of the `for` loop in this code as follows:

<i>init-expr</i>	<code>i = 2</code>
<i>test-expr</i>	<code>i <= n</code>
<i>update-expr</i>	<code>i++</code>
<i>body-statement</i>	<code>result *= i;</code>

Substituting these components into the template we have shown to transform a `for` loop into a `while` loop yields the following:

```

long fact_for_while(long n)
{
    long i = 2;
    long result = 1;
    while (i <= n) {
        result *= i;
        i++;
    }
    return result;
}

```

Applying the jump-to-middle transformation to the `while` loop then yields the following version in `goto` code:

```

long fact_for_jm_goto(long n)
{
    long i = 2;
    long result = 1;
    goto test;
loop:
    result *= i;
    i++;
test:
    if (i <= n)
        goto loop;
    return result;
}

```

Indeed, a close examination of the assembly code produced by gcc with command-line option `-Og` closely follows this template:

```

long fact_for(long n)
n in %rdi
fact_for:
    movl    $1, %eax        Set result = 1
    movl    $2, %edx        Set i = 2
    jmp     .L8              Goto test
.L9:                loop:
    imulq   %rdx, %rax       Compute result *= i
    addq    $1, %rdx         Increment i
.L8:                test:
    cmpq    %rdi, %rdx       Compare i:n
    jle     .L9              If <=, goto loop
    rep; ret                Return

```

Practice Problem 3.27 (solution page 336)

Write goto code for `fact_for` based on first transforming it to a while loop and then applying the guarded-do transformation.

We see from this presentation that all three forms of loops in C—do-while, while, and for—can be translated by a simple strategy, generating code that contains one or more conditional branches. Conditional transfer of control provides the basic mechanism for translating loops into machine code.

Practice Problem 3.28 (solution page 336)

A function `fun_b` has the following overall structure:

```

long fun_b(unsigned long x) {
    long val = 0;
    long i;
    for ( ... ; ... ; ... ) {
        :
        :
    }
    return val;
}

```

The gcc C compiler generates the following assembly code:

```

long fun_b(unsigned long x)
x in %rdi
1  fun_b:
2  movl    $64, %edx

```

```

3     movl    $0, %eax
4     .L10:
5     movq    %rdi, %rcx
6     andl    $1, %ecx
7     addq    %rax, %rax
8     orq     %rcx, %rax
9     shrq    %rdi           Shift right by 1
10    subq    $1, %rdx
11    jne     .L10
12    rep; ret

```

Reverse engineer the operation of this code and then do the following:

- A. Use the assembly-code version to fill in the missing parts of the C code.
 - B. Explain why there is neither an initial test before the loop nor an initial jump to the test portion of the loop.
 - C. Describe in English what this function computes.
-

Practice Problem 3.29 (solution page 337)

Executing a `continue` statement in C causes the program to jump to the end of the current loop iteration. The stated rule for translating a `for` loop into a `while` loop needs some refinement when dealing with `continue` statements. For example, consider the following code:

```

/* Example of for loop containing a continue statement */
/* Sum even numbers between 0 and 9 */
long sum = 0;
long i;
for (i = 0; i < 10; i++) {
    if (i & 1)
        continue;
    sum += i;
}

```

- A. What would we get if we naively applied our rule for translating the `for` loop into a `while` loop? What would be wrong with this code?
 - B. How could you replace the `continue` statement with a `goto` statement to ensure that the `while` loop correctly duplicates the behavior of the `for` loop?
-

3.6.8 Switch Statements

A `switch` statement provides a multiway branching capability based on the value of an integer index. They are particularly useful when dealing with tests where

there can be a large number of possible outcomes. Not only do they make the C code more readable, but they also allow an efficient implementation using a data structure called a *jump table*. A jump table is an array where entry i is the address of a code segment implementing the action the program should take when the switch index equals i . The code performs an array reference into the jump table using the switch index to determine the target for a jump instruction. The advantage of using a jump table over a long sequence of if-else statements is that the time taken to perform the switch is independent of the number of switch cases. Gcc selects the method of translating a switch statement based on the number of cases and the sparsity of the case values. Jump tables are used when there are a number of cases (e.g., four or more) and they span a small range of values.

Figure 3.22(a) shows an example of a C switch statement. This example has a number of interesting features, including case labels that do not span a contiguous range (there are no labels for cases 101 and 105), cases with multiple labels (cases 104 and 106), and cases that *fall through* to other cases (case 102) because the code for the case does not end with a break statement.

Figure 3.23 shows the assembly code generated when compiling `switch_eg`. The behavior of this code is shown in C as the procedure `switch_eg_impl` in Figure 3.22(b). This code makes use of support provided by gcc for jump tables, as an extension to the C language. The array `jt` contains seven entries, each of which is the address of a block of code. These locations are defined by labels in the code and indicated in the entries in `jt` by code pointers, consisting of the labels prefixed by `&&`. (Recall that the operator `&` creates a pointer for a data value. In making this extension, the authors of gcc created a new operator `&&` to create a pointer for a code location.) We recommend that you study the C procedure `switch_eg_impl` and how it relates to the assembly-code version.

Our original C code has cases for values 100, 102–104, and 106, but the switch variable `n` can be an arbitrary integer. The compiler first shifts the range to between 0 and 6 by subtracting 100 from `n`, creating a new program variable that we call `index` in our C version. It further simplifies the branching possibilities by treating `index` as an *unsigned* value, making use of the fact that negative numbers in a two's-complement representation map to large positive numbers in an unsigned representation. It can therefore test whether `index` is outside of the range 0–6 by testing whether it is greater than 6. In the C and assembly code, there are five distinct locations to jump to, based on the value of `index`. These are `loc_A` (identified in the assembly code as `.L3`), `loc_B` (`.L5`), `loc_C` (`.L6`), `loc_D` (`.L7`), and `loc_def` (`.L8`), where the latter is the destination for the default case. Each of these labels identifies a block of code implementing one of the case branches. In both the C and the assembly code, the program compares `index` to 6 and jumps to the code for the default case if it is greater.

The key step in executing a switch statement is to access a code location through the jump table. This occurs in line 16 in the C code, with a `goto` statement that references the jump table `jt`. This *computed goto* is supported by gcc as an extension to the C language. In our assembly-code version, a similar operation occurs on line 5, where the `jmp` instruction's operand is prefixed with `*`, indicating

(a) Switch statement

```

void switch_eg(long x, long n,
               long *dest)
{
    long val = x;

    switch (n) {

    case 100:
        val *= 13;
        break;

    case 102:
        val += 10;
        /* Fall through */

    case 103:
        val += 11;
        break;

    case 104:
    case 106:
        val *= val;
        break;

    default:
        val = 0;
    }

    *dest = val;
}

```

(b) Translation into extended C

```

1  void switch_eg_impl(long x, long n,
2                      long *dest)
3  {
4      /* Table of code pointers */
5      static void *jt[7] = {
6          &&loc_A, &&loc_def, &&loc_B,
7          &&loc_C, &&loc_D, &&loc_def,
8          &&loc_D
9      };
10     unsigned long index = n - 100;
11     long val;
12
13     if (index > 6)
14         goto loc_def;
15     /* Multiway branch */
16     goto *jt[index];
17
18 loc_A:    /* Case 100 */
19     val = x * 13;
20     goto done;
21 loc_B:    /* Case 102 */
22     x = x + 10;
23     /* Fall through */
24 loc_C:    /* Case 103 */
25     val = x + 11;
26     goto done;
27 loc_D:    /* Cases 104, 106 */
28     val = x * x;
29     goto done;
30 loc_def:  /* Default case */
31     val = 0;
32 done:
33     *dest = val;
34 }

```

Figure 3.22 Example switch statement and its translation into extended C. The translation shows the structure of jump table `jt` and how it is accessed. Such tables are supported by GCC as an extension to the C language.

an indirect jump, and the operand specifies a memory location indexed by register `%eax`, which holds the value of `index`. (We will see in Section 3.8 how array references are translated into machine code.)

Our C code declares the jump table as an array of seven elements, each of which is a pointer to a code location. These elements span values 0–6 of


```

void switch_eg(long x, long n, long *dest)
x in %rdi, n in %rsi, dest in %rdx
1  switch_eg:
2      subq    $100, %rsi           Compute index = n-100
3      cmpq    $6, %rsi           Compare index:6
4      ja      .L8                 If >, goto loc_def
5      jmp     *.L4(,%rsi,8)       Goto *jg[index]
6  .L3:                               loc_A:
7      leaq    (%rdi,%rdi,2), %rax  3*x
8      leaq    (%rdi,%rax,4), %rdi  val = 13*x
9      jmp     .L2                 Goto done
10 .L5:                               loc_B:
11      addq    $10, %rdi          x = x + 10
12 .L6:                               loc_C:
13      addq    $11, %rdi          val = x + 11
14      jmp     .L2                 Goto done
15 .L7:                               loc_D:
16      imulq   %rdi, %rdi         val = x * x
17      jmp     .L2                 Goto done
18 .L8:                               loc_def:
19      movl    $0, %edi           val = 0
20 .L2:                               done:
21      movq    %rdi, (%rdx)       *dest = val
22      ret                               Return

```

Figure 3.23 Assembly code for switch statement example in Figure 3.22.

index, corresponding to values 100–106 of *n*. Observe that the jump table handles duplicate cases by simply having the same code label (*loc_D*) for entries 4 and 6, and it handles missing cases by using the label for the default case (*loc_def*) as entries 1 and 5.

In the assembly code, the jump table is indicated by the following declarations, to which we have added comments:

```

1      .section      .rodata
2      .align 8      Align address to multiple of 8
3  .L4:
4      .quad .L3      Case 100: loc_A
5      .quad .L8      Case 101: loc_def
6      .quad .L5      Case 102: loc_B
7      .quad .L6      Case 103: loc_C
8      .quad .L7      Case 104: loc_D
9      .quad .L8      Case 105: loc_def
10     .quad .L7      Case 106: loc_D

```

These declarations state that within the segment of the object-code file called `.rodata` (for “read-only data”), there should be a sequence of seven “quad” (8-byte) words, where the value of each word is given by the instruction address associated with the indicated assembly-code labels (e.g., `.L3`). Label `.L4` marks the start of this allocation. The address associated with this label serves as the base for the indirect jump (line 5).

The different code blocks (C labels `loc_A` through `loc_D` and `loc_def`) implement the different branches of the `switch` statement. Most of them simply compute a value for `val` and then go to the end of the function. Similarly, the assembly-code blocks compute a value for register `%rdi` and jump to the position indicated by label `.L2` at the end of the function. Only the code for case label 102 does not follow this pattern, to account for the way the code for this case falls through to the block with label 103 in the original C code. This is handled in the assembly-code block starting with label `.L5`, by omitting the `jmp` instruction at the end of the block, so that the code continues execution of the next block. Similarly, the C version `switch_eg_impl` has no `goto` statement at the end of the block starting with label `loc_B`.

Examining all of this code requires careful study, but the key point is to see that the use of a jump table allows a very efficient way to implement a multiway branch. In our case, the program could branch to five distinct locations with a single jump table reference. Even if we had a `switch` statement with hundreds of cases, they could be handled by a single jump table access.

Practice Problem 3.30 (solution page 338)

In the C function that follows, we have omitted the body of the `switch` statement. In the C code, the case labels did not span a contiguous range, and some cases had multiple labels.

```
void switch2(long x, long *dest) {
    long val = 0;
    switch (x) {
        :   Body of switch statement omitted
        :
    }
    *dest = val;
}
```

In compiling the function, GCC generates the assembly code that follows for the initial part of the procedure, with variable `x` in `%rdi`:

```
void switch2(long x, long *dest)
x in %rdi
1  switch2:
2      addq    $1, %rdi
3      cmpq    $8, %rdi
4      ja     .L2
5      jmp     *.L4(,%rdi,8)
```

It generates the following code for the jump table:

```

1  .L4:
2  .quad  .L9
3  .quad  .L5
4  .quad  .L6
5  .quad  .L7
6  .quad  .L2
7  .quad  .L7
8  .quad  .L8
9  .quad  .L2
10 .quad  .L5

```

Based on this information, answer the following questions:

- A. What were the values of the case labels in the switch statement?
- B. What cases had multiple labels in the C code?

Practice Problem 3.31 (solution page 338)

For a C function switcher with the general structure

```

void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
        case _____:      /* Case A */
            c = _____;
            /* Fall through */
        case _____:      /* Case B */
            val = _____;
            break;
        case _____:      /* Case C */
        case _____:      /* Case D */
            val = _____;
            break;
        case _____:      /* Case E */
            val = _____;
            break;
        default:
            val = _____;
    }
    *dest = val;
}

```

gcc generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

(a) Code

```

void switcher(long a, long b, long c, long *dest)
a in %rsi, b in %rdi, c in %rdx, d in %rcx
1  switcher:
2      cmpq    $7, %rdi
3      ja      .L2
4      jmp     *.L4(,%rdi,8)
5      .section      .rodata
6      .L7:
7      xorq    $15, %rsi
8      movq    %rsi, %rdx
9      .L3:
10     leaq    112(%rdx), %rdi
11     jmp     .L6
12     .L5:
13     leaq    (%rdx,%rsi), %rdi
14     salq    $2, %rdi
15     jmp     .L6
16     .L2:
17     movq    %rsi, %rdi
18     .L6:
19     movq    %rdi, (%rcx)
20     ret

```

(b) Jump table

```

1      .L4:
2      .quad   .L3
3      .quad   .L2
4      .quad   .L5
5      .quad   .L2
6      .quad   .L6
7      .quad   .L7
8      .quad   .L2
9      .quad   .L5

```

Figure 3.24 Assembly code and jump table for Problem 3.31.

3.7 Procedures

Procedures are a key abstraction in software. They provide a way to package code that implements some functionality with a designated set of arguments and an optional return value. This function can then be invoked from different points in a program. Well-designed software uses procedures as an abstraction mechanism, hiding the detailed implementation of some action while providing a clear and concise interface definition of what values will be computed and what effects the procedure will have on the program state. Procedures come in many guises

in different programming languages—functions, methods, subroutines, handlers, and so on—but they all share a general set of features.

There are many different attributes that must be handled when providing machine-level support for procedures. For discussion purposes, suppose procedure P calls procedure Q, and Q then executes and returns back to P. These actions involve one or more of the following mechanisms:

Passing control. The program counter must be set to the starting address of the code for Q upon entry and then set to the instruction in P following the call to Q upon return.

Passing data. P must be able to provide one or more parameters to Q, and Q must be able to return a value back to P.

Allocating and deallocating memory. Q may need to allocate space for local variables when it begins and then free that storage before it returns.

The x86-64 implementation of procedures involves a combination of special instructions and a set of conventions on how to use the machine resources, such as the registers and the program memory. Great effort has been made to minimize the overhead involved in invoking a procedure. As a consequence, it follows what can be seen as a minimalist strategy, implementing only as much of the above set of mechanisms as is required for each particular procedure. In our presentation, we build up the different mechanisms step by step, first describing control, then data passing, and, finally, memory management.

3.7.1 The Run-Time Stack

A key feature of the procedure-calling mechanism of C, and of most other languages, is that it can make use of the last-in, first-out memory management discipline provided by a stack data structure. Using our example of procedure P calling procedure Q, we can see that while Q is executing, P, along with any of the procedures in the chain of calls up to P, is temporarily suspended. While Q is running, only it will need the ability to allocate new storage for its local variables or to set up a call to another procedure. On the other hand, when Q returns, any local storage it has allocated can be freed. Therefore, a program can manage the storage required by its procedures using a stack, where the stack and the program registers store the information required for passing control and data, and for allocating memory. As P calls Q, control and data information are added to the end of the stack. This information gets deallocated when P returns.

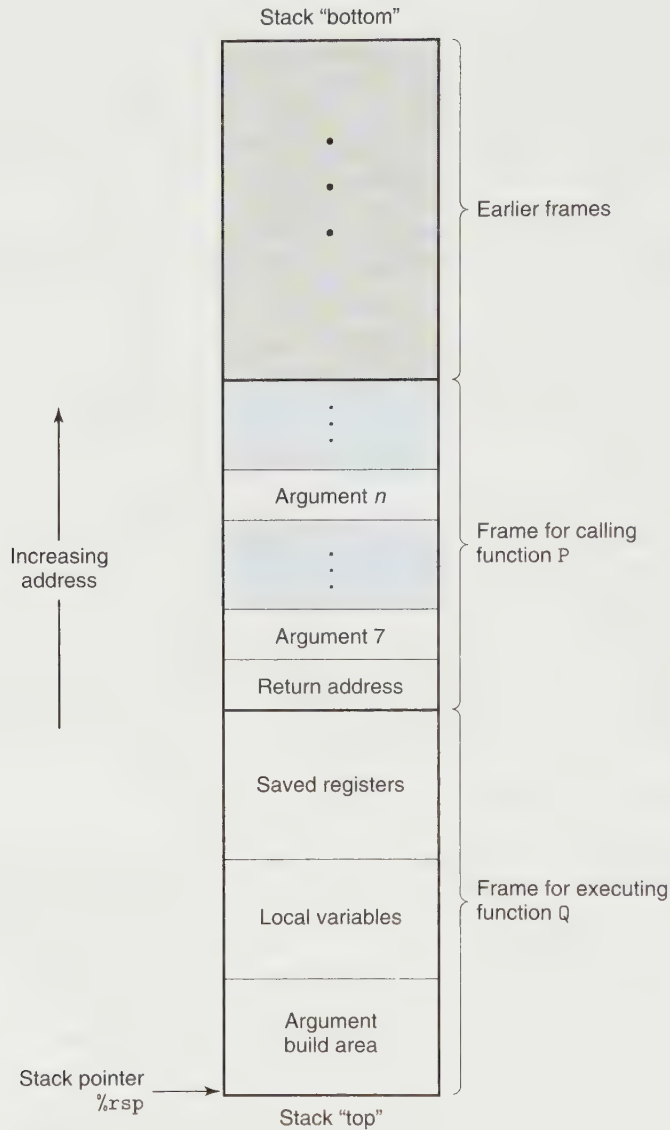
As described in Section 3.4.4, the x86-64 stack grows toward lower addresses and the stack pointer `%rsp` points to the top element of the stack. Data can be stored on and retrieved from the stack using the `pushq` and `popq` instructions. Space for data with no specified initial value can be allocated on the stack by simply decrementing the stack pointer by an appropriate amount. Similarly, space can be deallocated by incrementing the stack pointer.

When an x86-64 procedure requires storage beyond what it can hold in registers, it allocates space on the stack. This region is referred to as the procedure's

Figure 3.25

General stack frame

structure. The stack can be used for passing arguments, for storing return information, for saving registers, and for local storage. Portions may be omitted when not needed.



stack frame. Figure 3.25 shows the overall structure of the run-time stack, including its partitioning into stack frames, in its most general form. The frame for the currently executing procedure is always at the top of the stack. When procedure P calls procedure Q, it will push the *return address* onto the stack, indicating where within P the program should resume execution once Q returns. We consider the return address to be part of P's stack frame, since it holds state relevant to P. The code for Q allocates the space required for its stack frame by extending the current stack boundary. Within that space, it can save the values of registers, allocate

space for local variables, and set up arguments for the procedures it calls. The stack frames for most procedures are of fixed size, allocated at the beginning of the procedure. Some procedures, however, require variable-size frames. This issue is discussed in Section 3.10.5. Procedure P can pass up to six integral values (i.e., pointers and integers) on the stack, but if Q requires more arguments, these can be stored by P within its stack frame prior to the call.

In the interest of space and time efficiency, x86-64 procedures allocate only the portions of stack frames they require. For example, many procedures have six or fewer arguments, and so all of their parameters can be passed in registers. Thus, parts of the stack frame diagrammed in Figure 3.25 may be omitted. Indeed, many functions do not even require a stack frame. This occurs when all of the local variables can be held in registers and the function does not call any other functions (sometimes referred to as a *leaf procedure*, in reference to the tree structure of procedure calls). For example, none of the functions we have examined thus far required stack frames.

3.7.2 Control Transfer

Passing control from function P to function Q involves simply setting the program counter (PC) to the starting address of the code for Q.^① However, when it later comes time for Q to return, the processor must have some record of the code location where it should resume the execution of P. This information is recorded in x86-64 machines by invoking procedure Q with the instruction `call Q`. This instruction pushes an address A onto the stack and sets the PC to the beginning of Q. The pushed address A is referred to as the *return address* and is computed as the address of the instruction immediately following the `call` instruction. The counterpart instruction `ret` pops an address A off the stack and sets the PC to A.

The general forms of the `call` and `ret` instructions are described as follows:

Instruction	Description
<code>call Label</code>	Procedure call
<code>call *Operand</code>	Procedure call
<code>ret</code>	Return from call

(These instructions are referred to as `callq` and `retq` in the disassembly outputs generated by the program `OBJDUMP`. The added suffix ‘q’ simply emphasizes that these are x86-64 versions of `call` and `return` instructions, not IA32. In x86-64 assembly code, both versions can be used interchangeably.)

The `call` instruction has a target indicating the address of the instruction where the called procedure starts. Like jumps, a `call` can be either direct or indirect. In assembly code, the target of a direct call is given as a label, while the target of an indirect call is given by ‘*’ followed by an operand specifier using one of the formats described in Figure 3.3.

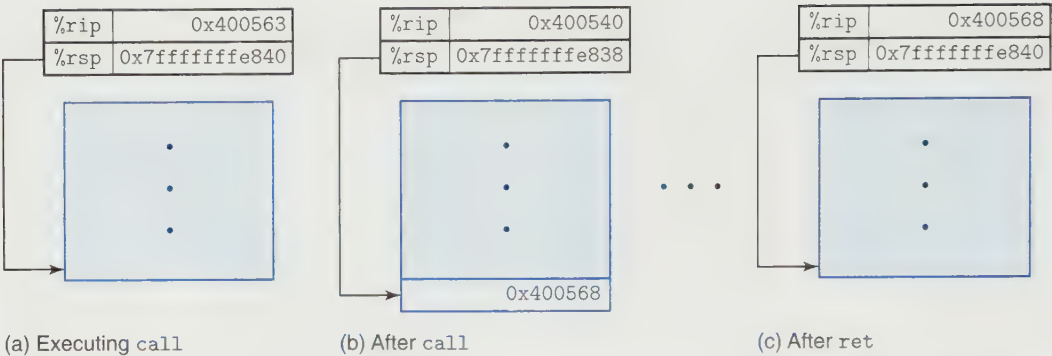


Figure 3.26 Illustration of `call` and `ret` functions. The `call` instruction transfers control to the start of a function, while the `ret` instruction returns back to the instruction following the `call`.

Figure 3.26 illustrates the execution of the `call` and `ret` instructions for the `multstore` and `main` functions introduced in Section 3.2.2. The following are excerpts of the disassembled code for the two functions:

```
Beginning of function multstore
1  0000000000400540 <multstore>:
2    400540: 53                      push  %rbx
3    400541: 48 89 d3                mov   %rdx,%rbx
...
Return from function multstore
4    40054d: c3                      retq
...
Call to multstore from main
5    400563: e8 d8 ff ff ff         callq 400540 <multstore>
6    400568: 48 8b 54 24 08         mov   0x8(%rsp),%rdx
```

In this code, we can see that the `call` instruction with address `0x400563` in `main` calls function `multstore`. This status is shown in Figure 3.26(a), with the indicated values for the stack pointer `%rsp` and the program counter `%rip`. The effect of the `call` is to push the return address `0x400568` onto the stack and to jump to the first instruction in function `multstore`, at address `0x0400540` (3.26(b)). The execution of function `multstore` continues until it hits the `ret` instruction at address `0x40054d`. This instruction pops the value `0x400568` from the stack and jumps to this address, resuming the execution of `main` just after the `call` instruction (3.26(c)).

As a more detailed example of passing control to and from procedures, Figure 3.27(a) shows the disassembled code for two functions, `top` and `leaf`, as well as the portion of code in function `main` where `top` gets called. Each instruction is identified by labels `L1–L2` (in `leaf`), `T1–T4` (in `top`), and `M1–M2` in `main`. Part (b) of the figure shows a detailed trace of the code execution, in which `main` calls `top(100)`, causing `top` to call `leaf(95)`. Function `leaf` returns 97 to `top`, which

(a) Disassembled code for demonstrating procedure calls and returns

```

Disassembly of leaf(long y)
y in %rdi
1  0000000000400540 <leaf>:
2      400540:  48 8d 47 02          lea    0x2(%rdi),%rax    L1: z+2
3      400544:  c3                  retq                    L2: Return

4  0000000000400545 <top>:
Disassembly of top(long x)
x in %rdi
5      400545:  48 83 ef 05          sub    $0x5,%rdi        T1: x-5
6      400549:  e8 f2 ff ff ff       callq  400540 <leaf>     T2: Call leaf(x-5)
7      40054e:  48 01 c0             add    %rax,%rax         T3: Double result
8      400551:  c3                  retq                    T4: Return

...

Call to top from function main
9      40055b:  e8 e5 ff ff ff       callq  400545 <top>      M1: Call top(100)
10     400560:  48 89 c2             mov    %rax,%rdx        M2: Resume

```

(b) Execution trace of example code

Label	Instruction		State values (at beginning)				Description
	PC	Instruction	%rdi	%rax	%rsp	*%rsp	
M1	0x40055b	callq	100	—	0x7fffffff820	—	Call top(100)
T1	0x400545	sub	100	—	0x7fffffff818	0x400560	Entry of top
T2	0x400549	callq	95	—	0x7fffffff818	0x400560	Call leaf(95)
L1	0x400540	lea	95	—	0x7fffffff810	0x40054e	Entry of leaf
L2	0x400544	retq	—	97	0x7fffffff810	0x40054e	Return 97 from leaf
T3	0x40054e	add	—	97	0x7fffffff818	0x400560	Resume top
T4	0x400551	retq	—	194	0x7fffffff818	0x400560	Return 194 from top
M2	0x400560	mov	—	194	0x7fffffff820	—	Resume main

Figure 3.27 Detailed execution of program involving procedure calls and returns. Using the stack to store return addresses makes it possible to return to the right point in the procedures.

then returns 194 to main. The first three columns describe the instruction being executed, including the instruction label, the address, and the instruction type. The next four columns show the state of the program *before* the instruction is executed, including the contents of registers %rdi, %rax, and %rsp, as well as the value at the top of the stack. The contents of this table should be studied carefully, as they

demonstrate the important role of the run-time stack in managing the storage needed to support procedure calls and returns.

Instruction L1 of `leaf` sets `%rax` to 97, the value to be returned. Instruction L2 then returns. It pops `0x400054e` from the stack. In setting the PC to this popped value, control transfers back to instruction T3 of `top`. The program has successfully completed the call to `leaf` and returned to `top`.

Instruction T3 sets `%rax` to 194, the value to be returned from `top`. Instruction T4 then returns. It pops `0x4000560` from the stack, thereby setting the PC to instruction M2 of `main`. The program has successfully completed the call to `top` and returned to `main`. We see that the stack pointer has also been restored to `0x7fffffff820`, the value it had before the call to `top`.

We can see that this simple mechanism of pushing the return address onto the stack makes it possible for the function to later return to the proper point in the program. The standard call/return mechanism of C (and of most programming languages) conveniently matches the last-in, first-out memory management discipline provided by a stack.

Practice Problem 3.32 (solution page 339)

The disassembled code for two functions `first` and `last` is shown below, along with the code for a call of `first` by function `main`:

```

Disassembly of last(long u, long v)
u in %rdi, v in %rsi
1 0000000000400540 <last>:
2 400540: 48 89 f8                mov    %rdi,%rax          L1: u
3 400543: 48 0f af c6             imul   %rsi,%rax          L2: u*v
4 400547: c3                     retq                     L3: Return

Disassembly of last(long x)
x in %rdi
5 0000000000400548 <first>:
6 400548: 48 8d 77 01             lea    0x1(%rdi),%rsi      F1: x+1
7 40054c: 48 83 ef 01             sub    $0x1,%rdi          F2: x-1
8 400550: e8 eb ff ff ff         callq  400540 <last>       F3: Call last(x-1,x+1)
9 400555: f3 c3                  repz retq                 F4: Return
:
:
10 400560: e8 e3 ff ff ff         callq  400548 <first>      M1: Call first(10)
11 400565: 48 89 c2                mov    %rax,%rdx          M2: Resume

```

Each of these instructions is given a label, similar to those in Figure 3.27(a). Starting with the calling of `first(10)` by `main`, fill in the following table to trace instruction execution through to the point where the program returns back to `main`.

Instruction			State values (at beginning)					Description
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	
M1	0x400560	callq	10	—	—	0x7fffffff820	—	Call first(10)
F1								
F2								
F3								
L1								
L2								
L3								
F4								
M2								

3.7.3 Data Transfer

In addition to passing control to a procedure when called, and then back again when the procedure returns, procedure calls may involve passing data as arguments, and returning from a procedure may also involve returning a value. With x86-64, most of these data passing to and from procedures take place via registers. For example, we have already seen numerous examples of functions where arguments are passed in registers %rdi, %rsi, and others, and where values are returned in register %rax. When procedure P calls procedure Q, the code for P must first copy the arguments into the proper registers. Similarly, when Q returns back to P, the code for P can access the returned value in register %rax. In this section, we explore these conventions in greater detail.

With x86-64, up to six integral (i.e., integer and pointer) arguments can be passed via registers. The registers are used in a specified order, with the name used for a register depending on the size of the data type being passed. These are shown in Figure 3.28. Arguments are allocated to these registers according to their

Operand size (bits)	Argument number					
	1	2	3	4	5	6
64	%rdi	%rsi	%rdx	%rcx	%r8	%r9
32	%edi	%esi	%edx	%ecx	%r8d	%r9d
16	%di	%si	%dx	%cx	%r8w	%r9w
8	%dil	%sil	%dl	%cl	%r8b	%r9b

Figure 3.28 Registers for passing function arguments. The registers are used in a specified order and named according to the argument sizes.

ordering in the argument list. Arguments smaller than 64 bits can be accessed using the appropriate subsection of the 64-bit register. For example, if the first argument is 32 bits, it can be accessed as `%edi`.

When a function has more than six integral arguments, the other ones are passed on the stack. Assume that procedure P calls procedure Q with n integral arguments, such that $n > 6$. Then the code for P must allocate a stack frame with enough storage for arguments 7 through n , as illustrated in Figure 3.25. It copies arguments 1–6 into the appropriate registers, and it puts arguments 7 through n onto the stack, with argument 7 at the top of the stack. When passing parameters on the stack, all data sizes are rounded up to be multiples of eight. With the arguments in place, the program can then execute a `call` instruction to transfer control to procedure Q. Procedure Q can access its arguments via registers and possibly from the stack. If Q, in turn, calls some function that has more than six arguments, it can allocate space within its stack frame for these, as is illustrated by the area labeled “Argument build area” in Figure 3.25.

As an example of argument passing, consider the C function `proc` shown in Figure 3.29(a). This function has eight arguments, including integers with different numbers of bytes (8, 4, 2, and 1), as well as different types of pointers, each of which is 8 bytes.

The assembly code generated for `proc` is shown in Figure 3.29(b). The first six arguments are passed in registers. The last two are passed on the stack, as documented by the diagram of Figure 3.30. This diagram shows the state of the stack during the execution of `proc`. We can see that the return address was pushed onto the stack as part of the procedure call. The two arguments, therefore, are at positions 8 and 16 relative to the stack pointer. Within the code, we can see that different versions of the `ADD` instruction are used according to the sizes of the operands: `addq` for `a1` (long), `addl` for `a2` (int), `addw` for `a3` (short), and `addb` for `a4` (char). Observe that the `movl` instruction of line 6 reads 4 bytes from memory; the following `addb` instruction only makes use of the low-order byte.

Practice Problem 3.33 (solution page 339)

A C function `procpb` has four arguments `u`, `a`, `v`, and `b`. Each is either a signed number or a pointer to a signed number, where the numbers have different sizes. The function has the following body:

```
*u += a;
*v += b;
return sizeof(a) + sizeof(b);
```

It compiles to the following x86-64 code:

```
1  procpb:
2      movslq  %edi, %rdi
3      addq    %rdi, (%rdx)
4      addb    %sil, (%rcx)
```

(a) C code

```

void proc(long a1, long *a1p,
          int a2, int *a2p,
          short a3, short *a3p,
          char a4, char *a4p)
{
    *a1p += a1;
    *a2p += a2;
    *a3p += a3;
    *a4p += a4;
}

```

(b) Generated assembly code

```

void proc(a1, a1p, a2, a2p, a3, a3p, a4, a4p)
Arguments passed as follows:
    a1 in %rdi          (64 bits)
    a1p in %rsi         (64 bits)
    a2 in %edx          (32 bits)
    a2p in %rcx         (64 bits)
    a3 in %r8w          (16 bits)
    a3p in %r9          (64 bits)
    a4 at %rsp+8        ( 8 bits)
    a4p at %rsp+16      (64 bits)

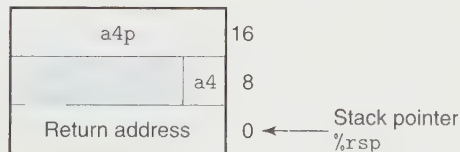
1  proc:
2  movq    16(%rsp), %rax    Fetch a4p (64 bits)
3  addq    %rdi, (%rsi)     *a1p += a1 (64 bits)
4  addl    %edx, (%rcx)     *a2p += a2 (32 bits)
5  addw    %r8w, (%r9)      *a3p += a3 (16 bits)
6  movl    8(%rsp), %edx     Fetch a4  ( 8 bits)
7  addb    %dl, (%rax)      *a4p += a4 ( 8 bits)
8  ret                                Return

```

Figure 3.29 Example of function with multiple arguments of different types. Arguments 1–6 are passed in registers, while arguments 7–8 are passed on the stack.

Figure 3.30

Stack frame structure for function `proc`. Arguments `a4` and `a4p` are passed on the stack.



```

5      movl    $6, %eax
6      ret

```

Determine a valid ordering and types of the four parameters. There are two correct answers.

3.7.4 Local Storage on the Stack

Most of the procedure examples we have seen so far did not require any local storage beyond what could be held in registers. At times, however, local data must be stored in memory. Common cases of this include these:

- There are not enough registers to hold all of the local data.
- The address operator ‘&’ is applied to a local variable, and hence we must be able to generate an address for it.
- Some of the local variables are arrays or structures and hence must be accessed by array or structure references. We will discuss this possibility when we describe how arrays and structures are allocated.

Typically, a procedure allocates space on the stack frame by decrementing the stack pointer. This results in the portion of the stack frame labeled “Local variables” in Figure 3.25.

As an example of the handling of the address operator, consider the two functions shown in Figure 3.31(a). The function `swap_add` swaps the two values designated by pointers `xp` and `yp` and also returns the sum of the two values. The function caller creates pointers to local variables `arg1` and `arg2` and passes these to `swap_add`. Figure 3.31(b) shows how caller uses a stack frame to implement these local variables. The code for caller starts by decrementing the stack pointer by 16; this effectively allocates 16 bytes on the stack. Letting S denote the value of the stack pointer, we can see that the code computes `&arg2` as $S + 8$ (line 5), `&arg1` as S (line 6). We can therefore infer that local variables `arg1` and `arg2` are stored within the stack frame at offsets 0 and 8 relative to the stack pointer. When the call to `swap_add` completes, the code for caller then retrieves the two values from the stack (lines 8–9), computes their difference, and multiplies this by the value returned by `swap_add` in register `%rax` (line 10). Finally, the function deallocates its stack frame by incrementing the stack pointer by 16 (line 11.) We can see with this example that the run-time stack provides a simple mechanism for allocating local storage when it is required and deallocating it when the function completes.

As a more complex example, the function `call_proc`, shown in Figure 3.32, illustrates many aspects of the x86-64 stack discipline. Despite the length of this example, it is worth studying carefully. It shows a function that must allocate storage on the stack for local variables, as well as to pass values to the 8-argument function `proc` (Figure 3.29). The function creates a stack frame, diagrammed in Figure 3.33.

Looking at the assembly code for `call_proc` (Figure 3.32(b)), we can see that a large portion of the code (lines 2–15) involves preparing to call function

(a) Code for `swap_add` and calling function

```
long swap_add(long *xp, long *yp)
{
    long x = *xp;
    long y = *yp;
    *xp = y;
    *yp = x;
    return x + y;
}

long caller()
{
    long arg1 = 534;
    long arg2 = 1057;
    long sum = swap_add(&arg1, &arg2);
    long diff = arg1 - arg2;
    return sum * diff;
}
```

(b) Generated assembly code for calling function

```
long caller()
1 caller:
2     subq    $16, %rsp        Allocate 16 bytes for stack frame
3     movq    $534, (%rsp)     Store 534 in arg1
4     movq    $1057, 8(%rsp)   Store 1057 in arg2
5     leaq    8(%rsp), %rsi     Compute &arg2 as second argument
6     movq    %rsp, %rdi       Compute &arg1 as first argument
7     call    swap_add         Call swap_add(&arg1, &arg2)
8     movq    (%rsp), %rdx     Get arg1
9     subq    8(%rsp), %rdx     Compute diff = arg1 - arg2
10    imulq   %rdx, %rax        Compute sum * diff
11    addq    $16, %rsp        Deallocate stack frame
12    ret                                Return
```

Figure 3.31 Example of procedure definition and call. The calling code must allocate a stack frame due to the presence of address operators.

`proc`. This includes setting up the stack frame for the local variables and function parameters, and for loading function arguments into registers. As Figure 3.33 shows, local variables `x1`–`x4` are allocated on the stack and have different sizes. Expressing their locations as offsets relative to the stack pointer, they occupy bytes 24–31 (`x1`), 20–23 (`x2`), 18–19 (`x3`), and 17 (`s3`). Pointers to these locations are generated by `leaq` instructions (lines 7, 10, 12, and 14). Arguments 7 (with value 4) and 8 (a pointer to the location of `x4`) are stored on the stack at offsets 0 and 8 relative to the stack pointer.

(a) C code for calling function

```
long call_proc()
{
    long x1 = 1; int x2 = 2;
    short x3 = 3; char x4 = 4;
    proc(x1, &x1, x2, &x2, x3, &x3, x4, &x4);
    return (x1+x2)*(x3-x4);
}
```

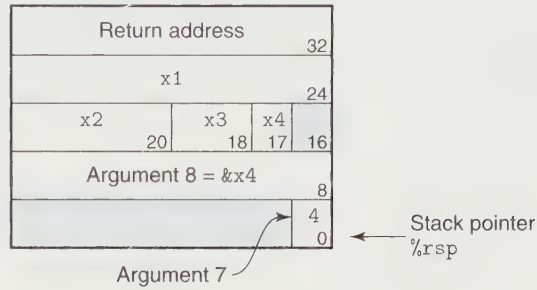
(b) Generated assembly code

```
long call_proc()
1  call_proc:
    Set up arguments to proc
2      subq    $32, %rsp           Allocate 32-byte stack frame
3      movq    $1, 24(%rsp)       Store 1 in &x1
4      movl    $2, 20(%rsp)       Store 2 in &x2
5      movw    $3, 18(%rsp)       Store 3 in &x3
6      movb    $4, 17(%rsp)       Store 4 in &x4
7      leaq    17(%rsp), %rax      Create &x4
8      movq    %rax, 8(%rsp)       Store &x4 as argument 8
9      movl    $4, (%rsp)         Store 4 as argument 7
10     leaq    18(%rsp), %r9       Pass &x3 as argument 6
11     movl    $3, %r8d            Pass 3 as argument 5
12     leaq    20(%rsp), %rcx      Pass &x2 as argument 4
13     movl    $2, %edx            Pass 2 as argument 3
14     leaq    24(%rsp), %rsi      Pass &x1 as argument 2
15     movl    $1, %edi            Pass 1 as argument 1
    Call proc
16     call    proc
    Retrieve changes to memory
17     movslq   20(%rsp), %rdx      Get x2 and convert to long
18     addq     24(%rsp), %rdx      Compute x1+x2
19     movswl   18(%rsp), %eax      Get x3 and convert to int
20     movsbl   17(%rsp), %ecx      Get x4 and convert to int
21     subl     %ecx, %eax          Compute x3-x4
22     cltq                                           Convert to long
23     imulq    %rdx, %rax          Compute (x1+x2) * (x3-x4)
24     addq     $32, %rsp           Deallocate stack frame
25     ret                                           Return
```

Figure 3.32 Example of code to call function `proc`, defined in Figure 3.29. This code creates a stack frame.

Figure 3.33

Stack frame for function `call_proc`. The stack frame contains local variables, as well as two of the arguments to pass to function `proc`.



When procedure `proc` is called, the program will begin executing the code shown in Figure 3.29(b). As shown in Figure 3.30, arguments 7 and 8 are now at offsets 8 and 16 relative to the stack pointer, because the return address was pushed onto the stack.

When the program returns to `call_proc`, the code retrieves the values of the four local variables (lines 17–20) and performs the final computations. It finishes by incrementing the stack pointer by 32 to deallocate the stack frame.

3.7.5 Local Storage in Registers

The set of program registers acts as a single resource shared by all of the procedures. Although only one procedure can be active at a given time, we must make sure that when one procedure (the *caller*) calls another (the *callee*), the callee does not overwrite some register value that the caller planned to use later. For this reason, x86-64 adopts a uniform set of conventions for register usage that must be respected by all procedures, including those in program libraries.

By convention, registers `%rbx`, `%rbp`, and `%r12–%r15` are classified as *callee-saved* registers. When procedure `P` calls procedure `Q`, `Q` must *preserve* the values of these registers, ensuring that they have the same values when `Q` returns to `P` as they did when `Q` was called. Procedure `Q` can preserve a register value by either not changing it at all or by pushing the original value on the stack, altering it, and then popping the old value from the stack before returning. The pushing of register values has the effect of creating the portion of the stack frame labeled “Saved registers” in Figure 3.25. With this convention, the code for `P` can safely store a value in a callee-saved register (after saving the previous value on the stack, of course), call `Q`, and then use the value in the register without risk of it having been corrupted.

All other registers, except for the stack pointer `%rsp`, are classified as *caller-saved* registers. This means that they can be modified by any function. The name “caller saved” can be understood in the context of a procedure `P` having some local data in such a register and calling procedure `Q`. Since `Q` is free to alter this register, it is incumbent upon `P` (the caller) to first save the data before it makes the call.

As an example, consider the function `P` shown in Figure 3.34(a). It calls `Q` twice. During the first call, it must retain the value of `x` for use later. Similarly, during the second call, it must retain the value computed for `Q(y)`. In Figure 3.34(b),

(a) Calling function

```

long P(long x, long y)
{
    long u = Q(y);
    long v = Q(x);
    return u + v;
}

```

(b) Generated assembly code for the calling function

```

    long P(long x, long y)
    x in %rdi, y in %rsi
1   P:
2   pushq   %rbp                Save %rbp
3   pushq   %rbx                Save %rbx
4   subq    $8, %rsp            Align stack frame
5   movq    %rdi, %rbp          Save x
6   movq    %rsi, %rdi          Move y to first argument
7   call    Q                   Call Q(y)
8   movq    %rax, %rbx          Save result
9   movq    %rbp, %rdi          Move x to first argument
10  call    Q                   Call Q(x)
11  addq    %rbx, %rax           Add saved Q(y) to Q(x)
12  addq    $8, %rsp            Deallocate last part of stack
13  popq    %rbx                Restore %rbx
14  popq    %rbp                Restore %rbp
15  ret

```

Figure 3.34 Code demonstrating use of callee-saved registers. Value x must be preserved during the first call, and value $Q(y)$ must be preserved during the second.

we can see that the code generated by GCC uses two callee-saved registers: `%rbp` to hold x , and `%rbx` to hold the computed value of $Q(y)$. At the beginning of the function, it saves the values of these two registers on the stack (lines 2–3). It copies argument x to `%rbp` before the first call to Q (line 5). It copies the result of this call to `%rbx` before the second call to Q (line 8). At the end of the function (lines 13–14), it restores the values of the two callee-saved registers by popping them off the stack. Note how they are popped in the reverse order from how they were pushed, to account for the last-in, first-out discipline of a stack.

Practice Problem 3.34 (solution page 340)

Consider a function P , which generates local values, named a_0 – a_8 . It then calls function Q using these generated values as arguments. GCC produces the following code for the first part of P :

```

long P(long x)
x in %rdi
1  P:
2      pushq    %r15
3      pushq    %r14
4      pushq    %r13
5      pushq    %r12
6      pushq    %rbp
7      pushq    %rbx
8      subq     $24, %rsp
9      movq     %rdi, %rbx
10     leaq     1(%rdi), %r15
11     leaq     2(%rdi), %r14
12     leaq     3(%rdi), %r13
13     leaq     4(%rdi), %r12
14     leaq     5(%rdi), %rbp
15     leaq     6(%rdi), %rax
16     movq     %rax, (%rsp)
17     leaq     7(%rdi), %rdx
18     movq     %rdx, 8(%rsp)
19     movl     $0, %eax
20     call     Q

```

- A. Identify which local values get stored in callee-saved registers.
 - B. Identify which local values get stored on the stack.
 - C. Explain why the program could not store all of the local values in callee-saved registers.
-

3.7.6 Recursive Procedures

The conventions we have described for using the registers and the stack allow x86-64 procedures to call themselves recursively. Each procedure call has its own private space on the stack, and so the local variables of the multiple outstanding calls do not interfere with one another. Furthermore, the stack discipline naturally provides the proper policy for allocating local storage when the procedure is called and deallocating it before returning.

Figure 3.35 shows both the C code and the generated assembly code for a recursive factorial function. We can see that the assembly code uses register `%rbx` to hold the parameter `n`, after first saving the existing value on the stack (line 2) and later restoring the value before returning (line 11). Due to the stack discipline, and the register-saving conventions, we can be assured that when the recursive call to `rfact(n-1)` returns (line 9) that (1) the result of the call will be held in register

(a) C code

```

long rfact(long n)
{
    long result;
    if (n <= 1)
        result = 1;
    else
        result = n * rfact(n-1);
    return result;
}

```

(b) Generated assembly code

```

    long rfact(long n)
    n in %rdi
1   rfact:
2   pushq   %rbx                Save %rbx
3   movq    %rdi, %rbx          Store n in callee-saved register
4   movl    $1, %eax            Set return value = 1
5   cmpq    $1, %rdi            Compare n:1
6   jle     .L35                If <=, goto done
7   leaq    -1(%rdi), %rdi       Compute n-1
8   call    rfact                Call rfact(n-1)
9   imulq   %rbx, %rax           Multiply result by n
10  .L35:                        done:
11  popq     %rbx                Restore %rbx
12  ret                                Return

```

Figure 3.35 Code for recursive factorial program. The standard procedure handling mechanisms suffice for implementing recursive functions.

%rax, and (2) the value of argument n will held in register %rbx. Multiplying these two values then computes the desired result.

We can see from this example that calling a function recursively proceeds just like any other function call. Our stack discipline provides a mechanism where each invocation of a function has its own private storage for state information (saved values of the return location and callee-saved registers). If need be, it can also provide storage for local variables. The stack discipline of allocation and deallocation naturally matches the call-return ordering of functions. This method of implementing function calls and returns even works for more complex patterns, including mutual recursion (e.g., when procedure P calls Q , which in turn calls P).

Practice Problem 3.35 (solution page 340)

For a C function having the general structure


```

long rfun(unsigned long x) {
    if ( _____ )
        return _____ ;
    unsigned long nx = _____;
    long rv = rfun(nx);
    return _____;
}

```

gcc generates the following assembly code:

```

    long rfun(unsigned long x)
    x in %rdi
1   rfun:
2       pushq    %rbx
3       movq     %rdi, %rbx
4       movl     $0, %eax
5       testq    %rdi, %rdi
6       je       .L2
7       shrq     $2, %rdi
8       call     rfun
9       addq     %rbx, %rax
10      .L2:
11      popq     %rbx
12      ret

```

- A. What value does rfun store in the callee-saved register %rbx?
 - B. Fill in the missing expressions in the C code shown above.
-

3.8 Array Allocation and Access

Arrays in C are one means of aggregating scalar data into larger data types. C uses a particularly simple implementation of arrays, and hence the translation into machine code is fairly straightforward. One unusual feature of C is that we can generate pointers to elements within arrays and perform arithmetic with these pointers. These are translated into address computations in machine code.

Optimizing compilers are particularly good at simplifying the address computations used by array indexing. This can make the correspondence between the C code and its translation into machine code somewhat difficult to decipher.

3.8.1 Basic Principles

For data type T and integer constant N , consider a declaration of the form

```
 $T$  A[N];
```

Let us denote the starting location as x_A . The declaration has two effects. First, it allocates a contiguous region of $L \cdot N$ bytes in memory, where L is the size (in bytes) of data type T . Second, it introduces an identifier A that can be used as a pointer to the beginning of the array. The value of this pointer will be x_A . The array elements can be accessed using an integer index ranging between 0 and $N - 1$. Array element i will be stored at address $x_A + L \cdot i$.

As examples, consider the following declarations:

```

char    A[12];
char    *B[8];
int      C[6];
double  *D[5];

```

These declarations will generate arrays with the following parameters:

Array	Element size	Total size	Start address	Element i
A	1	12	x_A	$x_A + i$
B	8	64	x_B	$x_B + 8i$
C	4	24	x_C	$x_C + 4i$
D	8	40	x_D	$x_D + 8i$

Array A consists of 12 single-byte (char) elements. Array C consists of 6 integers, each requiring 4 bytes. B and D are both arrays of pointers, and hence the array elements are 8 bytes each.

The memory referencing instructions of x86-64 are designed to simplify array access. For example, suppose E is an array of values of type `int` and we wish to evaluate $E[i]$, where the address of E is stored in register `%rdx` and i is stored in register `%rcx`. Then the instruction

```

movl (%rdx,%rcx,4),%eax

```

will perform the address computation $x_E + 4i$, read that memory location, and copy the result to register `%eax`. The allowed scaling factors of 1, 2, 4, and 8 cover the sizes of the common primitive data types.

Practice Problem 3.36 (solution page 341)

Consider the following declarations:

```

short    S[7];
short    *T[3];
short    **U[6];
int      V[8];
double   *W[4];

```

Fill in the following table describing the element size, the total size, and the address of element i for each of these arrays.

Array	Element size	Total size	Start address	Element i
S	_____	_____	x_S	_____
T	_____	_____	x_T	_____
U	_____	_____	x_U	_____
V	_____	_____	x_V	_____
W	_____	_____	x_W	_____

3.8.2 Pointer Arithmetic

C allows arithmetic on pointers, where the computed value is scaled according to the size of the data type referenced by the pointer. That is, if p is a pointer to data of type T , and the value of p is x_p , then the expression $p+i$ has value $x_p + L \cdot i$, where L is the size of data type T .

The unary operators ‘&’ and ‘*’ allow the generation and dereferencing of pointers. That is, for an expression $Expr$ denoting some object, $\&Expr$ is a pointer giving the address of the object. For an expression $AExpr$ denoting an address, $*AExpr$ gives the value at that address. The expressions $Expr$ and $*\&Expr$ are therefore equivalent. The array subscripting operation can be applied to both arrays and pointers. The array reference $A[i]$ is identical to the expression $*(A+i)$. It computes the address of the i th array element and then accesses this memory location.

Expanding on our earlier example, suppose the starting address of integer array E and integer index i are stored in registers $\%rdx$ and $\%rcx$, respectively. The following are some expressions involving E . We also show an assembly-code implementation of each expression, with the result being stored in either register $\%eax$ (for data) or register $\%rax$ (for pointers).

Expression	Type	Value	Assembly code
E	$\text{int} *$	x_E	<code>movl %rdx,%rax</code>
$E[0]$	int	$M[x_E]$	<code>movl (%rdx),%eax</code>
$E[i]$	int	$M[x_E + 4i]$	<code>movl (%rdx,%rcx,4),%eax</code>
$\&E[2]$	$\text{int} *$	$x_E + 8$	<code>leaq 8(%rdx),%rax</code>
$E+i-1$	$\text{int} *$	$x_E + 4i - 4$	<code>leaq -4(%rdx,%rcx,4),%rax</code>
$*(E+i-3)$	int	$M[x_E + 4i - 12]$	<code>movl -12(%rdx,%rcx,4),%eax</code>
$\&E[i]-E$	long	i	<code>movq %rcx,%rax</code>

In these examples, we see that operations that return array values have type int , and hence involve 4-byte operations (e.g., `movl`) and registers (e.g., $\%eax$). Those that return pointers have type $\text{int} *$, and hence involve 8-byte operations (e.g., `leaq`) and registers (e.g., $\%rax$). The final example shows that one can compute the difference of two pointers within the same data structure, with the result being data having type long and value equal to the difference of the two addresses divided by the size of the data type.

Practice Problem 3.37 (solution page 341)

Suppose x_S , the address of short integer array S , and long integer index i are stored in registers $\%rdx$ and $\%rcx$, respectively. For each of the following expressions, give its type, a formula for its value, and an assembly-code implementation. The result should be stored in register $\%rax$ if it is a pointer and register element $\%ax$ if it has data type short.

Expression	Type	Value	Assembly code
$S+1$			
$S[3]$			
$\&S[i]$			
$S[4 \cdot i + 1]$			
$S+i-5$			

3.8.3 Nested Arrays

The general principles of array allocation and referencing hold even when we create arrays of arrays. For example, the declaration

```
int A[5][3];
```

is equivalent to the declaration

```
typedef int row3_t[3];
row3_t A[5];
```

Data type `row3_t` is defined to be an array of three integers. Array A contains five such elements, each requiring 12 bytes to store the three integers. The total array size is then $4 \cdot 5 \cdot 3 = 60$ bytes.

Array A can also be viewed as a two-dimensional array with five rows and three columns, referenced as $A[0][0]$ through $A[4][2]$. The array elements are ordered in memory in *row-major* order, meaning all elements of row 0, which can be written $A[0]$, followed by all elements of row 1 ($A[1]$), and so on. This is illustrated in Figure 3.36.

This ordering is a consequence of our nested declaration. Viewing A as an array of five elements, each of which is an array of three `int`'s, we first have $A[0]$, followed by $A[1]$, and so on.

To access elements of multidimensional arrays, the compiler generates code to compute the offset of the desired element and then uses one of the `mov` instructions with the start of the array as the base address and the (possibly scaled) offset as an index. In general, for an array declared as

```
T D[R][C];
```

array element $D[i][j]$ is at memory address

$$\&D[i][j] = x_D + L(C \cdot i + j) \quad (3.1)$$

Figure 3.36
Elements of array in
row-major order.

Row	Element	Address
A[0]	A[0][0]	x_A
	A[0][1]	$x_A + 4$
	A[0][2]	$x_A + 8$
A[1]	A[1][0]	$x_A + 12$
	A[1][1]	$x_A + 16$
	A[1][2]	$x_A + 20$
A[2]	A[2][0]	$x_A + 24$
	A[2][1]	$x_A + 28$
	A[2][2]	$x_A + 32$
A[3]	A[3][0]	$x_A + 36$
	A[3][1]	$x_A + 40$
	A[3][2]	$x_A + 44$
A[4]	A[4][0]	$x_A + 48$
	A[4][1]	$x_A + 52$
	A[4][2]	$x_A + 56$

where L is the size of data type T in bytes. As an example, consider the 5×3 integer array A defined earlier. Suppose x_A , i , and j are in registers `%rdi`, `%rsi`, and `%rdx`, respectively. Then array element $A[i][j]$ can be copied to register `%eax` by the following code:

```

A in %rdi, i in %rsi, and j in %rdx
1   leaq    (%rsi,%rsi,2), %rax    Compute 3i
2   leaq    (%rdi,%rax,4), %rax    Compute  $x_A + 12i$ 
3   movl    (%rax,%rdx,4), %eax    Read from  $M[x_A + 12i + 4j]$ 

```

As can be seen, this code computes the element's address as $x_A + 12i + 4j = x_A + 4(3i + j)$ using the scaling and addition capabilities of x86-64 address arithmetic.

Practice Problem 3.38 (solution page 341)

Consider the following source code, where M and N are constants declared with `#define`:

```

long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}

```

In compiling this program, gcc generates the following assembly code:


```

    long sum_element(long i, long j)
    i in %rdi, j in %rsi
1   sum_element:
2       leaq    0(,%rdi,8), %rdx
3       subq    %rdi, %rdx
4       addq    %rsi, %rdx
5       leaq    (%rsi,%rsi,4), %rax
6       addq    %rax, %rdi
7       movq    Q(,%rdi,8), %rax
8       addq    P(,%rdx,8), %rax
9       ret

```

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

3.8.4 Fixed-Size Arrays

The C compiler is able to make many optimizations for code operating on multi-dimensional arrays of fixed size. Here we demonstrate some of the optimizations made by gcc when the optimization level is set with the flag `-O1`. Suppose we declare data type `fix_matrix` to be 16×16 arrays of integers as follows:

```

#define N 16
typedef int fix_matrix[N][N];

```

(This example illustrates a good coding practice. Whenever a program uses some constant as an array dimension or buffer size, it is best to associate a name with it via a `#define` declaration, and then use this name consistently, rather than the numeric value. That way, if an occasion ever arises to change the value, it can be done by simply modifying the `#define` declaration.) The code in Figure 3.37(a) computes element i, k of the product of arrays A and B—that is, the inner product of row i from A and column k from B. This product is given by the formula $\sum_{0 \leq j < N} a_{i,j} \cdot b_{j,k}$. Gcc generates code that we then recoded into C, shown as function `fix_prod_ele_opt` in Figure 3.37(b). This code contains a number of clever optimizations. It removes the integer index j and converts all array references to pointer dereferences. This involves (1) generating a pointer, which we have named `Aptr`, that points to successive elements in row i of A, (2) generating a pointer, which we have named `Bptr`, that points to successive elements in column k of B, and (3) generating a pointer, which we have named `Bend`, that equals the value `Bptr` will have when it is time to terminate the loop. The initial value for `Aptr` is the address of the first element of row i of A, given by the C expression `&A[i][0]`. The initial value for `Bptr` is the address of the first element of column k of B, given by the C expression `&B[0][k]`. The value for `Bend` is the index of what would be the $(n + 1)$ st element in column j of B, given by the C expression `&B[N][k]`.

(a) Original C code

```
/* Compute i,k of fixed matrix product */
int fix_prod_ele (fix_matrix A, fix_matrix B, long i, long k) {
    long j;
    int result = 0;

    for (j = 0; j < N; j++)
        result += A[i][j] * B[j][k];

    return result;
}
```

(b) Optimized C code

```
1  /* Compute i,k of fixed matrix product */
2  int fix_prod_ele_opt(fix_matrix A, fix_matrix B, long i, long k) {
3      int *Aptr = &A[i][0];    /* Points to elements in row i of A */
4      int *Bptr = &B[0][k];    /* Points to elements in column k of B */
5      int *Bend = &B[N][k];    /* Marks stopping point for Bptr */
6      int result = 0;
7      do {                      /* No need for initial test */
8          result += *Aptr * *Bptr; /* Add next product to sum */
9          Aptr++;                /* Move Aptr to next column */
10         Bptr += N;              /* Move Bptr to next row */
11     } while (Bptr != Bend);    /* Test for stopping point */
12     return result;
13 }
```

Figure 3.37 Original and optimized code to compute element i, k of matrix product for fixed-length arrays. The compiler performs these optimizations automatically.

The following is the actual assembly code generated by gcc for function `fix_prod_ele`. We see that four registers are used as follows: `%eax` holds `result`, `%rdi` holds `Aptr`, `%rcx` holds `Bptr`, and `%rsi` holds `Bend`.

```
int fix_prod_ele_opt(fix_matrix A, fix_matrix B, long i, long k)
A in %rdi, B in %rsi, i in %rdx, k in %rcx
1  fix_prod_ele:
2      salq    $6, %rdx                Compute 64 * i
3      addq    %rdx, %rdi              Compute Aptr =  $x_A + 64i = \&A[i][0]$ 
4      leaq    (%rsi,%rcx,4), %rcx     Compute Bptr =  $x_B + 4k = \&B[0][k]$ 
5      leaq    1024(%rcx), %rsi        Compute Bend =  $x_B + 4k + 1024 = \&B[N][k]$ 
6      movl    $0, %eax                Set result = 0
7  .L7:                                loop:
8      movl    (%rdi), %edx            Read *Aptr
9      imull   (%rcx), %edx            Multiply by *Bptr
10     addl    %edx, %eax              Add to result
```

11	addq	\$4, %rdi	<i>Increment Aptr ++</i>
12	addq	\$64, %rcx	<i>Increment Bptr += N</i>
13	cmpq	%rsi, %rcx	<i>Compare Bptr:Bend</i>
14	jne	.L7	<i>If !=, goto loop</i>
15	rep; ret		<i>Return</i>

Practice Problem 3.39 (solution page 342)

Use Equation 3.1 to explain how the computations of the initial values for `Aptr`, `Bptr`, and `Bend` in the C code of Figure 3.37(b) (lines 3–5) correctly describe their computations in the assembly code generated for `fix_prod_ele` (lines 3–5).

Practice Problem 3.40 (solution page 342)

The following C code sets the diagonal elements of one of our fixed-size arrays to `val`:

```
/* Set all diagonal elements to val */
void fix_set_diag(fix_matrix A, int val) {
    long i;
    for (i = 0; i < N; i++)
        A[i][i] = val;
}
```

When compiled with optimization level `-O1`, gcc generates the following assembly code:

```
1  fix_set_diag:
   void fix_set_diag(fix_matrix A, int val)
   A in %rdi, val in %rsi
2      movl    $0, %eax
3  .L13:
4      movl    %esi, (%rdi,%rax)
5      addq    $64, %rax
6      cmpq    $1088, %rax
7      jne     .L13
8      rep; ret
```

Create a C code program `fix_set_diag_opt` that uses optimizations similar to those in the assembly code, in the same style as the code in Figure 3.37(b). Use expressions involving the parameter `N` rather than integer constants, so that your code will work correctly if `N` is redefined.

3.8.5 Variable-Size Arrays

Historically, C only supported multidimensional arrays where the sizes (with the possible exception of the first dimension) could be determined at compile time.

Programmers requiring variable-size arrays had to allocate storage for these arrays using functions such as `malloc` or `calloc`, and they had to explicitly encode the mapping of multidimensional arrays into single-dimension ones via row-major indexing, as expressed in Equation 3.1. ISO C99 introduced the capability of having array dimension expressions that are computed as the array is being allocated.

In the C version of variable-size arrays, we can declare an array

```
int A[expr1][expr2]
```

either as a local variable or as an argument to a function, and then the dimensions of the array are determined by evaluating the expressions *expr1* and *expr2* at the time the declaration is encountered. So, for example, we can write a function to access element *i*, *j* of an $n \times n$ array as follows:

```
int var_ele(long n, int A[n][n], long i, long j) {
    return A[i][j];
}
```

The parameter *n* must precede the parameter `A[n][n]`, so that the function can compute the array dimensions as the parameter is encountered.

Gcc generates code for this referencing function as

```
int var_ele(long n, int A[n][n], long i, long j)
n in %rdi, A in %rsi, i in %rdx, j in %rcx
1  var_ele:
2      imulq    %rdx, %rdi                Compute  $n \cdot i$ 
3      leaq     (%rsi,%rdi,4), %rax        Compute  $x_A + 4(n \cdot i$ 
4      movl     (%rax,%rcx,4), %eax        Read from  $M[x_A + 4(n \cdot i) + 4j]$ 
5      ret
```

As the annotations show, this code computes the address of element *i*, *j* as $x_A + 4(n \cdot i) + 4j = x_A + 4(n \cdot i + j)$. The address computation is similar to that of the fixed-size array (Section 3.8.3), except that (1) the register usage changes due to added parameter *n*, and (2) a multiply instruction is used (line 2) to compute $n \cdot i$, rather than an `leaq` instruction to compute $3i$. We see therefore that referencing variable-size arrays requires only a slight generalization over fixed-size ones. The dynamic version must use a multiplication instruction to scale *i* by *n*, rather than a series of shifts and adds. In some processors, this multiplication can incur a significant performance penalty, but it is unavoidable in this case.

When variable-size arrays are referenced within a loop, the compiler can often optimize the index computations by exploiting the regularity of the access patterns. For example, Figure 3.38(a) shows C code to compute element *i*, *k* of the product of two $n \times n$ arrays A and B. Gcc generates assembly code, which we have recast into C (Figure 3.38(b)). This code follows a different style from the optimized code for the fixed-size array (Figure 3.37), but that is more an artifact of the choices made by the compiler, rather than a fundamental requirement for the two different functions. The code of Figure 3.38(b) retains loop variable *j*, both to detect when

(a) Original C code

```

1  /* Compute i,k of variable matrix product */
2  int var_prod_ele(long n, int A[n][n], int B[n][n], long i, long k) {
3      long j;
4      int result = 0;
5
6      for (j = 0; j < n; j++)
7          result += A[i][j] * B[j][k];
8
9      return result;
10 }
```

(b) Optimized C code

```

/* Compute i,k of variable matrix product */
int var_prod_ele_opt(long n, int A[n][n], int B[n][n], long i, long k) {
    int *Arow = A[i];
    int *Bptr = &B[0][k];
    int result = 0;
    long j;
    for (j = 0; j < n; j++) {
        result += Arow[j] * *Bptr;
        Bptr += n;
    }
    return result;
}
```

Figure 3.38 Original and optimized code to compute element i, k of matrix product for variable-size arrays. The compiler performs these optimizations automatically.

the loop has terminated and to index into an array consisting of the elements of row i of A .

The following is the assembly code for the loop of `var_prod_ele`:

```

Registers: n in %rdi, Arow in %rsi, Bptr in %rcx
          4n in %r9, result in %eax, j in %edx

1  .L24:                                loop:
2      movl    (%rsi,%rdx,4), %r8d      Read Arow[j]
3      imull   (%rcx), %r8d             Multiply by *Bptr
4      addl    %r8d, %eax               Add to result
5      addq    $1, %rdx                 j++
6      addq    %r9, %rcx                Bptr += n
7      cmpq    %rdi, %rdx              Compare j:n
8      jne     .L24                    If !=, goto loop
```

We see that the program makes use of both a scaled value $4n$ (register `%r9`) for incrementing `Bptr` as well as the value of n (register `%rdi`) to check the loop

bounds. The need for two values does not show up in the C code, due to the scaling of pointer arithmetic.

We have seen that, with optimizations enabled, gcc is able to recognize patterns that arise when a program steps through the elements of a multidimensional array. It can then generate code that avoids the multiplication that would result from a direct application of Equation 3.1. Whether it generates the pointer-based code of Figure 3.37(b) or the array-based code of Figure 3.38(b), these optimizations will significantly improve program performance.

3.9 Heterogeneous Data Structures

C provides two mechanisms for creating data types by combining objects of different types: *structures*, declared using the keyword `struct`, aggregate multiple objects into a single unit; *unions*, declared using the keyword `union`, allow an object to be referenced using several different types.

3.9.1 Structures

The C `struct` declaration creates a data type that groups objects of possibly different types into a single object. The different components of a structure are referenced by names. The implementation of structures is similar to that of arrays in that all of the components of a structure are stored in a contiguous region of memory and a pointer to a structure is the address of its first byte. The compiler maintains information about each structure type indicating the byte offset of each field. It generates references to structure elements using these offsets as displacements in memory referencing instructions.

As an example, consider the following structure declaration:

```
struct rec {
    int i;
    int j;
    int a[2];
    int *p;
};
```

This structure contains four fields: two 4-byte values of type `int`, a two-element array of type `int`, and an 8-byte integer pointer, giving a total of 24 bytes:

Offset	0	4	8	16	24
Contents	i	j	a[0]	a[1]	p

Observe that array `a` is embedded within the structure. The numbers along the top of the diagram give the byte offsets of the fields from the beginning of the structure.

To access the fields of a structure, the compiler generates code that adds the appropriate offset to the address of the structure. For example, suppose variable `r`

New to C? Representing an object as a struct

The struct data type constructor is the closest thing C provides to the objects of C++ and Java. It allows the programmer to keep information about some entity in a single data structure and to reference that information with names.

For example, a graphics program might represent a rectangle as a structure:

```
struct rect {
    long llx;           /* X coordinate of lower-left corner */
    long lly;           /* Y coordinate of lower-left corner */
    unsigned long width; /* Width (in pixels) */
    unsigned long height; /* Height (in pixels) */
    unsigned color;      /* Coding of color */
};
```

We can declare a variable `r` of type `struct rect` and set its field values as follows:

```
struct rect r;
r.llx = r.lly = 0;
r.color = 0xFF00FF;
r.width = 10;
r.height = 20;
```

where the expression `r.llx` selects field `llx` of structure `r`.

Alternatively, we can both declare the variable and initialize its fields with a single statement:

```
struct rect r = { 0, 0, 0xFF00FF, 10, 20 };
```

It is common to pass pointers to structures from one place to another rather than copying them. For example, the following function computes the area of a rectangle, where a pointer to the rectangle struct is passed to the function:

```
long area(struct rect *rp) {
    return (*rp).width * (*rp).height;
}
```

The expression `(*rp).width` dereferences the pointer and selects the width field of the resulting structure. Parentheses are required, because the compiler would interpret the expression `*rp.width` as `*(rp.width)`, which is not valid. This combination of dereferencing and field selection is so common that C provides an alternative notation using `->`. That is, `rp->width` is equivalent to the expression `(*rp).width`. For example, we can write a function that rotates a rectangle counterclockwise by 90 degrees as

```
void rotate_left(struct rect *rp) {
    /* Exchange width and height */
    long t = rp->height;
    rp->height = rp->width;
    rp->width = t;
    /* Shift to new lower-left corner */
    rp->llx -= t;
}
```

New to C? Representing an object as a struct (*continued*)

The objects of C++ and Java are more elaborate than structures in C, in that they also associate a set of *methods* with an object that can be invoked to perform computation. In C, we would simply write these as ordinary functions, such as the functions `area` and `rotate_left` shown previously.

of type `struct rec *` is in register `%rdi`. Then the following code copies element `r->i` to element `r->j`:

```
Registers: r in %rdi
1    movl    (%rdi), %eax      Get r->i
2    movl    %eax, 4(%rdi)     Store in r->j
```

Since the offset of field `i` is 0, the address of this field is simply the value of `r`. To store into field `j`, the code adds offset 4 to the address of `r`.

To generate a pointer to an object within a structure, we can simply add the field's offset to the structure address. For example, we can generate the pointer `&(r->a[1])` by adding offset $8 + 4 \cdot 1 = 12$. For pointer `r` in register `%rdi` and long integer variable `i` in register `%rsi`, we can generate the pointer value `&(r->a[i])` with the single instruction

```
Registers: r in %rdi, i %rsi
1    leaq    8(%rdi,%rsi,4), %rax  Set %rax to &r->a[i]
```

As a final example, the following code implements the statement

```
r->p = &r->a[r->i + r->j];
```

starting with `r` in register `%rdi`:

```
Registers: r in %rdi
1    movl    4(%rdi), %eax      Get r->j
2    addl    (%rdi), %eax      Add r->i
3    cltq                               Extend to 8 bytes
4    leaq    8(%rdi,%rax,4), %rax  Compute &r->a[r->i + r->j]
5    movq    %rax, 16(%rdi)     Store in r->p
```

As these examples show, the selection of the different fields of a structure is handled completely at compile time. The machine code contains no information about the field declarations or the names of the fields.

Practice Problem 3.41 (solution page 343)

Consider the following structure declaration:

```
struct prob {
    int *p;
    struct {
        int x;
        int y;
    } s;
    struct prob *next;
};
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void sp_init(struct prob *sp) {
    sp->s.x = _____;
    sp->p   = _____;
    sp->next = _____;
}
```

A. What are the offsets (in bytes) of the following fields?

```
p: _____
s.x: _____
s.y: _____
next: _____
```

B. How many total bytes does the structure require?

C. The compiler generates the following assembly code for `sp_init`:

```
void sp_init(struct prob *sp)
sp in %rdi
1  sp_init:
2      movl    12(%rdi), %eax
3      movl    %eax, 8(%rdi)
4      leaq    8(%rdi), %rax
5      movq    %rax, (%rdi)
6      movq    %rdi, 16(%rdi)
7      ret
```

On the basis of this information, fill in the missing expressions in the code for `sp_init`.

Practice Problem 3.42 (solution page 343)

The following code shows the declaration of a structure of type `ELE` and the prototype for a function `fun`:

```
struct ELE {
    long    v;
    struct ELE *p;
};

long fun(struct ELE *ptr);
```

When the code for `fun` is compiled, gcc generates the following assembly code:

```
    long fun(struct ELE *ptr)
    ptr in %rdi
1   fun:
2       movl    $0, %eax
3       jmp     .L2
4   .L3:
5       addq    (%rdi), %rax
6       movq    8(%rdi), %rdi
7   .L2:
8       testq   %rdi, %rdi
9       jne     .L3
10      rep; ret
```

- A. Use your reverse engineering skills to write C code for `fun`.
 - B. Describe the data structure that this structure implements and the operation performed by `fun`.
-

3.9.2 Unions

Unions provide a way to circumvent the type system of C, allowing a single object to be referenced according to multiple types. The syntax of a union declaration is identical to that for structures, but its semantics are very different. Rather than having the different fields reference different blocks of memory, they all reference the same block.

Consider the following declarations:

```
struct S3 {
    char c;
    int i[2];
    double v;
};
```



```
union U3 {
    char c;
    int i[2];
    double v;
};
```

When compiled on an x86-64 Linux machine, the offsets of the fields, as well as the total size of data types S3 and U3, are as shown in the following table:

Type	c	i	v	Size
S3	0	4	16	24
U3	0	0	0	8

(We will see shortly why `i` has offset 4 in S3 rather than 1, and why `v` has offset 16, rather than 9 or 12.) For pointer `p` of type union U3 *, references `p->c`, `p->i[0]`, and `p->v` would all reference the beginning of the data structure. Observe also that the overall size of a union equals the maximum size of any of its fields.

Unions can be useful in several contexts. However, they can also lead to nasty bugs, since they bypass the safety provided by the C type system. One application is when we know in advance that the use of two different fields in a data structure will be mutually exclusive. Then, declaring these two fields as part of a union rather than a structure will reduce the total space allocated.

For example, suppose we want to implement a binary tree data structure where each leaf node has two double data values and each internal node has pointers to two children but no data. If we declare this as

```
struct node_s {
    struct node_s *left;
    struct node_s *right;
    double data[2];
};
```

then every node requires 32 bytes, with half the bytes wasted for each type of node. On the other hand, if we declare a node as

```
union node_u {
    struct {
        union node_u *left;
        union node_u *right;
    } internal;
    double data[2];
};
```

then every node will require just 16 bytes. If `n` is a pointer to a node of type union node_u *, we would reference the data of a leaf node as `n->data[0]` and `n->data[1]`, and the children of an internal node as `n->internal.left` and `n->internal.right`.

With this encoding, however, there is no way to determine whether a given node is a leaf or an internal node. A common method is to introduce an enumerated type defining the different possible choices for the union, and then create a structure containing a tag field and the union:

```
typedef enum { N_LEAF, N_INTERNAL } nodetype_t;

struct node_t {
    nodetype_t type;
    union {
        struct {
            struct node_t *left;
            struct node_t *right;
        } internal;
        double data[2];
    } info;
};
```

This structure requires a total of 24 bytes: 4 for type, and either 8 each for `info.internal.left` and `info.internal.right` or 16 for `info.data`. As we will discuss shortly, an additional 4 bytes of padding is required between the field for type and the union elements, bringing the total structure size to $4 + 4 + 16 = 24$. In this case, the savings gain of using a union is small relative to the awkwardness of the resulting code. For data structures with more fields, the savings can be more compelling.

Unions can also be used to access the bit patterns of different data types. For example, suppose we use a simple cast to convert a value `d` of type `double` to a value `u` of type `unsigned long`:

```
unsigned long u = (unsigned long) d;
```

Value `u` will be an integer representation of `d`. Except for the case where `d` is 0.0, the bit representation of `u` will be very different from that of `d`. Now consider the following code to generate a value of type `unsigned long` from a `double`:

```
unsigned long double2bits(double d) {
    union {
        double d;
        unsigned long u;
    } temp;
    temp.d = d;
    return temp.u;
};
```

In this code, we store the argument in the union using one data type and access it using another. The result will be that `u` will have the same bit representation as `d`, including fields for the sign bit, the exponent, and the significand, as described in

Section 3.11. The numeric value of *u* will bear no relation to that of *d*, except for the case when *d* is 0.0.

When using unions to combine data types of different sizes, byte-ordering issues can become important. For example, suppose we write a procedure that will create an 8-byte double using the bit patterns given by two 4-byte unsigned values:

```
double uu2double(unsigned word0, unsigned word1)
{
    union {
        double d;
        unsigned u[2];
    } temp;

    temp.u[0] = word0;
    temp.u[1] = word1;
    return temp.d;
}
```

On a little-endian machine, such as an x86-64 processor, argument *word0* will become the low-order 4 bytes of *d*, while *word1* will become the high-order 4 bytes. On a big-endian machine, the role of the two arguments will be reversed.

Practice Problem 3.43 (solution page 344)

Suppose you are given the job of checking that a C compiler generates the proper code for structure and union access. You write the following structure declaration:

```
typedef union {
    struct {
        long    u;
        short   v;
        char    w;
    } t1;
    struct {
        int a[2];
        char *p;
    } t2;
} u_type;
```

You write a series of functions of the form

```
void get(u_type *up, type *dest) {
    *dest = expr;
}
```

with different access expressions *expr* and with destination data type *type* set according to type associated with *expr*. You then examine the code generated when compiling the functions to see if they match your expectations.

Suppose in these functions that `up` and `dest` are loaded into registers `%rdi` and `%rsi`, respectively. Fill in the following table with data type *type* and sequences of one to three instructions to compute the expression and store the result at `dest`.

<i>expr</i>	<i>type</i>	Code
<code>up->t1.u</code>	long	<code>movq (%rdi), %rax</code> <code>movq %rax, (%rsi)</code>
<code>up->t1.v</code>		
<code>&up->t1.w</code>		
<code>up->t2.a</code>		
<code>up->t2.a[up->t1.u]</code>		
<code>*up->t2.p</code>		

3.9.3 Data Alignment

Many computer systems place restrictions on the allowable addresses for the primitive data types, requiring that the address for some objects must be a multiple of some value *K* (typically 2, 4, or 8). Such *alignment restrictions* simplify the design of the hardware forming the interface between the processor and the memory system. For example, suppose a processor always fetches 8 bytes from memory with an address that must be a multiple of 8. If we can guarantee that any double will be aligned to have its address be a multiple of 8, then the value can be read or written with a single memory operation. Otherwise, we may need to perform two memory accesses, since the object might be split across two 8-byte memory blocks.

The x86-64 hardware will work correctly regardless of the alignment of data. However, Intel recommends that data be aligned to improve memory system performance. Their alignment rule is based on the principle that any primitive object of *K* bytes must have an address that is a multiple of *K*. We can see that this rule leads to the following alignments:

<i>K</i>	Types
1	char
2	short
4	int, float
8	long, double, char *

Alignment is enforced by making sure that every data type is organized and allocated in such a way that every object within the type satisfies its alignment restrictions. The compiler places directives in the assembly code indicating the desired alignment for global data. For example, the assembly-code declaration of the jump table on page 235 contains the following directive on line 2:

```
.align 8
```

This ensures that the data following it (in this case the start of the jump table) will start with an address that is a multiple of 8. Since each table entry is 8 bytes long, the successive elements will obey the 8-byte alignment restriction.

For code involving structures, the compiler may need to insert gaps in the field allocation to ensure that each structure element satisfies its alignment requirement. The structure will then have some required alignment for its starting address.

For example, consider the structure declaration

```
struct S1 {
    int i;
    char c;
    int j;
};
```

Suppose the compiler used the minimal 9-byte allocation, diagrammed as follows:

Offset	0		4	5		9
Contents	i		c	j		

Then it would be impossible to satisfy the 4-byte alignment requirement for both fields *i* (offset 0) and *j* (offset 5). Instead, the compiler inserts a 3-byte gap (shown here as shaded in blue) between fields *c* and *j*:

Offset	0		4	5		8		12
Contents	i		c				j	

As a result, *j* has offset 8, and the overall structure size is 12 bytes. Furthermore, the compiler must ensure that any pointer *p* of type `struct S1*` satisfies a 4-byte alignment. Using our earlier notation, let pointer *p* have value x_p . Then x_p must be a multiple of 4. This guarantees that both $p \rightarrow i$ (address x_p) and $p \rightarrow j$ (address $x_p + 8$) will satisfy their 4-byte alignment requirements.

In addition, the compiler may need to add padding to the end of the structure so that each element in an array of structures will satisfy its alignment requirement. For example, consider the following structure declaration:


```

struct S2 {
    int i;   st. 0-3
    int j;   st 4-7
    char c;  st 8
};

```

If we pack this structure into 9 bytes, we can still satisfy the alignment requirements for fields *i* and *j* by making sure that the starting address of the structure satisfies a 4-byte alignment requirement. Consider, however, the following declaration:

```
struct S2 d[4];
```

With the 9-byte allocation, it is not possible to satisfy the alignment requirement for each element of *d*, because these elements will have addresses x_d , $x_d + 9$, $x_d + 18$, and $x_d + 27$. Instead, the compiler allocates 12 bytes for structure *S2*, with the final 3 bytes being wasted space:

Offset	0	4	8	9	12
Contents	i	j	c		

That way, the elements of *d* will have addresses x_d , $x_d + 12$, $x_d + 24$, and $x_d + 36$. As long as x_d is a multiple of 4, all of the alignment restrictions will be satisfied.

Practice Problem 3.44 (solution page 345)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

- struct P1 { int i; char c; int j; char d; };
- struct P2 { int i; char c; char d; long j; };
- struct P3 { short w[3]; char c[3] };
- struct P4 { short w[5]; char *c[3] };
- struct P5 { struct P3 a[2]; struct P2 t };

Practice Problem 3.45 (solution page 345)

Answer the following for the structure declaration

```

struct {
    char    *a;
    short   b;
    double  c;
    char    d;
    float   e;
    char    f;
};

```

Aside A case of mandatory alignment

For most x86-64 instructions, keeping data aligned improves efficiency, but it does not affect program behavior. On the other hand, some models of Intel and AMD processors will not work correctly with unaligned data for some of the SSE instructions implementing multimedia operations. These instructions operate on 16-byte blocks of data, and the instructions that transfer data between the SSE unit and memory require the memory addresses to be multiples of 16. Any attempt to access memory with an address that does not satisfy this alignment will lead to an *exception* (see Section 8.1), with the default behavior for the program to terminate.

As a result, any compiler and run-time system for an x86-64 processor must ensure that any memory allocated to hold a data structure that may be read from or stored into an SSE register must satisfy a 16-byte alignment. This requirement has the following two consequences:

- The starting address for any block generated by a memory allocation function (`alloca`, `malloc`, `calloc`, or `realloc`) must be a multiple of 16.
- The stack frame for most functions must be aligned on a 16-byte boundary. (This requirement has a number of exceptions.)

More recent versions of x86-64 processors implement the AVX multimedia instructions. In addition to providing a superset of the SSE instructions, processors supporting AVX also do not have a mandatory alignment requirement.

```

        long    g;
        int     h;
    } rec;
```

- What are the byte offsets of all the fields in the structure?
 - What is the total size of the structure?
 - Rearrange the fields of the structure to minimize wasted space, and then show the byte offsets and total size for the rearranged structure.
-

3.10 Combining Control and Data in Machine-Level Programs

So far, we have looked separately at how machine-level code implements the control aspects of a program and how it implements different data structures. In this section, we look at ways in which data and control interact with each other. We start by taking a deep look into pointers, one of the most important concepts in the C programming language, but one for which many programmers only have a shallow understanding. We review the use of the symbolic debugger GDB for examining the detailed operation of machine-level programs. Next, we see how understanding machine-level programs enables us to study buffer overflow, an important security vulnerability in many real-world systems. Finally, we examine

how machine-level programs implement cases where the amount of stack storage required by a function can vary from one execution to another.

3.10.1 Understanding Pointers

Pointers are a central feature of the C programming language. They serve as a uniform way to generate references to elements within different data structures. Pointers are a source of confusion for novice programmers, but the underlying concepts are fairly simple. Here we highlight some key principles of pointers and their mapping into machine code.

- *Every pointer has an associated type.* This type indicates what kind of object the pointer points to. Using the following pointer declarations as illustrations

```
int *ip;
char **cpp;
```

variable `ip` is a pointer to an object of type `int`, while `cpp` is a pointer to an object that itself is a pointer to an object of type `char`. In general, if the object has type T , then the pointer has type $*T$. The special `void *` type represents a generic pointer. For example, the `malloc` function returns a generic pointer, which is converted to a typed pointer via either an explicit cast or by the implicit casting of the assignment operation. Pointer types are not part of machine code; they are an abstraction provided by C to help programmers avoid addressing errors.

- *Every pointer has a value.* This value is an address of some object of the designated type. The special `NULL` (0) value indicates that the pointer does not point anywhere.
- *Pointers are created with the ‘&’ operator.* This operator can be applied to any C expression that is categorized as an *lvalue*, meaning an expression that can appear on the left side of an assignment. Examples include variables and the elements of structures, unions, and arrays. We have seen that the machine-code realization of the ‘&’ operator often uses the `leaq` instruction to compute the expression value, since this instruction is designed to compute the address of a memory reference.
- *Pointers are dereferenced with the ‘*’ operator.* The result is a value having the type associated with the pointer. Dereferencing is implemented by a memory reference, either storing to or retrieving from the specified address.
- *Arrays and pointers are closely related.* The name of an array can be referenced (but not updated) as if it were a pointer variable. Array referencing (e.g., `a[3]`) has the exact same effect as pointer arithmetic and dereferencing (e.g., `*(a+3)`). Both array referencing and pointer arithmetic require scaling the offsets by the object size. When we write an expression `p+i` for pointer `p` with value p , the resulting address is computed as $p + L \cdot i$, where L is the size of the data type associated with `p`.

- *Casting from one type of pointer to another changes its type but not its value.* One effect of casting is to change any scaling of pointer arithmetic. So, for example, if p is a pointer of type `char *` having value p , then the expression `(int *) p+7` computes $p + 28$, while `(int *) (p+7)` computes $p + 7$. (Recall that casting has higher precedence than addition.)
- *Pointers can also point to functions.* This provides a powerful capability for storing and passing references to code, which can be invoked in some other part of the program. For example, if we have a function defined by the prototype

```
int fun(int x, int *p);
```

then we can declare and assign a pointer `fp` to this function by the following code sequence:

```
int (*fp)(int, int *);
fp = fun;
```

We can then invoke the function using this pointer:

```
int y = 1;
int result = fp(3, &y);
```

The value of a function pointer is the address of the first instruction in the machine-code representation of the function.

New to C? Function pointers

The syntax for declaring function pointers is especially difficult for novice programmers to understand. For a declaration such as

```
int (*f)(int*);
```

it helps to read it starting from the inside (starting with ‘`f`’) and working outward. Thus, we see that `f` is a pointer, as indicated by `(*)`. It is a pointer to a function that has a single `int *` as an argument, as indicated by `(*) (int*)`. Finally, we see that it is a pointer to a function that takes an `int *` as an argument and returns `int`.

The parentheses around `*f` are required, because otherwise the declaration

```
int *f(int*);
```

would be read as

```
(int *) f(int*);
```

That is, it would be interpreted as a function prototype, declaring a function `f` that has an `int *` as its argument and returns an `int *`.

Kernighan and Ritchie [61, Sect. 5.12] present a helpful tutorial on reading C declarations.

3.10.2 Life in the Real World: Using the GDB Debugger

The GNU debugger GDB provides a number of useful features to support the run-time evaluation and analysis of machine-level programs. With the examples and exercises in this book, we attempt to infer the behavior of a program by just looking at the code. Using GDB, it becomes possible to study the behavior by watching the program in action while having considerable control over its execution.

Figure 3.39 shows examples of some GDB commands that help when working with machine-level x86-64 programs. It is very helpful to first run `OBJDUMP` to get a disassembled version of the program. Our examples are based on running GDB on the file `prog`, described and disassembled on page 175. We start GDB with the following command line:

```
linux> gdb prog
```

The general scheme is to set breakpoints near points of interest in the program. These can be set to just after the entry of a function or at a program address. When one of the breakpoints is hit during program execution, the program will halt and return control to the user. From a breakpoint, we can examine different registers and memory locations in various formats. We can also single-step the program, running just a few instructions at a time, or we can proceed to the next breakpoint.

As our examples suggest, GDB has an obscure command syntax, but the online help information (invoked within GDB with the `help` command) overcomes this shortcoming. Rather than using the command-line interface to GDB, many programmers prefer using `DDD`, an extension to GDB that provides a graphical user interface.

3.10.3 Out-of-Bounds Memory References and Buffer Overflow

We have seen that C does not perform any bounds checking for array references, and that local variables are stored on the stack along with state information such as saved register values and return addresses. This combination can lead to serious program errors, where the state stored on the stack gets corrupted by a write to an out-of-bounds array element. When the program then tries to reload the register or execute a `ret` instruction with this corrupted state, things can go seriously wrong.

A particularly common source of state corruption is known as *buffer overflow*. Typically, some character array is allocated on the stack to hold a string, but the size of the string exceeds the space allocated for the array. This is demonstrated by the following program example:

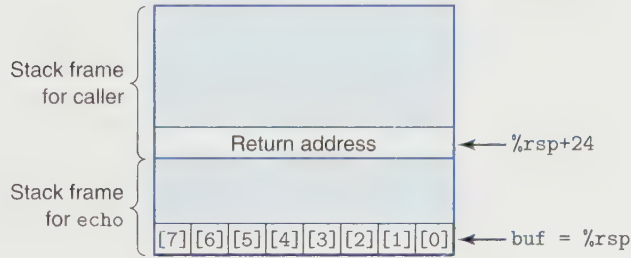
```
/* Implementation of library function gets() */
char *gets(char *s)
{
    int c;
    char *dest = s;
```


Command	Effect
Starting and stopping	
quit	Exit GDB
run	Run your program (give command-line arguments here)
kill	Stop your program
Breakpoints	
break multstore	Set breakpoint at entry to function multstore
break *0x400540	Set breakpoint at address 0x400540
delete 1	Delete breakpoint 1
delete	Delete all breakpoints
Execution	
stepi	Execute one instruction
stepi 4	Execute four instructions
nexti	Like stepi, but proceed through function calls
continue	Resume execution
finish	Run until current function returns
Examining code	
disas	Disassemble current function
disas multstore	Disassemble function multstore
disas 0x400544	Disassemble function around address 0x400544
disas 0x400540, 0x40054d	Disassemble code within specified address range
print /x \$rip	Print program counter in hex
Examining data	
print \$rax	Print contents of %rax in decimal
print /x \$rax	Print contents of %rax in hex
print /t \$rax	Print contents of %rax in binary
print 0x100	Print decimal representation of 0x100
print /x 555	Print hex representation of 555
print /x (\$rsp+8)	Print contents of %rsp plus 8 in hex
print *(long *) 0x7fffffff818	Print long integer at address 0x7fffffff818
print *(long *) (\$rsp+8)	Print long integer at address %rsp + 8
x/2g 0x7fffffff818	Examine two (8-byte) words starting at address 0x7fffffff818
x/20b multstore	Examine first 20 bytes of function multstore
Useful information	
info frame	Information about current stack frame
info registers	Values of all the registers
help	Get information about GDB

Figure 3.39 Example GDB commands. These examples illustrate some of the ways GDB supports debugging of machine-level programs.

Figure 3.40

Stack organization for echo function. Character array `buf` is just part of the saved state. An out-of-bounds write to `buf` can corrupt the program state.



```
while ((c = getchar()) != '\n' && c != EOF)
    *dest++ = c;
if (c == EOF && dest == s)
    /* No characters read */
    return NULL;
*dest++ = '\0'; /* Terminate string */
return s;
}
```

```
/* Read input line and write it back */
void echo()
{
    char buf[8]; /* Way too small! */
    gets(buf);
    puts(buf);
}
```

The preceding code shows an implementation of the library function `gets` to demonstrate a serious problem with this function. It reads a line from the standard input, stopping when either a terminating newline character or some error condition is encountered. It copies this string to the location designated by argument `s` and terminates the string with a null character. We show the use of `gets` in the function `echo`, which simply reads a line from standard input and echos it back to standard output.

The problem with `gets` is that it has no way to determine whether sufficient space has been allocated to hold the entire string. In our `echo` example, we have purposely made the buffer very small—just eight characters long. Any string longer than seven characters will cause an out-of-bounds write.

By examining the assembly code generated by GCC for `echo`, we can infer how the stack is organized:

```
void echo()
1  echo:
2      subq    $24, %rsp        Allocate 24 bytes on stack
3      movq    %rsp, %rdi      Compute buf as %rsp
4      call    gets            Call gets
5      movq    %rsp, %rdi      Compute buf as %rsp
```

```
6      call    puts                Call puts
7      addq    $24, %rsp           Deallocate stack space
8      ret                                Return
```

Figure 3.40 illustrates the stack organization during the execution of `echo`. The program allocates 24 bytes on the stack by subtracting 24 from the stack pointer (line 2). Character `buf` is positioned at the top of the stack, as can be seen by the fact that `%rsp` is copied to `%rdi` to be used as the argument to the calls to both `gets` and `puts`. The 16 bytes between `buf` and the stored return pointer are not used. As long as the user types at most seven characters, the string returned by `gets` (including the terminating null) will fit within the space allocated for `buf`. A longer string, however, will cause `gets` to overwrite some of the information stored on the stack. As the string gets longer, the following information will get corrupted:

Characters typed	Additional corrupted state
0–7	None
9–23	Unused stack space
24–31	Return address
32+	Saved state in caller

No serious consequence occurs for strings of up to 23 characters, but beyond that, the value of the return pointer, and possibly additional saved state, will be corrupted. If the stored value of the return address is corrupted, then the `ret` instruction (line 8) will cause the program to jump to a totally unexpected location. None of these behaviors would seem possible based on the C code. The impact of out-of-bounds writing to memory by functions such as `gets` can only be understood by studying the program at the machine-code level.

Our code for `echo` is simple but sloppy. A better version involves using the function `fgets`, which includes as an argument a count on the maximum number of bytes to read. Problem 3.71 asks you to write an `echo` function that can handle an input string of arbitrary length. In general, using `gets` or any function that can overflow storage is considered a bad programming practice. Unfortunately, a number of commonly used library functions, including `strcpy`, `strcat`, and `sprintf`, have the property that they can generate a byte sequence without being given any indication of the size of the destination buffer [97]. Such conditions can lead to vulnerabilities to buffer overflow.

Practice Problem 3.46 (solution page 346)

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string

0123456789012345678901234

(a) C code

```

/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}

```

(b) Disassembly up through call to gets

```

char *get_line()
1  0000000000400720 <get_line>:
2      400720: 53                push    %rbx
3      400721: 48 83 ec 10       sub     $0x10,%rsp
      Diagram stack at this point
4      400725: 48 89 e7          mov     %rsp,%rdi
5      400728: e8 73 ff ff ff   callq   4006a0 <gets>
      Modify diagram to show stack contents at this point

```

Figure 3.41 C and disassembled code for Practice Problem 3.46.

The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

- A. Fill in the diagram that follows, indicating as much as you can about the stack just after executing the instruction at line 3 in the disassembly. Label the quantities stored on the stack (e.g., “Return address”) on the right, and their hexadecimal values (if known) within the box. Each box represents 8 bytes. Indicate the position of `%rsp`. Recall that the ASCII codes for characters 0–9 are 0x30–0x39.

00 00 00 00 00 40 00 76	Return address

- B. Modify your diagram to show the effect of the call to `gets` (line 5).
- C. To what address does the program attempt to return?

- D. What register(s) have corrupted value(s) when `get_line` returns?
 - E. Besides the potential for buffer overflow, what two other things are wrong with the code for `get_line`?
-

A more pernicious use of buffer overflow is to get a program to perform a function that it would otherwise be unwilling to do. This is one of the most common methods to attack the security of a system over a computer network. Typically, the program is fed with a string that contains the byte encoding of some executable code, called the *exploit code*, plus some extra bytes that overwrite the return address with a pointer to the exploit code. The effect of executing the `ret` instruction is then to jump to the exploit code.

In one form of attack, the exploit code then uses a system call to start up a shell program, providing the attacker with a range of operating system functions. In another form, the exploit code performs some otherwise unauthorized task, repairs the damage to the stack, and then executes `ret` a second time, causing an (apparently) normal return to the caller.

As an example, the famous Internet worm of November 1988 used four different ways to gain access to many of the computers across the Internet. One was a buffer overflow attack on the finger daemon `fingerd`, which serves requests by the `FINGER` command. By invoking `FINGER` with an appropriate string, the worm could make the daemon at a remote site have a buffer overflow and execute code that gave the worm access to the remote system. Once the worm gained access to a system, it would replicate itself and consume virtually all of the machine's computing resources. As a consequence, hundreds of machines were effectively paralyzed until security experts could determine how to eliminate the worm. The author of the worm was caught and prosecuted. He was sentenced to 3 years probation, 400 hours of community service, and a \$10,500 fine. Even to this day, however, people continue to find security leaks in systems that leave them vulnerable to buffer overflow attacks. This highlights the need for careful programming. Any interface to the external environment should be made "bulletproof" so that no behavior by an external agent can cause the system to misbehave.

3.10.4 Thwarting Buffer Overflow Attacks

Buffer overflow attacks have become so pervasive and have caused so many problems with computer systems that modern compilers and operating systems have implemented mechanisms to make it more difficult to mount these attacks and to limit the ways by which an intruder can seize control of a system via a buffer overflow attack. In this section, we will present mechanisms that are provided by recent versions of GCC for Linux.

Stack Randomization

In order to insert exploit code into a system, the attacker needs to inject both the code as well as a pointer to this code as part of the attack string. Generating

Aside Worms and viruses

Both worms and viruses are pieces of code that attempt to spread themselves among computers. As described by Spafford [105], a *worm* is a program that can run by itself and can propagate a fully working version of itself to other machines. A *virus* is a piece of code that adds itself to other programs, including operating systems. It cannot run independently. In the popular press, the term “virus” is used to refer to a variety of different strategies for spreading attacking code among systems, and so you will hear people saying “virus” for what more properly should be called a “worm.”

this pointer requires knowing the stack address where the string will be located. Historically, the stack addresses for a program were highly predictable. For all systems running the same combination of program and operating system version, the stack locations were fairly stable across many machines. So, for example, if an attacker could determine the stack addresses used by a common Web server, it could devise an attack that would work on many machines. Using infectious disease as an analogy, many systems were vulnerable to the exact same strain of a virus, a phenomenon often referred to as a *security monoculture* [96].

The idea of *stack randomization* is to make the position of the stack vary from one run of a program to another. Thus, even if many machines are running identical code, they would all be using different stack addresses. This is implemented by allocating a random amount of space between 0 and n bytes on the stack at the start of a program, for example, by using the allocation function `alloca`, which allocates space for a specified number of bytes on the stack. This allocated space is not used by the program, but it causes all subsequent stack locations to vary from one execution of a program to another. The allocation range n needs to be large enough to get sufficient variations in the stack addresses, yet small enough that it does not waste too much space in the program.

The following code shows a simple way to determine a “typical” stack address:

```
int main() {
    long local;
    printf("local at %p\n", &local);
    return 0;
}
```

This code simply prints the address of a local variable in the `main` function. Running the code 10,000 times on a Linux machine in 32-bit mode, the addresses ranged from `0xff7fc59c` to `0xffffd09c`, a range of around 2^{23} . Running in 64-bit mode on the newer machine, the addresses ranged from `0x7fff0001b698` to `0x7fffffaa4a8`, a range of nearly 2^{32} .

Stack randomization has become standard practice in Linux systems. It is one of a larger class of techniques known as *address-space layout randomization*, or ASLR [99]. With ASLR, different parts of the program, including program code, library code, stack, global variables, and heap data, are loaded into different

regions of memory each time a program is run. That means that a program running on one machine will have very different address mappings than the same program running on other machines. This can thwart some forms of attack.

Overall, however, a persistent attacker can overcome randomization by brute force, repeatedly attempting attacks with different addresses. A common trick is to include a long sequence of `nop` (pronounced “no op,” short for “no operation”) instructions before the actual exploit code. Executing this instruction has no effect, other than incrementing the program counter to the next instruction. As long as the attacker can guess an address somewhere within this sequence, the program will run through the sequence and then hit the exploit code. The common term for this sequence is a “nop sled” [97], expressing the idea that the program “slides” through the sequence. If we set up a 256-byte nop sled, then the randomization over $n = 2^{23}$ can be cracked by enumerating $2^{15} = 32,768$ starting addresses, which is entirely feasible for a determined attacker. For the 64-bit case, trying to enumerate $2^{24} = 16,777,216$ is a bit more daunting. We can see that stack randomization and other aspects of ASLR can increase the effort required to successfully attack a system, and therefore greatly reduce the rate at which a virus or worm can spread, but it cannot provide a complete safeguard.

Practice Problem 3.47 (solution page 347)

Running our stack-checking code 10,000 times on a system running Linux version 2.6.16, we obtained addresses ranging from a minimum of `0xfffffb754` to a maximum of `0xffffd754`.

- A. What is the approximate range of addresses?
- B. If we attempted a buffer overrun with a 128-byte nop sled, about how many attempts would it take to test all starting addresses?

Stack Corruption Detection

A second line of defense is to be able to detect when a stack has been corrupted. We saw in the example of the `echo` function (Figure 3.40) that the corruption typically occurs when the program overruns the bounds of a local buffer. In C, there is no reliable way to prevent writing beyond the bounds of an array. Instead, the program can attempt to detect when such a write has occurred before it can have any harmful effects.

Recent versions of `gcc` incorporate a mechanism known as a *stack protector* into the generated code to detect buffer overruns. The idea is to store a special *canary* value⁴ in the stack frame between any local buffer and the rest of the stack state, as illustrated in Figure 3.42 [26, 97]. This canary value, also referred to as a *guard value*, is generated randomly each time the program runs, and so there is no

4. The term “canary” refers to the historic use of these birds to detect the presence of dangerous gases in coal mines.

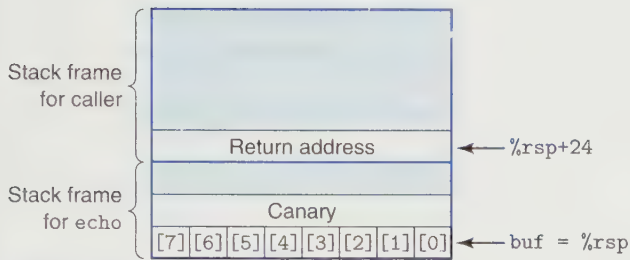


Figure 3.42 Stack organization for `echo` function with stack protector enabled. A special “canary” value is positioned between array `buf` and the saved state. The code checks the canary value to determine whether or not the stack state has been corrupted.

easy way for an attacker to determine what it is. Before restoring the register state and returning from the function, the program checks if the canary has been altered by some operation of this function or one that it has called. If so, the program aborts with an error.

Recent versions of gcc try to determine whether a function is vulnerable to a stack overflow and insert this type of overflow detection automatically. In fact, for our earlier demonstration of stack overflow, we had to give the command-line option `-fno-stack-protector` to prevent gcc from inserting this code. Compiling the function `echo` without this option, and hence with the stack protector enabled, gives the following assembly code:

```
void echo()
1  echo:
2      subq    $24, %rsp           Allocate 24 bytes on stack
3      movq    %fs:40, %rax        Retrieve canary
4      movq    %rax, 8(%rsp)       Store on stack
5      xorl    %eax, %eax         Zero out register
6      movq    %rsp, %rdi         Compute buf as %rsp
7      call    gets               Call gets
8      movq    %rsp, %rdi         Compute buf as %rsp
9      call    puts               Call puts
10     movq    8(%rsp), %rax       Retrieve canary
11     xorq    %fs:40, %rax        Compare to stored value
12     je      .L9                 If =, goto ok
13     call    __stack_chk_fail    Stack corrupted!
14  .L9:                                ok:
15     addq    $24, %rsp           Deallocate stack space
16     ret
```

We see that this version of the function retrieves a value from memory (line 3) and stores it on the stack at offset 8 from `%rsp`, just beyond the region allocated for `buf`. The instruction argument `%fs:40` is an indication that the canary value is read from memory using *segmented addressing*, an addressing mechanism that dates

back to the 80286 and is seldom found in programs running on modern systems. By storing the canary in a special segment, it can be marked as “read only,” so that an attacker cannot overwrite the stored canary value. Before restoring the register state and returning, the function compares the value stored at the stack location with the canary value (via the `xorq` instruction on line 11). If the two are identical, the `xorq` instruction will yield zero, and the function will complete in the normal fashion. A nonzero value indicates that the canary on the stack has been modified, and so the code will call an error routine.

Stack protection does a good job of preventing a buffer overflow attack from corrupting state stored on the program stack. It incurs only a small performance penalty, especially because gcc only inserts it when there is a local buffer of type `char` in the function. Of course, there are other ways to corrupt the state of an executing program, but reducing the vulnerability of the stack thwarts many common attack strategies.

Practice Problem 3.48 (solution page 347)

The functions `intlen`, `len`, and `iptoa` provide a very convoluted way to compute the number of decimal digits required to represent an integer. We will use this as a way to study some aspects of the gcc stack protector facility.

```
int len(char *s) {
    return strlen(s);
}

void iptoa(char *s, long *p) {
    long val = *p;
    sprintf(s, "%ld", val);
}

int intlen(long x) {
    long v;
    char buf[12];
    v = x;
    iptoa(buf, &v);
    return len(buf);
}
```

The following show portions of the code for `intlen`, compiled both with and without stack protector:

(a) Without protector

```
int intlen(long x)
x in %rdi
1  intlen:
2      subq    $40, %rsp
3      movq    %rdi, 24(%rsp)
```

```

4    leaq    24(%rsp), %rsi
5    movq    %rsp, %rdi
6    call    iptoa

```

(b) With protector

```

    int intlen(long x)
    x in %rdi
1   intlen:
2       subq    $56, %rsp
3       movq    %fs:40, %rax
4       movq    %rax, 40(%rsp)
5       xorl    %eax, %eax
6       movq    %rdi, 8(%rsp)
7       leaq    8(%rsp), %rsi
8       leaq    16(%rsp), %rdi
9       call    iptoa

```

- A. For both versions: What are the positions in the stack frame for `buf`, `v`, and (when present) the canary value?
 - B. How does the rearranged ordering of the local variables in the protected code provide greater security against a buffer overrun attack?
-

Limiting Executable Code Regions

A final step is to eliminate the ability of an attacker to insert executable code into a system. One method is to limit which memory regions hold executable code. In typical programs, only the portion of memory holding the code generated by the compiler need be executable. The other portions can be restricted to allow just reading and writing. As we will see in Chapter 9, the virtual memory space is logically divided into *pages*, typically with 2,048 or 4,096 bytes per page. The hardware supports different forms of *memory protection*, indicating the forms of access allowed by both user programs and the operating system kernel. Many systems allow control over three forms of access: read (reading data from memory), write (storing data into memory), and execute (treating the memory contents as machine-level code). Historically, the x86 architecture merged the read and execute access controls into a single 1-bit flag, so that any page marked as readable was also executable. The stack had to be kept both readable and writable, and therefore the bytes on the stack were also executable. Various schemes were implemented to be able to limit some pages to being readable but not executable, but these generally introduced significant inefficiencies.

More recently, AMD introduced an NX (for “no-execute”) bit into the memory protection for its 64-bit processors, separating the read and execute access modes, and Intel followed suit. With this feature, the stack can be marked as being readable and writable, but not executable, and the checking of whether a page is executable is performed in hardware, with no penalty in efficiency.

Some types of programs require the ability to dynamically generate and execute code. For example, “just-in-time” compilation techniques dynamically generate code for programs written in interpreted languages, such as Java, to improve execution performance. Whether or not the run-time system can restrict the executable code to just that part generated by the compiler in creating the original program depends on the language and the operating system.

The techniques we have outlined—randomization, stack protection, and limiting which portions of memory can hold executable code—are three of the most common mechanisms used to minimize the vulnerability of programs to buffer overflow attacks. They all have the properties that they require no special effort on the part of the programmer and incur very little or no performance penalty. Each separately reduces the level of vulnerability, and in combination they become even more effective. Unfortunately, there are still ways to attack computers [85, 97], and so worms and viruses continue to compromise the integrity of many machines.

3.10.5 Supporting Variable-Size Stack Frames

We have examined the machine-level code for a variety of functions so far, but they all have the property that the compiler can determine in advance the amount of space that must be allocated for their stack frames. Some functions, however, require a variable amount of local storage. This can occur, for example, when the function calls `alloca`, a standard library function that can allocate an arbitrary number of bytes of storage on the stack. It can also occur when the code declares a local array of variable size.

Although the information presented in this section should rightfully be considered an aspect of how procedures are implemented, we have deferred the presentation to this point, since it requires an understanding of arrays and alignment.

The code of Figure 3.43(a) gives an example of a function containing a variable-size array. The function declares local array `p` of n pointers, where n is given by the first argument. This requires allocating $8n$ bytes on the stack, where the value of n may vary from one call of the function to another. The compiler therefore cannot determine how much space it must allocate for the function’s stack frame. In addition, the program generates a reference to the address of local variable `i`, and so this variable must also be stored on the stack. During execution, the program must be able to access both local variable `i` and the elements of array `p`. On returning, the function must deallocate the stack frame and set the stack pointer to the position of the stored return address.

To manage a variable-size stack frame, x86-64 code uses register `%rbp` to serve as a *frame pointer* (sometimes referred to as a *base pointer*, and hence the letters `bp` in `%rbp`). When using a frame pointer, the stack frame is organized as shown for the case of function `vframe` in Figure 3.44. We see that the code must save the previous version of `%rbp` on the stack, since it is a callee-saved register. It then keeps `%rbp` pointing to this position throughout the execution of the function, and it references fixed-length local variables, such as `i`, at offsets relative to `%rbp`.

(a) C code

```

long vframe(long n, long idx, long *q) {
    long i;
    long *p[n];
    p[0] = &i;
    for (i = 1; i < n; i++)
        p[i] = q;
    return *p[idx];
}

```

(b) Portions of generated assembly code

```

    long vframe(long n, long idx, long *q)
    n in %rdi, idx in %rsi, q in %rdx
    Only portions of code shown
1   vframe:
2       pushq    %rbp                                Save old %rbp
3       movq     %rsp, %rbp                          Set frame pointer
4       subq     $16, %rsp                          Allocate space for i (%rsp = s1)
5       leaq     22(,%rdi,8), %rax
6       andq     $-16, %rax
7       subq     %rax, %rsp                          Allocate space for array p (%rsp = s2)
8       leaq     7(%rsp), %rax
9       shrq     $3, %rax
10      leaq     0(,%rax,8), %r8                      Set %r8 to &p[0]
11      movq     %r8, %rcx                          Set %rcx to &p[0] (%rcx = p)
    . . .
    Code for initialization loop
    i in %rax and on stack, n in %rdi, p in %rcx, q in %rdx
12     .L3:                                          loop:
13         movq     %rdx, (%rcx,%rax,8)              Set p[i] to q
14         addq     $1, %rax                        Increment i
15         movq     %rax, -8(%rbp)                  Store on stack
16     .L2:
17         movq     -8(%rbp), %rax                  Retrieve i from stack
18         cmpq     %rdi, %rax                      Compare i:n
19         jl       .L3                            If <, goto loop
    . . .
    Code for function exit
20     leave                                Restore %rbp and %rsp
21     ret                                Return

```

Figure 3.43 Function requiring the use of a frame pointer. The variable-size array implies that the size of the stack frame cannot be determined at compile time.

Figure 3.44

Stack frame structure for function `vframe`.

The function uses register `%rbp` as a frame pointer. The annotations along the right-hand side are in reference to Practice Problem 3.49.

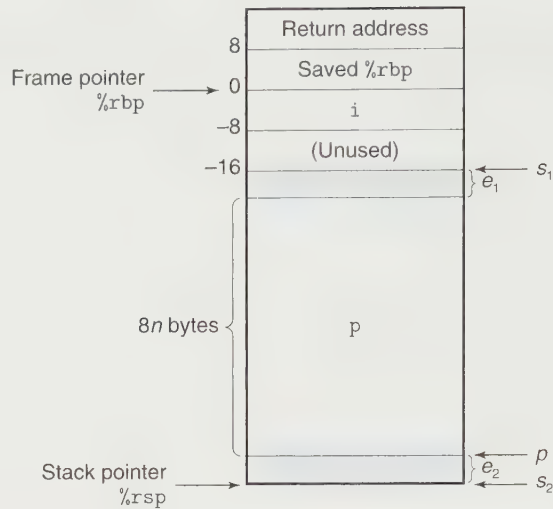


Figure 3.43(b) shows portions of the code gcc generates for function `vframe`. At the beginning of the function, we see code that sets up the stack frame and allocates space for array `p`. The code starts by pushing the current value of `%rbp` onto the stack and setting `%rbp` to point to this stack position (lines 2–3). Next, it allocates 16 bytes on the stack, the first 8 of which are used to store local variable `i`, and the second 8 of which are unused. Then it allocates space for array `p` (lines 5–11). The details of how much space it allocates and where it positions `p` within this space are explored in Practice Problem 3.49. Suffice it to say that by the time the program reaches line 11, it has (1) allocated at least $8n$ bytes on the stack and (2) positioned array `p` within the allocated region such that at least $8n$ bytes are available for its use.

The code for the initialization loop shows examples of how local variables `i` and `p` are referenced. Line 13 shows array element `p[i]` being set to `q`. This instruction uses the value in register `%rcx` as the address for the start of `p`. We can see instances where local variable `i` is updated (line 15) and read (line 17). The address of `i` is given by reference `-8(%rbp)`—that is, at offset `-8` relative to the frame pointer.

At the end of the function, the frame pointer is restored to its previous value using the `leave` instruction (line 20). This instruction takes no arguments. It is equivalent to executing the following two instructions:

```
movq %rbp, %rsp    Set stack pointer to beginning of frame
popq %rbp          Restore saved %rbp and set stack ptr
                   to end of caller's frame
```

That is, the stack pointer is first set to the position of the saved value of `%rbp`, and then this value is popped from the stack into `%rbp`. This instruction combination has the effect of deallocating the entire stack frame.

In earlier versions of x86 code, the frame pointer was used with every function call. With x86-64 code, it is used only in cases where the stack frame may be of variable size, as is the case for function `vframe`. Historically, most compilers used frame pointers when generating IA32 code. Recent versions of gcc have dropped this convention. Observe that it is acceptable to mix code that uses frame pointers with code that does not, as long as all functions treat `%rbp` as a callee-saved register.

Practice Problem 3.49 (solution page 347)

In this problem, we will explore the logic behind the code in lines 5–11 of Figure 3.43(b), where space is allocated for variable-size array `p`. As the annotations of the code indicate, let us let s_1 denote the address of the stack pointer after executing the `subq` instruction of line 4. This instruction allocates the space for local variable `i`. Let s_2 denote the value of the stack pointer after executing the `subq` instruction of line 7. This instruction allocates the storage for local array `p`. Finally, let p denote the value assigned to registers `%r8` and `%rcx` in the instructions of lines 10–11. Both of these registers are used to reference array `p`.

The right-hand side of Figure 3.44 diagrams the positions of the locations indicated by s_1 , s_2 , and p . It also shows that there may be an offset of e_2 bytes between the values of s_1 and p . This space will not be used. There may also be an offset of e_1 bytes between the end of array `p` and the position indicated by s_1 .

- Explain, in mathematical terms, the logic in the computation of s_2 on lines 5–7. *Hint:* Think about the bit-level representation of -16 and its effect in the `andq` instruction of line 6.
- Explain, in mathematical terms, the logic in the computation of p on lines 8–10. *Hint:* You may want to refer to the discussion on division by powers of 2 in Section 2.3.7.
- For the following values of n and s_1 , trace the execution of the code to determine what the resulting values would be for s_2 , p , e_1 , and e_2 .

n	s_1	s_2	p	e_1	e_2
5	2,065				
6	2,064				

- What alignment properties does this code guarantee for the values of s_2 and p ?

3.11 Floating-Point Code

The *floating-point architecture* for a processor consists of the different aspects that affect how programs operating on floating-point data are mapped onto the machine, including

- How floating-point values are stored and accessed. This is typically via some form of registers.

- The instructions that operate on floating-point data.
- The conventions used for passing floating-point values as arguments to functions and for returning them as results.
- The conventions for how registers are preserved during function calls—for example, with some registers designated as caller saved, and others as callee saved.

To understand the x86-64 floating-point architecture, it is helpful to have a brief historical perspective. Since the introduction of the Pentium/MMX in 1997, both Intel and AMD have incorporated successive generations of *media* instructions to support graphics and image processing. These instructions originally focused on allowing multiple operations to be performed in a parallel mode known as *single instruction, multiple data*, or *SIMD* (pronounced sim-dee). In this mode the same operation is performed on a number of different data values in parallel. Over the years, there has been a progression of these extensions. The names have changed through a series of major revisions from MMX to SSE (for “streaming SIMD extensions”) and most recently AVX (for “advanced vector extensions”). Within each generation, there have also been different versions. Each of these extensions manages data in sets of registers, referred to as “MM” registers for MMX, “XMM” for SSE, and “YMM” for AVX, ranging from 64 bits for MM registers, to 128 for XMM, to 256 for YMM. So, for example, each YMM register can hold eight 32-bit values, or four 64-bit values, where these values can be either integer or floating point.

Starting with SSE2, introduced with the Pentium 4 in 2000, the media instructions have included ones to operate on *scalar* floating-point data, using single values in the low-order 32 or 64 bits of XMM or YMM registers. This scalar mode provides a set of registers and instructions that are more typical of the way other processors support floating point. All processors capable of executing x86-64 code support SSE2 or higher, and hence x86-64 floating point is based on SSE or AVX, including conventions for passing procedure arguments and return values [77].

Our presentation is based on AVX2, the second version of AVX, introduced with the Core i7 Haswell processor in 2013. Gcc will generate AVX2 code when given the command-line parameter `-mavx2`. Code based on the different versions of SSE, as well as the first version of AVX, is conceptually similar, although they differ in the instruction names and formats. We present only instructions that arise in compiling floating-point programs with gcc. These are, for the most part, the scalar AVX instructions, although we document occasions where instructions intended for operating on entire data vectors arise. A more complete coverage of how to exploit the SIMD capabilities of SSE and AVX is presented in Web Aside OPT:SIMD on page 546. Readers may wish to refer to the AMD and Intel documentation for the individual instructions [4, 51]. As with integer operations, note that the ATT format we use in our presentation differs from the Intel format used in these documents. In particular, the instruction operands are listed in a different order in these two versions.



Figure 3.45 Media registers. These registers are used to hold floating-point data. Each YMM register holds 32 bytes. The low-order 16 bytes can be accessed as an XMM register.

As is illustrated in Figure 3.45, the AVX floating-point architecture allows data to be stored in 16 YMM registers, named `%ymm0`–`%ymm15`. Each YMM register is 256 bits (32 bytes) long. When operating on scalar data, these registers only hold floating-point data, and only the low-order 32 bits (for `float`) or 64 bits (for `double`) are used. The assembly code refers to the registers by their SSE XMM register names `%xmm0`–`%xmm15`, where each XMM register is the low-order 128 bits (16 bytes) of the corresponding YMM register.

Instruction	Source	Destination	Description
<code>vmovss</code>	M_{32}	X	Move single precision
<code>vmovss</code>	X	M_{32}	Move single precision
<code>vmovsd</code>	M_{64}	X	Move double precision
<code>vmovsd</code>	X	M_{64}	Move double precision
<code>vmovaps</code>	X	X	Move aligned, packed single precision
<code>vmovapd</code>	X	X	Move aligned, packed double precision

Figure 3.46 Floating-point movement instructions. These operations transfer values between memory and registers, as well as between pairs of registers. (X : XMM register (e.g., `%xmm3`); M_{32} : 32-bit memory range; M_{64} : 64-bit memory range)

3.11.1 Floating-Point Movement and Conversion Operations

Figure 3.46 shows a set of instructions for transferring floating-point data between memory and XMM registers, as well as from one XMM register to another without any conversions. Those that reference memory are *scalar* instructions, meaning that they operate on individual, rather than packed, data values. The data are held either in memory (indicated in the table as M_{32} and M_{64}) or in XMM registers (shown in the table as X). These instructions will work correctly regardless of the alignment of data, although the code optimization guidelines recommend that 32-bit memory data satisfy a 4-byte alignment and that 64-bit data satisfy an 8-byte alignment. Memory references are specified in the same way as for the integer `mov` instructions, with all of the different possible combinations of displacement, base register, index register, and scaling factor.

Gcc uses the scalar movement operations only to transfer data from memory to an XMM register or from an XMM register to memory. For transferring data between two XMM registers, it uses one of two different instructions for copying the entire contents of one XMM register to another—namely, `vmovaps` for single-precision and `vmovapd` for double-precision values. For these cases, whether the program copies the entire register or just the low-order value affects neither the program functionality nor the execution speed, and so using these instructions rather than ones specific to scalar data makes no real difference. The letter ‘a’ in these instruction names stands for “aligned.” When used to read and write memory, they will cause an exception if the address does not satisfy a 16-byte alignment. For transferring between two registers, there is no possibility of an incorrect alignment.

As an example of the different floating-point move operations, consider the C function

```
float float_mov(float v1, float *src, float *dst) {
    float v2 = *src;
    *dst = v1;
    return v2;
}
```

Instruction	Source	Destination	Description
<code>vcvttss2si</code>	X/M_{32}	R_{32}	Convert with truncation single precision to integer
<code>vcvttss2si</code>	X/M_{64}	R_{32}	Convert with truncation double precision to integer
<code>vcvttss2siq</code>	X/M_{32}	R_{64}	Convert with truncation single precision to quad word integer
<code>vcvttss2siq</code>	X/M_{64}	R_{64}	Convert with truncation double precision to quad word integer

Figure 3.47 Two-operand floating-point conversion operations. These convert floating-point data to integers. (X : XMM register (e.g., `%xmm3`); R_{32} : 32-bit general-purpose register (e.g., `%eax`); R_{64} : 64-bit general-purpose register (e.g., `%rax`); M_{32} : 32-bit memory range; M_{64} : 64-bit memory range)

Instruction	Source 1	Source 2	Destination	Description
<code>vcvtsi2ss</code>	M_{32}/R_{32}	X	X	Convert integer to single precision
<code>vcvtsi2sd</code>	M_{32}/R_{32}	X	X	Convert integer to double precision
<code>vcvtsi2ssq</code>	M_{64}/R_{64}	X	X	Convert quad word integer to single precision
<code>vcvtsi2sdq</code>	M_{64}/R_{64}	X	X	Convert quad word integer to double precision

Figure 3.48 Three-operand floating-point conversion operations. These instructions convert from the data type of the first source to the data type of the destination. The second source value has no effect on the low-order bytes of the result. (X : XMM register (e.g., `%xmm3`); M_{32} : 32-bit memory range; M_{64} : 64-bit memory range)

and its associated x86-64 assembly code

```
float float_mov(float v1, float *src, float *dst)
v1 in %xmm0, src in %rdi, dst in %rsi
1 float_mov:
2   vmovaps %xmm0, %xmm1      Copy v1
3   vmovss  (%rdi), %xmm0     Read v2 from src
4   vmovss  %xmm1, (%rsi)     Write v1 to dst
5   ret                      Return v2 in %xmm0
```

We can see in this example the use of the `vmovaps` instruction to copy data from one register to another and the use of the `vmovss` instruction to copy data from memory to an XMM register and from an XMM register to memory.

Figures 3.47 and 3.48 show sets of instructions for converting between floating-point and integer data types, as well as between different floating-point formats. These are all scalar instructions operating on individual data values. Those in Figure 3.47 convert from a floating-point value read from either an XMM register or memory and write the result to a general-purpose register (e.g., `%rax`, `%ebx`, etc.). When converting floating-point values to integers, they perform *truncation*, rounding values toward zero, as is required by C and most other programming languages.

The instructions in Figure 3.48 convert from integer to floating point. They use an unusual three-operand format, with two sources and a destination. The

first operand is read from memory or from a general-purpose register. For our purposes, we can ignore the second operand, since its value only affects the upper bytes of the result. The destination must be an XMM register. In common usage, both the second source and the destination operands are identical, as in the instruction

```
vcvtsi2sdq    %rax, %xmm1, %xmm1
```

This instruction reads a long integer from register `%rax`, converts it to data type `double`, and stores the result in the lower bytes of XMM register `%xmm1`.

Finally, for converting between two different floating-point formats, current versions of GCC generate code that requires separate documentation. Suppose the low-order 4 bytes of `%xmm0` hold a single-precision value; then it would seem straightforward to use the instruction

```
vcvtss2sd    %xmm0, %xmm0, %xmm0
```

to convert this to a double-precision value and store the result in the lower 8 bytes of register `%xmm0`. Instead, we find the following code generated by GCC:

```

Conversion from single to double precision
1  vunpcklps    %xmm0, %xmm0, %xmm0    Replicate first vector element
2  vcvtps2pd    %xmm0, %xmm0           Convert two vector elements to double

```

The `vunpcklps` instruction is normally used to interleave the values in two XMM registers and store them in a third. That is, if one source register contains words $[s_3, s_2, s_1, s_0]$ and the other contains words $[d_3, d_2, d_1, d_0]$, then the value of the destination register will be $[s_1, d_1, s_0, d_0]$. In the code above, we see the same register being used for all three operands, and so if the original register held values $[x_3, x_2, x_1, x_0]$, then the instruction will update the register to hold values $[x_1, x_1, x_0, x_0]$. The `vcvtps2pd` instruction expands the two low-order single-precision values in the source XMM register to be the two double-precision values in the destination XMM register. Applying this to the result of the preceding `vunpcklps` instruction would give values $[dx_0, dx_0]$, where dx_0 is the result of converting x to double precision. That is, the net effect of the two instructions is to convert the original single-precision value in the low-order 4 bytes of `%xmm0` to double precision and store two copies of it in `%xmm0`. It is unclear why GCC generates this code. There is neither benefit nor need to have the value duplicated within the XMM register.

GCC generates similar code for converting from double precision to single precision:

```

Conversion from double to single precision
1  vmovddup     %xmm0, %xmm0           Replicate first vector element
2  vcvtpd2psx   %xmm0, %xmm0           Convert two vector elements to single

```

Suppose these instructions start with register `%xmm0` holding two double-precision values $[x_1, x_0]$. Then the `vmovddup` instruction will set it to $[x_0, x_0]$. The `vcvtpd2psx` instruction will convert these values to single precision, pack them into the low-order half of the register, and set the upper half to 0, yielding a result $[0.0, 0.0, x_0, x_0]$ (recall that floating-point value 0.0 is represented by a bit pattern of all zeros). Again, there is no clear value in computing the conversion from one precision to another this way, rather than by using the single instruction

```
vcvtsd2ss %xmm0, %xmm0, %xmm0
```

As an example of the different floating-point conversion operations, consider the C function

```
double fcvt(int i, float *fp, double *dp, long *lp)
{
    float f = *fp; double d = *dp; long l = *lp;
    *lp = (long)    d;
    *fp = (float)   i;
    *dp = (double)  l;
    return (double) f;
}
```

and its associated x86-64 assembly code

```
double fcvt(int i, float *fp, double *dp, long *lp)
    i in %edi, fp in %rsi, dp in %rdx, lp in %rcx
1  fcvt:
2      vmovss    (%rsi), %xmm0           Get f = *fp
3      movq      (%rcx), %rax           Get l = *lp
4      vcvtt2sdq (%rdx), %r8           Get d = *dp and convert to long
5      movq      %r8, (%rcx)           Store at lp
6      vcvtsi2ss %edi, %xmm1, %xmm1     Convert i to float
7      vmovss    %xmm1, (%rsi)         Store at fp
8      vcvtsi2sdq %rax, %xmm1, %xmm1    Convert l to double
9      vmovsd    %xmm1, (%rdx)         Store at dp
    The following two instructions convert f to double
10     vunpcklps  %xmm0, %xmm0, %xmm0
11     vcvtps2pd  %xmm0, %xmm0
12     ret                               Return f
```

All of the arguments to `fcvt` are passed through the general-purpose registers, since they are either integers or pointers. The result is returned in register `%xmm0`. As is documented in Figure 3.45, this is the designated return register for float or double values. In this code, we see a number of the movement and conversion instructions of Figures 3.46–3.48, as well as gcc’s preferred method of converting from single to double precision.

Practice Problem 3.50 (solution page 347)

For the following C code, the expressions `val1–val4` all map to the program values `i`, `f`, `d`, and `l`:

```
double fcvt2(int *ip, float *fp, double *dp, long l)
{
    int i = *ip; float f = *fp; double d = *dp;
    *ip = (int)      val1;
    *fp = (float)    val2;
    *dp = (double)   val3;
    return (double) val4;
}
```

Determine the mapping, based on the following x86-64 code for the function:

```
double fcvt2(int *ip, float *fp, double *dp, long l)
ip in %rdi, fp in %rsi, dp in %rdx, l in %rcx
Result returned in %xmm0
1  fcvt2:
2      movl    (%rdi), %eax
3      vmovss  (%rsi), %xmm0
4      vcvtsd2si    (%rdx), %r8d
5      movl    %r8d, (%rdi)
6      vcvtsi2ss    %eax, %xmm1, %xmm1
7      vmovss  %xmm1, (%rsi)
8      vcvtsi2sdq    %rcx, %xmm1, %xmm1
9      vmovsd  %xmm1, (%rdx)
10     vunpcklps    %xmm0, %xmm0, %xmm0
11     vcvtps2pd    %xmm0, %xmm0
12     ret
```

Practice Problem 3.51 (solution page 348)

The following C function converts an argument of type `src_t` to a return value of type `dst_t`, where these two types are defined using `typedef`:

```
dest_t cvt(src_t x)
{
    dest_t y = (dest_t) x;
    return y;
}
```

For execution on x86-64, assume that argument `x` is either in `%xmm0` or in the appropriately named portion of register `%rdi` (i.e., `%rdi` or `%edi`). One or two instructions are to be used to perform the type conversion and to copy the value to the appropriately named portion of register `%rax` (integer result) or

%xmm0 (floating-point result). Show the instruction(s), including the source and destination registers.

T_x	T_y	Instruction(s)
long	double	vcvtsi2sdq %rdi, %xmm0
double	int	
double	float	
long	float	
float	long	

3.11.2 Floating-Point Code in Procedures

With x86-64, the XMM registers are used for passing floating-point arguments to functions and for returning floating-point values from them. As is illustrated in Figure 3.45, the following conventions are observed:

- Up to eight floating-point arguments can be passed in XMM registers %xmm0–%xmm7. These registers are used in the order the arguments are listed. Additional floating-point arguments can be passed on the stack.
- A function that returns a floating-point value does so in register %xmm0.
- All XMM registers are caller saved. The callee may overwrite any of these registers without first saving it.

When a function contains a combination of pointer, integer, and floating-point arguments, the pointers and integers are passed in general-purpose registers, while the floating-point values are passed in XMM registers. This means that the mapping of arguments to registers depends on both their types and their ordering. Here are several examples:

```
double f1(int x, double y, long z);
```

This function would have x in %edi, y in %xmm0, and z in %rsi.

```
double f2(double y, int x, long z);
```

This function would have the same register assignment as function f1.

```
double f1(float x, double *y, long *z);
```

This function would have x in %xmm0, y in %rdi, and z in %rsi.

Practice Problem 3.52 (solution page 348)

For each of the following function declarations, determine the register assignments for the arguments:

A. `double g1(double a, long b, float c, int d);`

```
B. double g2(int a, double *b, float *c, long d);  
C. double g3(double *a, double b, int c, float d);  
D. double g4(float a, int *b, float c, double d);
```

3.11.3 Floating-Point Arithmetic Operations

Figure 3.49 documents a set of scalar AVX2 floating-point instructions that perform arithmetic operations. Each has either one (S_1) or two (S_1, S_2) source operands and a destination operand D . The first source operand S_1 can be either an XMM register or a memory location. The second source operand and the destination operands must be XMM registers. Each operation has an instruction for single precision and an instruction for double precision. The result is stored in the destination register.

As an example, consider the following floating-point function:

```
double funct(double a, float x, double b, int i)  
{  
    return a*x - b/i;  
}
```

The x86-64 code is as follows:

```
double funct(double a, float x, double b, int i)  
a in %xmm0, x in %xmm1, b in %xmm2, i in %edi  
1  funct:  
    The following two instructions convert x to double  
2  vunpcklps    %xmm1, %xmm1, %xmm1  
3  vcvtps2pd    %xmm1, %xmm1  
4  vmulsd      %xmm0, %xmm1, %xmm0      Multiply a by x  
5  vcvtsi2sd    %edi, %xmm1, %xmm1      Convert i to double  
6  vdivsd      %xmm1, %xmm2, %xmm2      Compute b/i
```

Single	Double	Effect	Description
vaddss	vaddsd	$D \leftarrow S_2 + S_1$	Floating-point add
vsubss	vsubsd	$D \leftarrow S_2 - S_1$	Floating-point subtract
vmulss	vmulsd	$D \leftarrow S_2 \times S_1$	Floating-point multiply
vdivss	vdivsd	$D \leftarrow S_2 / S_1$	Floating-point divide
vmaxss	vmaxsd	$D \leftarrow \max(S_2, S_1)$	Floating-point maximum
vminss	vminsd	$D \leftarrow \min(S_2, S_1)$	Floating-point minimum
sqrtps	sqrtsd	$D \leftarrow \sqrt{S_1}$	Floating-point square root

Figure 3.49 Scalar floating-point arithmetic operations. These have either one or two source operands and a destination operand.

```

7      vsubsd  %xmm2, %xmm0, %xmm0      Subtract from a*x
8      ret                                Return

```

The three floating-point arguments *a*, *x*, and *b* are passed in XMM registers %xmm0–%xmm2, while integer argument *i* is passed in register %edi. The standard two-instruction sequence is used to convert argument *x* to double (lines 2–3). Another conversion instruction is required to convert argument *i* to double (line 5). The function value is returned in register %xmm0.

Practice Problem 3.53 (solution page 348)

For the following C function, the types of the four arguments are defined by typedef:

```

double funct1(arg1_t p, arg2_t q, arg3_t r, arg4_t s)
{
    return p/(q+r) - s;
}

```

When compiled, GCC generates the following code:

```

double funct1(arg1_t p, arg2_t q, arg3_t r, arg4_t s)
1  funct1:
2      vcvtsi2ssq    %rsi, %xmm2, %xmm2
3      vaddss  %xmm0, %xmm2, %xmm0
4      vcvtsi2ss    %edi, %xmm2, %xmm2
5      vdivss  %xmm0, %xmm2, %xmm0
6      vunpcklps    %xmm0, %xmm0, %xmm0
7      vcvtps2pd    %xmm0, %xmm0
8      vsubsd  %xmm1, %xmm0, %xmm0
9      ret

```

Determine the possible combinations of types of the four arguments (there may be more than one).

Practice Problem 3.54 (solution page 349)

Function `funct2` has the following prototype:

```
double funct2(double w, int x, float y, long z);
```

Gcc generates the following code for the function:

```

double funct2(double w, int x, float y, long z)
w in %xmm0, x in %edi, y in %xmm1, z in %rsi
1  funct2:
2      vcvtsi2ss    %edi, %xmm2, %xmm2
3      vmulss  %xmm1, %xmm2, %xmm1

```

```

4      vunpcklps      %xmm1, %xmm1, %xmm1
5      vcvtps2pd      %xmm1, %xmm2
6      vcvtsi2sdq      %rsi, %xmm1, %xmm1
7      vdivsd %xmm1, %xmm0, %xmm0
8      vsubsd %xmm0, %xmm2, %xmm0
9      ret

```

Write a C version of funct2.

3.11.4 Defining and Using Floating-Point Constants

Unlike integer arithmetic operations, AVX floating-point operations cannot have immediate values as operands. Instead, the compiler must allocate and initialize storage for any constant values. The code then reads the values from memory. This is illustrated by the following Celsius to Fahrenheit conversion function:

```

double cel2fahr(double temp)
{
    return 1.8 * temp + 32.0;
}

```

The relevant parts of the x86-64 assembly code are as follows:

```

double cel2fahr(double temp)
temp in %xmm0
1  cel2fahr:
2      vmulsd .LC2(%rip), %xmm0, %xmm0    Multiply by 1.8
3      vaddsd .LC3(%rip), %xmm0, %xmm0    Add 32.0
4      ret
5  .LC2:
6      .long  3435973837                  Low-order 4 bytes of 1.8
7      .long  1073532108                  High-order 4 bytes of 1.8
8  .LC3:
9      .long  0                          Low-order 4 bytes of 32.0
10     .long  1077936128                  High-order 4 bytes of 32.0

```

We see that the function reads the value 1.8 from the memory location labeled .LC2 and the value 32.0 from the memory location labeled .LC3. Looking at the values associated with these labels, we see that each is specified by a pair of .long declarations with the values given in decimal. How should these be interpreted as floating-point values? Looking at the declaration labeled .LC2, we see that the two values are 3435973837 (0xcccccccd) and 1073532108 (0x3ffcccc.) Since the machine uses little-endian byte ordering, the first value gives the low-order 4 bytes, while the second gives the high-order 4 bytes. From the high-order bytes, we can extract an exponent field of 0x3ff (1023), from which we subtract a bias of 1023 to get an exponent of 0. Concatenating the fraction bits of the two values, we get a fraction field of 0cccccccccccd, which can be shown to be the fractional binary representation of 0.8, to which we add the implied leading one to get 1.8.

Single	Double	Effect	Description
<code>vxorps</code>	<code>xorpd</code>	$D \leftarrow S_2 \sim S_1$	Bitwise EXCLUSIVE-OR
<code>vandps</code>	<code>andpd</code>	$D \leftarrow S_2 \& S_1$	Bitwise AND

Figure 3.50 Bitwise operations on packed data. These instructions perform Boolean operations on all 128 bits in an XMM register.

Practice Problem 3.55 (solution page 349)

Show how the numbers declared at label `.LC3` encode the number 32.0.

3.11.5 Using Bitwise Operations in Floating-Point Code

At times, we find gcc generating code that performs bitwise operations on XMM registers to implement useful floating-point results. Figure 3.50 shows some relevant instructions, similar to their counterparts for operating on general-purpose registers. These operations all act on packed data, meaning that they update the entire destination XMM register, applying the bitwise operation to all the data in the two source registers. Once again, our only interest for scalar data is the effect these instructions have on the low-order 4 or 8 bytes of the destination. These operations are often simple and convenient ways to manipulate floating-point values, as is explored in the following problem.

Practice Problem 3.56 (solution page 350)

Consider the following C function, where `EXPR` is a macro defined with `#define`:

```
double simplefun(double x) {
    return EXPR(x);
}
```

Below, we show the AVX2 code generated for different definitions of `EXPR`, where value `x` is held in `%xmm0`. All of them correspond to some useful operation on floating-point values. Identify what the operations are. Your answers will require you to understand the bit patterns of the constant words being retrieved from memory.

- A.
1 `vmovsd .LC1(%rip), %xmm1`
2 `vandpd %xmm1, %xmm0, %xmm0`
3 `.LC1:`
4 `.long 4294967295`
5 `.long 2147483647`
6 `.long 0`
7 `.long 0`
- B.
1 `vxorpd %xmm0, %xmm0, %xmm0`

```

C. 1      vmovsd  .LC2(%rip), %xmm1
    2      vxorpd %xmm1, %xmm0, %xmm0
    3      .LC2:
    4      .long  0
    5      .long  -2147483648
    6      .long  0
    7      .long  0

```

3.11.6 Floating-Point Comparison Operations

AVX2 provides two instructions for comparing floating-point values:

Instruction		Based on	Description
<code>ucomiss</code>	S_1, S_2	$S_2 - S_1$	Compare single precision
<code>ucomisd</code>	S_1, S_2	$S_2 - S_1$	Compare double precision

These instructions are similar to the `CMP` instructions (see Section 3.6), in that they compare operands S_1 and S_2 (but in the opposite order one might expect) and set the condition codes to indicate their relative values. As with `cmpq`, they follow the ATT-format convention of listing the operands in reverse order. Argument S_2 must be in an XMM register, while S_1 can be either in an XMM register or in memory.

The floating-point comparison instructions set three condition codes: the zero flag ZF, the carry flag CF, and the parity flag PF. We did not document the parity flag in Section 3.6.1, because it is not commonly found in GCC-generated x86 code. For integer operations, this flag is set when the most recent arithmetic or logical operation yielded a value where the least significant byte has even parity (i.e., an even number of ones in the byte). For floating-point comparisons, however, the flag is set when either operand is *NaN*. By convention, any comparison in C is considered to fail when one of the arguments is *NaN*, and this flag is used to detect such a condition. For example, even the comparison `x == x` yields 0 when `x` is *NaN*.

The condition codes are set as follows:

Ordering $S_2:S_1$	CF	ZF	PF
Unordered	1	1	1
$S_2 < S_1$	1	0	0
$S_2 = S_1$	0	1	0
$S_2 > S_1$	0	0	0

The *unordered* case occurs when either operand is *NaN*. This can be detected with the parity flag. Commonly, the `jp` (for “jump on parity”) instruction is used to conditionally jump when a floating-point comparison yields an unordered result. Except for this case, the values of the carry and zero flags are the same as those for an unsigned comparison: ZF is set when the two operands are equal, and CF is

(a) C code

```
typedef enum {NEG, ZERO, POS, OTHER} range_t;

range_t find_range(float x)
{
    int result;
    if (x < 0)
        result = NEG;
    else if (x == 0)
        result = ZERO;
    else if (x > 0)
        result = POS;
    else
        result = OTHER;
    return result;
}
```

(b) Generated assembly code

<i>range_t find_range(float x)</i>	
<i>x in %xmm0</i>	
1 find_range:	
2 vxorps %xmm1, %xmm1, %xmm1	<i>Set %xmm1 = 0</i>
3 vucomiss %xmm0, %xmm1	<i>Compare 0:x</i>
4 ja .L5	<i>If >, goto neg</i>
5 vucomiss %xmm1, %xmm0	<i>Compare x:0</i>
6 jp .L8	<i>If NaN, goto posornan</i>
7 movl \$1, %eax	<i>result = ZERO</i>
8 je .L3	<i>If =, goto done</i>
9 .L8:	posornan:
10 vucomiss .LC0(%rip), %xmm0	<i>Compare x:0</i>
11 setbe %al	<i>Set result = NaN ? 1 : 0</i>
12 movzbl %al, %eax	<i>Zero-extend</i>
13 addl \$2, %eax	<i>result += 2 (POS for > 0, OTHER for NaN)</i>
14 ret	<i>Return</i>
15 .L5:	neg:
16 movl \$0, %eax	<i>result = NEG</i>
17 .L3:	done:
18 rep; ret	<i>Return</i>

Figure 3.51 Illustration of conditional branching in floating-point code.

set when $S_2 < S_1$. Instructions such as `ja` and `jb` are used to conditionally jump on various combinations of these flags.

As an example of floating-point comparisons, the `C` function of Figure 3.51(a) classifies argument `x` according to its relation to 0.0, returning an enumerated type as the result. Enumerated types in `C` are encoded as integers, and so the possible function values are: 0 (NEG), 1 (ZERO), 2 (POS), and 3 (OTHER). This final outcome occurs when the value of `x` is *NaN*.

Gcc generates the code shown in Figure 3.51(b) for `find_range`. The code is not very efficient—it compares `x` to 0.0 three times, even though the required information could be obtained with a single comparison. It also generates floating-point constant 0.0 twice—once using `vxorps`, and once by reading the value from memory. Let us trace the flow of the function for the four possible comparison results:

`x < 0.0` The `ja` branch on line 4 will be taken, jumping to the end with a return value of 0.

`x = 0.0` The `ja` (line 4) and `jp` (line 6) branches will not be taken, but the `je` branch (line 8) will, returning with `%eax` equal to 1.

`x > 0.0` None of the three branches will be taken. The `setbe` (line 11) will yield 0, and this will be incremented by the `addl` instruction (line 13) to give a return value of 2.

`x = NaN` The `jp` branch (line 6) will be taken. The third `vucomiss` instruction (line 10) will set both the carry and the zero flag, and so the `setbe` instruction (line 11) and the following instruction will set `%eax` to 1. This gets incremented by the `addl` instruction (line 13) to give a return value of 3.

In Homework Problems 3.73 and 3.74, you are challenged to hand-generate more efficient implementations of `find_range`.

Practice Problem 3.57 (solution page 350)

Function `funct3` has the following prototype:

```
double funct3(int *ap, double b, long c, float *dp);
```

For this function, gcc generates the following code:

```
double funct3(int *ap, double b, long c, float *dp)
ap in %rdi, b in %xmm0, c in %rsi, dp in %rdx
1  funct3:
2      vmovss    (%rdx), %xmm1
3      vcvtsi2sd    (%rdi), %xmm2, %xmm2
4      vucomisd    %xmm2, %xmm0
5      jbe      .L8
6      vcvtsi2ssq    %rsi, %xmm0, %xmm0
7      vmulss    %xmm1, %xmm0, %xmm1
```

```

8      vunpcklps      %xmm1, %xmm1, %xmm1
9      vcvtps2pd      %xmm1, %xmm0
10     ret
11  .L8:
12     vaddss %xmm1, %xmm1, %xmm1
13     vcvtsi2ssq      %rsi, %xmm0, %xmm0
14     vaddss %xmm1, %xmm0, %xmm0
15     vunpcklps      %xmm0, %xmm0, %xmm0
16     vcvtps2pd      %xmm0, %xmm0
17     ret

```

Write a C version of `funct3`.

3.11.7 Observations about Floating-Point Code

We see that the general style of machine code generated for operating on floating-point data with AVX2 is similar to what we have seen for operating on integer data. Both use a collection of registers to hold and operate on values, and they use these registers for passing function arguments.

Of course, there are many complexities in dealing with the different data types and the rules for evaluating expressions containing a mixture of data types, and AVX2 code involves many more different instructions and formats than is usually seen with functions that perform only integer arithmetic.

AVX2 also has the potential to make computations run faster by performing parallel operations on packed data. Compiler developers are working on automating the conversion of scalar code to parallel code, but currently the most reliable way to achieve higher performance through parallelism is to use the extensions to the C language supported by gcc for manipulating vectors of data. See Web Aside OPT:SIMD on page 546 to see how this can be done.

3.12 Summary

In this chapter, we have peered beneath the layer of abstraction provided by the C language to get a view of machine-level programming. By having the compiler generate an assembly-code representation of the machine-level program, we gain insights into both the compiler and its optimization capabilities, along with the machine, its data types, and its instruction set. In Chapter 5, we will see that knowing the characteristics of a compiler can help when trying to write programs that have efficient mappings onto the machine. We have also gotten a more complete picture of how the program stores data in different memory regions. In Chapter 12, we will see many examples where application programmers need to know whether a program variable is on the run-time stack, in some dynamically allocated data structure, or part of the global program data. Understanding how programs map onto machines makes it easier to understand the differences between these kinds of storage.

Machine-level programs, and their representation by assembly code, differ in many ways from C programs. There is minimal distinction between different data types. The program is expressed as a sequence of instructions, each of which performs a single operation. Parts of the program state, such as registers and the run-time stack, are directly visible to the programmer. Only low-level operations are provided to support data manipulation and program control. The compiler must use multiple instructions to generate and operate on different data structures and to implement control constructs such as conditionals, loops, and procedures. We have covered many different aspects of C and how it gets compiled. We have seen that the lack of bounds checking in C makes many programs prone to buffer overflows. This has made many systems vulnerable to attacks by malicious intruders, although recent safeguards provided by the run-time system and the compiler help make programs more secure.

We have only examined the mapping of C onto x86-64, but much of what we have covered is handled in a similar way for other combinations of language and machine. For example, compiling C++ is very similar to compiling C. In fact, early implementations of C++ first performed a source-to-source conversion from C++ to C and generated object code by running a C compiler on the result. C++ objects are represented by structures, similar to a C struct. Methods are represented by pointers to the code implementing the methods. By contrast, Java is implemented in an entirely different fashion. The object code of Java is a special binary representation known as *Java byte code*. This code can be viewed as a machine-level program for a *virtual machine*. As its name suggests, this machine is not implemented directly in hardware. Instead, software interpreters process the byte code, simulating the behavior of the virtual machine. Alternatively, an approach known as *just-in-time compilation* dynamically translates byte code sequences into machine instructions. This approach provides faster execution when code is executed multiple times, such as in loops. The advantage of using byte code as the low-level representation of a program is that the same code can be “executed” on many different machines, whereas the machine code we have considered runs only on x86-64 machines.

Bibliographic Notes

Both Intel and AMD provide extensive documentation on their processors. This includes general descriptions of an assembly-language programmer’s view of the hardware [2, 50], as well as detailed references about the individual instructions [3, 51]. Reading the instruction descriptions is complicated by the facts that (1) all documentation is based on the Intel assembly-code format, (2) there are many variations for each instruction due to the different addressing and execution modes, and (3) there are no illustrative examples. Still, these remain the authoritative references about the behavior of each instruction.

The organization x86-64.org has been responsible for defining the *application binary interface* (ABI) for x86-64 code running on Linux systems [77]. This interface describes details for procedure linkages, binary code files, and a number of other features that are required for machine-code programs to execute properly.

As we have discussed, the ATT format used by gcc is very different from the Intel format used in Intel documentation and by other compilers (including the Microsoft compilers).

Muchnick's book on compiler design [80] is considered the most comprehensive reference on code-optimization techniques. It covers many of the techniques we discuss here, such as register usage conventions.

Much has been written about the use of buffer overflow to attack systems over the Internet. Detailed analyses of the 1988 Internet worm have been published by Spafford [105] as well as by members of the team at MIT who helped stop its spread [35]. Since then a number of papers and projects have generated ways both to create and to prevent buffer overflow attacks. Seacord's book [97] provides a wealth of information about buffer overflow and other attacks on code generated by C compilers.

Homework Problems

3.58 ♦

For a function with prototype

```
long decode2(long x, long y, long z);
```

gcc generates the following assembly code:

```
1  decode2:
2      subq    %rdx, %rsi
3      imulq   %rsi, %rdi
4      movq    %rsi, %rax
5      salq    $63, %rax
6      sarq    $63, %rax
7      xorq    %rdi, %rax
8      ret
```

Parameters x , y , and z are passed in registers `%rdi`, `%rsi`, and `%rdx`. The code stores the return value in register `%rax`.

Write C code for `decode2` that will have an effect equivalent to the assembly code shown.

3.59 ♦♦

The following code computes the 128-bit product of two 64-bit signed values x and y and stores the result in memory:

```
1  typedef __int128 int128_t;
2
3  void store_prod(int128_t *dest, int64_t x, int64_t y) {
4      *dest = x * (int128_t) y;
5  }
```

Gcc generates the following assembly code implementing the computation:

```

1  store_prod:
2      movq    %rdx, %rax
3      cqto
4      movq    %rsi, %rcx
5      sarq    $63, %rcx
6      imulq   %rax, %rcx
7      imulq   %rsi, %rdx
8      addq    %rdx, %rcx
9      mulq    %rsi
10     addq    %rcx, %rdx
11     movq    %rax, (%rdi)
12     movq    %rdx, 8(%rdi)
13     ret

```

This code uses three multiplications for the multiprecision arithmetic required to implement 128-bit arithmetic on a 64-bit machine. Describe the algorithm used to compute the product, and annotate the assembly code to show how it realizes your algorithm. *Hint:* When extending arguments of x and y to 128 bits, they can be rewritten as $x = 2^{64} \cdot x_h + x_l$ and $y = 2^{64} \cdot y_h + y_l$, where x_h, x_l, y_h , and y_l are 64-bit values. Similarly, the 128-bit product can be written as $p = 2^{64} \cdot p_h + p_l$, where p_h and p_l are 64-bit values. Show how the code computes the values of p_h and p_l in terms of x_h, x_l, y_h , and y_l .

3.60 ♦♦

Consider the following assembly code:

```

long loop(long x, int n)
x in %rdi, n in %esi
1  loop:
2      movl    %esi, %ecx
3      movl    $1, %edx
4      movl    $0, %eax
5      jmp     .L2
6  .L3:
7      movq    %rdi, %r8
8      andq    %rdx, %r8
9      orq     %r8, %rax
10     salq    %cl, %rdx
11  .L2:
12     testq   %rdx, %rdx
13     jne     .L3
14     rep; ret

```

The preceding code was generated by compiling C code that had the following overall form:

```

1  long loop(long x, long n)
2  {
3      long result = _____;
4      long mask;
5      for (mask = _____; mask _____; mask = _____) {
6          result |= _____;
7      }
8      return result;
9  }

```

Your task is to fill in the missing parts of the C code to get a program equivalent to the generated assembly code. Recall that the result of the function is returned in register %rax. You will find it helpful to examine the assembly code before, during, and after the loop to form a consistent mapping between the registers and the program variables.

- A. Which registers hold program values `x`, `n`, `result`, and `mask`?
- B. What are the initial values of `result` and `mask`?
- C. What is the test condition for `mask`?
- D. How does `mask` get updated?
- E. How does `result` get updated?
- F. Fill in all the missing parts of the C code.

3.61 ♦♦

In Section 3.6.6, we examined the following code as a candidate for the use of conditional data transfer:

```

long cread(long *xp) {
    return (xp ? *xp : 0);
}

```

We showed a trial implementation using a conditional move instruction but argued that it was not valid, since it could attempt to read from a null address.

Write a C function `cread_alt` that has the same behavior as `cread`, except that it can be compiled to use conditional data transfer. When compiled, the generated code should use a conditional move instruction rather than one of the jump instructions.

3.62 ♦♦

The code that follows shows an example of branching on an enumerated type value in a switch statement. Recall that enumerated types in C are simply a way to introduce a set of names having associated integer values. By default, the values assigned to the names count from zero upward. In our code, the actions associated with the different case labels have been omitted.

```

1  /* Enumerated type creates set of constants numbered 0 and upward */
2  typedef enum {MODE_A, MODE_B, MODE_C, MODE_D, MODE_E} mode_t;
3
4  long switch3(long *p1, long *p2, mode_t action)
5  {
6      long result = 0;
7      switch(action) {
8          case MODE_A:
9
10         case MODE_B:
11
12         case MODE_C:
13
14         case MODE_D:
15
16         case MODE_E:
17
18         default:
19
20     }
21     return result;
22 }

```

The part of the generated assembly code implementing the different actions is shown in Figure 3.52. The annotations indicate the argument locations, the register values, and the case labels for the different jump destinations.

Fill in the missing parts of the C code. It contained one case that fell through to another—try to reconstruct this.

3.63 ♦♦

This problem will give you a chance to reverse engineer a switch statement from disassembled machine code. In the following procedure, the body of the switch statement has been omitted:

```

1  long switch_prob(long x, long n) {
2      long result = x;
3      switch(n) {
4          /* Fill in code here */
5
6      }
7      return result;
8  }

```



```

    p1 in %rdi, p2 in %rsi, action in %edx
1   .L8:                                MODE_E
2       movl    $27, %eax
3       ret
4   .L3:                                MODE_A
5       movq    (%rsi), %rax
6       movq    (%rdi), %rdx
7       movq    %rdx, (%rsi)
8       ret
9   .L5:                                MODE_B
10      movq    (%rdi), %rax
11      addq    (%rsi), %rax
12      movq    %rax, (%rdi)
13      ret
14   .L6:                                MODE_C
15      movq    $59, (%rdi)
16      movq    (%rsi), %rax
17      ret
18   .L7:                                MODE_D
19      movq    (%rsi), %rax
20      movq    %rax, (%rdi)
21      movl    $27, %eax
22      ret
23   .L9:                                default
24      movl    $12, %eax
25      ret

```

Figure 3.52 Assembly code for Problem 3.62. This code implements the different branches of a switch statement.

Figure 3.53 shows the disassembled machine code for the procedure.

The jump table resides in a different area of memory. We can see from the indirect jump on line 5 that the jump table begins at address 0x4006f8. Using the GDB debugger, we can examine the six 8-byte words of memory comprising the jump table with the command `x/6gx 0x4006f8`. GDB prints the following:

```

(gdb) x/6gx 0x4006f8
0x4006f8:  0x0000000004005a1  0x0000000004005c3
0x400708:  0x0000000004005a1  0x0000000004005aa
0x400718:  0x0000000004005b2  0x0000000004005bf

```

Fill in the body of the switch statement with C code that will have the same behavior as the machine code.

```

long switch_prob(long x, long n)
x in %rdi, n in %rsi:
1  0000000000400590 <switch_prob>:
2      400590: 48 83 ee 3c          sub    $0x3c,%rsi
3      400594: 48 83 fe 05          cmp    $0x5,%rsi
4      400598: 77 29              ja     4005c3 <switch_prob+0x33>
5      40059a: ff 24 f5 f8 06 40 00 jmpq   *0x4006f8(,%rsi,8)
6      4005a1: 48 8d 04 fd 00 00 00 lea     0x0(,%rdi,8),%rax
7      4005a8: 00
8      4005a9: c3                retq
9      4005aa: 48 89 f8          mov    %rdi,%rax
10     4005ad: 48 c1 f8 03       sar    $0x3,%rax
11     4005b1: c3                retq
12     4005b2: 48 89 f8          mov    %rdi,%rax
13     4005b5: 48 c1 e0 04       shl    $0x4,%rax
14     4005b9: 48 29 f8          sub    %rdi,%rax
15     4005bc: 48 89 c7          mov    %rax,%rdi
16     4005bf: 48 0f af ff       imul   %rdi,%rdi
17     4005c3: 48 8d 47 4b       lea     0x4b(%rdi),%rax
18     4005c7: c3                retq

```

Figure 3.53 Disassembled code for Problem 3.63.

3.64 ◆◆◆

Consider the following source code, where R , S , and T are constants declared with `#define`:

```

1  long A[R][S][T];
2
3  long store_ele(long i, long j, long k, long *dest)
4  {
5      *dest = A[i][j][k];
6      return sizeof(A);
7  }

```

In compiling this program, GCC generates the following assembly code:

```

long store_ele(long i, long j, long k, long *dest)
i in %rdi, j in %rsi, k in %rdx, dest in %rcx
1  store_ele:
2      leaq    (%rsi,%rsi,2), %rax
3      leaq    (%rsi,%rax,4), %rax
4      movq    %rdi, %rsi
5      salq    $6, %rsi
6      addq    %rsi, %rdi
7      addq    %rax, %rdi

```

```

8      addq    %rdi, %rdx
9      movq    A(,%rdx,8), %rax
10     movq    %rax, (%rcx)
11     movl    $3640, %eax
12     ret

```

- A. Extend Equation 3.1 from two dimensions to three to provide a formula for the location of array element $A[i][j][k]$.
- B. Use your reverse engineering skills to determine the values of R , S , and T based on the assembly code.

3.65 ♦

The following code transposes the elements of an $M \times M$ array, where M is a constant defined by `#define`:

```

1  void transpose(long A[M][M]) {
2      long i, j;
3      for (i = 0; i < M; i++)
4          for (j = 0; j < i; j++) {
5              long t = A[i][j];
6              A[i][j] = A[j][i];
7              A[j][i] = t;
8          }
9  }

```

When compiled with optimization level `-O1`, gcc generates the following code for the inner loop of the function:

```

1  .L6:
2      movq    (%rdx), %rcx
3      movq    (%rax), %rsi
4      movq    %rsi, (%rdx)
5      movq    %rcx, (%rax)
6      addq    $8, %rdx
7      addq    $120, %rax
8      cmpq    %rdi, %rax
9      jne     .L6

```

We can see that gcc has converted the array indexing to pointer code.

- A. Which register holds a pointer to array element $A[i][j]$?
- B. Which register holds a pointer to array element $A[j][i]$?
- C. What is the value of M ?

3.66 ♦

Consider the following source code, where `NR` and `NC` are macro expressions declared with `#define` that compute the dimensions of array `A` in terms of parameter `n`. This code computes the sum of the elements of column `j` of the array.

```

1  long sum_col(long n, long A[NR(n)][NC(n)], long j) {
2      long i;
3      long result = 0;
4      for (i = 0; i < NR(n); i++)
5          result += A[i][j];
6      return result;
7  }

```

In compiling this program, GCC generates the following assembly code:

```

      long sum_col(long n, long A[NR(n)][NC(n)], long j)
      n in %rdi, A in %rsi, j in %rdx

1  sum_col:
2      leaq    1(,%rdi,4), %r8
3      leaq    (%rdi,%rdi,2), %rax
4      movq    %rax, %rdi
5      testq   %rax, %rax
6      jle     .L4
7      salq    $3, %r8
8      leaq    (%rsi,%rdx,8), %rcx
9      movl    $0, %eax
10     movl    $0, %edx
11     .L3:
12         addq    (%rcx), %rax
13         addq    $1, %rdx
14         addq    %r8, %rcx
15         cmpq    %rdi, %rdx
16         jne     .L3
17         rep; ret
18     .L4:
19         movl    $0, %eax
20         ret

```

Use your reverse engineering skills to determine the definitions of NR and NC.

3.67 ♦♦

For this exercise, we will examine the code generated by GCC for functions that have structures as arguments and return values, and from this see how these language features are typically implemented.

The following C code has a function `process` having structures as argument and return values, and a function `eval` that calls `process`:

```

1  typedef struct {
2      long a[2];
3      long *p;
4  } strA;
5

```

```

6  typedef struct {
7      long u[2];
8      long q;
9  } strB;
10
11  strB process(strA s) {
12      strB r;
13      r.u[0] = s.a[1];
14      r.u[1] = s.a[0];
15      r.q = *s.p;
16      return r;
17  }
18
19  long eval(long x, long y, long z) {
20      strA s;
21      s.a[0] = x;
22      s.a[1] = y;
23      s.p = &z;
24      strB r = process(s);
25      return r.u[0] + r.u[1] + r.q;
26  }

```

Gcc generates the following code for these two functions:

```

      strB process(strA s)
1  process:
2      movq    %rdi, %rax
3      movq    24(%rsp), %rdx
4      movq    (%rdx), %rdx
5      movq    16(%rsp), %rcx
6      movq    %rcx, (%rdi)
7      movq    8(%rsp), %rcx
8      movq    %rcx, 8(%rdi)
9      movq    %rdx, 16(%rdi)
10     ret

      long eval(long x, long y, long z)
      x in %rdi, y in %rsi, z in %rdx
1  eval:
2      subq    $104, %rsp
3      movq    %rdx, 24(%rsp)
4      leaq    24(%rsp), %rax
5      movq    %rdi, (%rsp)
6      movq    %rsi, 8(%rsp)
7      movq    %rax, 16(%rsp)
8      leaq    64(%rsp), %rdi
9      call    process

```



```

10    movq    72(%rsp), %rax
11    addq    64(%rsp), %rax
12    addq    80(%rsp), %rax
13    addq    $104, %rsp
14    ret

```

- A. We can see on line 2 of function `eval` that it allocates 104 bytes on the stack. Diagram the stack frame for `eval`, showing the values that it stores on the stack prior to calling `process`.
- B. What value does `eval` pass in its call to `process`?
- C. How does the code for `process` access the elements of structure argument `s`?
- D. How does the code for `process` set the fields of result structure `r`?
- E. Complete your diagram of the stack frame for `eval`, showing how `eval` accesses the elements of structure `r` following the return from `process`.
- F. What general principles can you discern about how structure values are passed as function arguments and how they are returned as function results?

3.68 ♦♦♦

In the following code, *A* and *B* are constants defined with `#define`:

```

1  typedef struct {
2      int x[A][B]; /* Unknown constants A and B */
3      long y;
4  } str1;
5
6  typedef struct {
7      char array[B];
8      int t;
9      short s[A];
10     long u;
11 } str2;
12
13 void setVal(str1 *p, str2 *q) {
14     long v1 = q->t;
15     long v2 = q->u;
16     p->y = v1+v2;
17 }

```

Gcc generates the following code for `setVal`:

```

void setVal(str1 *p, str2 *q)
p in %rdi, q in %rsi
1  setVal:
2      movslq 8(%rsi), %rax
3      addq   32(%rsi), %rax

```

```

4      movq    %rax, 184(%rdi)
5      ret

```

What are the values of A and B ? (The solution is unique.)

3.69 ♦♦♦

You are charged with maintaining a large C program, and you come across the following code:

```

1  typedef struct {
2      int first;
3      a_struct a[CNT];
4      int last;
5  } b_struct;
6
7  void test(long i, b_struct *bp)
8  {
9      int n = bp->first + bp->last;
10     a_struct *ap = &bp->a[i];
11     ap->x[ap->idx] = n;
12 }

```

The declarations of the compile-time constant `CNT` and the structure `a_struct` are in a file for which you do not have the necessary access privilege. Fortunately, you have a copy of the `.o` version of code, which you are able to disassemble with the `OBJDUMP` program, yielding the following disassembly:

```

void test(long i, b_struct *bp)
i in %rdi, bp in %rsi
1  0000000000000000 <test>:
2      0:  8b 8e 20 01 00 00      mov     0x120(%rsi),%ecx
3      6:  03 0e                  add     (%rsi),%ecx
4      8:  48 8d 04 bf          lea     (%rdi,%rdi,4),%rax
5      c:  48 8d 04 c6          lea     (%rsi,%rax,8),%rax
6     10:  48 8b 50 08          mov     0x8(%rax),%rdx
7     14:  48 63 c9          movslq  %ecx,%rcx
8     17:  48 89 4c d0 10      mov     %rcx,0x10(%rax,%rdx,8)
9     1c:  c3                  retq

```

Using your reverse engineering skills, deduce the following:

- A. The value of `CNT`.
- B. A complete declaration of structure `a_struct`. Assume that the only fields in this structure are `idx` and `x`, and that both of these contain signed values.

3.70 ♦♦♦

Consider the following union declaration:

```

1  union ele {
2      struct {
3          long *p;
4          long y;
5      } e1;
6      struct {
7          long x;
8          union ele *next;
9      } e2;
10 };

```

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```

1  void proc (union ele *up) {
2      up->_____ = *(___) - _____;
3  }

```

A. What are the offsets (in bytes) of the following fields:

```

e1.p
e1.y
e2.x
e2.next

```

B. How many total bytes does the structure require?

C. The compiler generates the following assembly code for `proc`:

```

      void proc (union ele *up)
      up in %rdi
1  proc:
2      movq    8(%rdi), %rax
3      movq    (%rax), %rdx
4      movq    (%rdx), %rdx
5      subq    8(%rax), %rdx
6      movq    %rdx, (%rdi)
7      ret

```

On the basis of this information, fill in the missing expressions in the code for `proc`. *Hint:* Some union references can have ambiguous interpretations. These ambiguities get resolved as you see where the references lead. There

is only one answer that does not perform any casting and does not violate any type constraints.

3.71 ♦

Write a function `good_echo` that reads a line from standard input and writes it to standard output. Your implementation should work for an input line of arbitrary length. You may use the library function `fgets`, but you must make sure your function works correctly even when the input line requires more space than you have allocated for your buffer. Your code should also check for error conditions and return when one is encountered. Refer to the definitions of the standard I/O functions for documentation [45, 61].

3.72 ♦♦

Figure 3.54(a) shows the code for a function that is similar to function `vfunc` (Figure 3.43(a)). We used `vfunc` to illustrate the use of a frame pointer in managing variable-size stack frames. The new function `aframe` allocates space for local

(a) C code

```

1  #include <alloca.h>
2
3  long aframe(long n, long idx, long *q) {
4      long i;
5      long **p = alloca(n * sizeof(long *));
6      p[0] = &i;
7      for (i = 1; i < n; i++)
8          p[i] = q;
9      return *p[idx];
10 }
```

(b) Portions of generated assembly code

```

    long aframe(long n, long idx, long *q)
    n in %rdi, idx in %rsi, q in %rdx

1  aframe:
2      pushq    %rbp
3      movq     %rsp, %rbp
4      subq     $16, %rsp                Allocate space for i (%rsp = s1)
5      leaq     30(,%rdi,8), %rax
6      andq     $-16, %rax
7      subq     %rax, %rsp                Allocate space for array p (%rsp = s2)
8      leaq     15(%rsp), %r8
9      andq     $-16, %r8                Set %r8 to &p[0]
   :
   :
```

Figure 3.54 Code for Problem 3.72. This function is similar to that of Figure 3.43.

array `p` by calling library function `alloca`. This function is similar to the more commonly used function `malloc`, except that it allocates space on the run-time stack. The space is automatically deallocated when the executing procedure returns.

Figure 3.54(b) shows the part of the assembly code that sets up the frame pointer and allocates space for local variables `i` and `p`. It is very similar to the corresponding code for `vframe`. Let us use the same notation as in Problem 3.49: The stack pointer is set to values s_1 at line 4 and s_2 at line 7. The start address of array `p` is set to value p at line 9. Extra space e_2 may arise between s_2 and p , and extra space e_1 may arise between the end of array `p` and s_1 .

- A. Explain, in mathematical terms, the logic in the computation of s_2 .
- B. Explain, in mathematical terms, the logic in the computation of p .
- C. Find values of n and s_1 that lead to minimum and maximum values of e_1 .
- D. What alignment properties does this code guarantee for the values of s_2 and p ?

3.73 ♦

Write a function in assembly code that matches the behavior of the function `find_range` in Figure 3.51. Your code should contain only one floating-point comparison instruction, and then it should use conditional branches to generate the correct result. Test your code on all 2^{32} possible argument values. Web Aside ASM:EASM on page 178 describes how to incorporate functions written in assembly code into C programs.

3.74 ♦♦

Write a function in assembly code that matches the behavior of the function `find_range` in Figure 3.51. Your code should contain only one floating-point comparison instruction, and then it should use conditional moves to generate the correct result. You might want to make use of the instruction `cmovp` (move if even parity). Test your code on all 2^{32} possible argument values. Web Aside ASM:EASM on page 178 describes how to incorporate functions written in assembly code into C programs.

3.75 ♦

ISO C99 includes extensions to support complex numbers. Any floating-point type can be modified with the keyword `complex`. Here are some sample functions that work with complex data and that call some of the associated library functions:

```

1  #include <complex.h>
2
3  double c_imag(double complex x) {
4      return cimag(x);
5  }
6
7  double c_real(double complex x) {
8      return creal(x);
9  }
10
```



```

11  double complex c_sub(double complex x, double complex y) {
12      return x - y;
13  }

```

When compiled, gcc generates the following assembly code for these functions:

```

    double c_imag(double complex x)
1  c_imag:
2      movapd  %xmm1, %xmm0
3      ret

    double c_real(double complex x)
4  c_real:
5      rep; ret

    double complex c_sub(double complex x, double complex y)
6  c_sub:
7      subsd  %xmm2, %xmm0
8      subsd  %xmm3, %xmm1
9      ret

```

Based on these examples, determine the following:

- A. How are complex arguments passed to a function?
- B. How are complex values returned from a function?

Solutions to Practice Problems

Solution to Problem 3.1 (page 182)

This exercise gives you practice with the different operand forms.

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute address
\$0x108	0x108	Immediate
(%rax)	0xFF	Address 0x100
4(%rax)	0xAB	Address 0x104
9(%rax,%rdx)	0x11	Address 0x10C
260(%rcx,%rdx)	0x13	Address 0x108
0xFC(,%rcx,4)	0xFF	Address 0x100
(%rax,%rdx,4)	0x11	Address 0x10C

Solution to Problem 3.2 (page 185)

As we have seen, the assembly code generated by gcc includes suffixes on the instructions, while the disassembler does not. Being able to switch between these

two forms is an important skill to learn. One important feature is that memory references in x86-64 are always given with quad word registers, such as `%rax`, even if the operand is a byte, single word, or double word.

Here is the code written with suffixes:

```
movl    %eax, (%rsp)
movw    (%rax), %dx
movb    $0xFF, %bl
movb    (%rsp,%rdx,4), %dl
movq    (%rdx), %rax
movw    %dx, (%rax)
```

Solution to Problem 3.3 (page 186)

Since we will rely on GCC to generate most of our assembly code, being able to write correct assembly code is not a critical skill. Nonetheless, this exercise will help you become more familiar with the different instruction and operand types.

Here is the code with explanations of the errors:

<code>movb \$0xF, (%ebx)</code>	<i>Cannot use %ebx as address register</i>
<code>movl %rax, (%rsp)</code>	<i>Mismatch between instruction suffix and register ID</i>
<code>movw (%rax), 4(%rsp)</code>	<i>Cannot have both source and destination be memory references</i>
<code>movb %al, %sl</code>	<i>No register named %sl</i>
<code>movl %eax, \$0x123</code>	<i>Cannot have immediate as destination</i>
<code>movl %eax, %dx</code>	<i>Destination operand incorrect size</i>
<code>movb %si, 8(%rbp)</code>	<i>Mismatch between instruction suffix and register ID</i>

Solution to Problem 3.4 (page 187)

This exercise gives you more experience with the different data movement instructions and how they relate to the data types and conversion rules of C. The nuances of conversions of both signedness and size, as well as integral promotion, add challenge to this problem.

src_t	dest_t	Instruction	Comments
long	long	<code>movq (%rdi), %rax</code>	Read 8 bytes
		<code>movq %rax, (%rsi)</code>	Store 8 bytes
char	int	<code>movsbl (%rdi), %eax</code>	Convert char to int
		<code>movl %eax, (%rsi)</code>	Store 4 bytes
char	unsigned	<code>movsbl (%rdi), %eax</code>	Convert char to int
		<code>movl %eax, (%rsi)</code>	Store 4 bytes
unsigned char	long	<code>movzbl (%rdi), %eax</code>	Read byte and zero-extend
		<code>movq %rax, (%rsi)</code>	Store 8 bytes

int	char	<code>movl (%rdi), %eax</code>	Read 4 bytes
		<code>movb %al, (%rsi)</code>	Store low-order byte
unsigned	unsigned	<code>movl (%rdi), %eax</code>	Read 4 bytes
	char	<code>movb %al, (%rsi)</code>	Store low-order byte
char	short	<code>movsbw (%rdi), %ax</code>	Read byte and sign-extend
		<code>movw %ax, (%rsi)</code>	Store 2 bytes

Solution to Problem 3.5 (page 189)

Reverse engineering is a good way to understand systems. In this case, we want to reverse the effect of the C compiler to determine what C code gave rise to this assembly code. The best way is to run a “simulation,” starting with values *x*, *y*, and *z* at the locations designated by pointers *xp*, *yp*, and *zp*, respectively. We would then get the following behavior:

```
void decode1(long *xp, long *yp, long *zp)
    xp in %rdi, yp in %rsi, zp in %rdx
decode1:
    movq    (%rdi), %r8      Get x = *xp
    movq    (%rsi), %rcx     Get y = *yp
    movq    (%rdx), %rax     Get z = *zp
    movq    %r8, (%rsi)      Store x at yp
    movq    %rcx, (%rdx)     Store y at zp
    movq    %rax, (%rdi)     Store z at xp
    ret
```

From this, we can generate the following C code:

```
void decode1(long *xp, long *yp, long *zp)
{
    long x = *xp;
    long y = *yp;
    long z = *zp;

    *yp = x;
    *zp = y;
    *xp = z;
}
```

Solution to Problem 3.6 (page 192)

This exercise demonstrates the versatility of the `leaq` instruction and gives you more practice in deciphering the different operand forms. Although the operand forms are classified as type “Memory” in Figure 3.3, no memory access occurs.

rax = x

Instruction	Result
leaq 6(%rax), %rdx	$6 + x$
leaq (%rax,%rcx), %rdx	$x + y$
leaq (%rax,%rcx,4), %rdx	$x + 4y$ ←
leaq 7(%rax,%rax,8), %rdx	$7 + 9x$
leaq 0xA(,%rcx,4), %rdx	$10 + 4y$
leaq 9(%rax,%rcx,2), %rdx	$9 + x + 2y$

Solution to Problem 3.7 (page 193)

Again, reverse engineering proves to be a useful way to learn the relationship between C code and the generated assembly code.

The best way to solve problems of this type is to annotate the lines of assembly code with information about the operations being performed. Here is a sample:

```

long scale2(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
scale2:
    leaq    (%rdi,%rdi,4), %rax    5 * x
    leaq    (%rax,%rsi,2), %rax    5 * x + 2 * y
    leaq    (%rax,%rdx,8), %rax    5 * x + 2 * y + 8 * z
    ret

```

From this, it is easy to generate the missing expression:

```

long t = 5 * x + 2 * y + 8 * z;

```

Solution to Problem 3.8 (page 194)

This problem gives you a chance to test your understanding of operands and the arithmetic instructions. The instruction sequence is designed so that the result of each instruction does not affect the behavior of subsequent ones.

Instruction	Destination	Value
addq %rcx, (%rax)	0x100	0x100
subq %rdx, 8(%rax)	0x108	0xA8
imulq \$16, (%rax,%rdx,8)	0x118	0x110
incq 16(%rax)	0x110	0x14
decq %rcx	%rcx	0x0
subq %rdx, %rax	%rax	0xFD

Solution to Problem 3.9 (page 195)

This exercise gives you a chance to generate a little bit of assembly code. The solution code was generated by gcc. By loading parameter n in register %ecx, it can then use byte register %cl to specify the shift amount for the sarq instruction. It might seem odd to use a movl instruction, given that n is eight bytes long, but keep in mind that only the least significant byte is required to specify the shift amount.

```

long shift_left4_rightn(long x, long n)
x in %rdi, n in %rsi
shift_left4_rightn:
    movq    %rdi, %rax    Get x
    salq    $4, %rax      x <<= 4
    movl    %esi, %ecx    Get n (4 bytes)
    sarq    %cl, %rax     x >>= n

```

Solution to Problem 3.10 (page 196)

This problem is fairly straightforward, since the assembly code follows the structure of the C code closely.

```

long t1 = x | y;
long t2 = t1 >> 3;
long t3 = ~t2;
long t4 = z-t3;

```

Solution to Problem 3.11 (page 197)

- A. This instruction is used to set register `%rdx` to zero, exploiting the property that $x \wedge x = 0$ for any x . It corresponds to the C statement $x = 0$.
- B. A more direct way of setting register `%rdx` to zero is with the instruction `movq $0,%rdx`.
- C. Assembling and disassembling this code, however, we find that the version with `xorq` requires only 3 bytes, while the version with `movq` requires 7. Other ways to set `%rdx` to zero rely on the property that any instruction that updates the lower 4 bytes will cause the high-order bytes to be set to zero. Thus, we could use either `xorl %edx,%edx` (2 bytes) or `movl $0,%edx` (5 bytes).

Solution to Problem 3.12 (page 200)

We can simply replace the `cqto` instruction with one that sets register `%rdx` to zero, and use `divq` rather than `idivq` as our division instruction, yielding the following code:

```

void uremdiv(unsigned long x, unsigned long y,
             unsigned long *qp, unsigned long *rp)
x in %rdi, y in %rsi, qp in %rdx, rp in %rcx
1  uremdiv:
2      movq    %rdx, %r8    Copy qp
3      movq    %rdi, %rax    Move x to lower 8 bytes of dividend
4      movl    $0, %edx     Set upper 8 bytes of dividend to 0
5      divq    %rsi         Divide by y
6      movq    %rax, (%r8)   Store quotient at qp
7      movq    %rdx, (%rcx)  Store remainder at rp
8      ret

```


Solution to Problem 3.13 (page 204)

It is important to understand that assembly code does not keep track of the type of a program value. Instead, the different instructions determine the operand sizes and whether they are signed or unsigned. When mapping from instruction sequences back to C code, we must do a bit of detective work to infer the data types of the program values.

- A. The suffix ‘l’ and the register identifiers indicate 32-bit operands, while the comparison is for a two’s-complement $<$. We can infer that `data_t` must be `int`.
- B. The suffix ‘w’ and the register identifiers indicate 16-bit operands, while the comparison is for a two’s-complement $>=$. We can infer that `data_t` must be `short`.
- C. The suffix ‘b’ and the register identifiers indicate 8-bit operands, while the comparison is for an unsigned $<=$. We can infer that `data_t` must be `unsigned char`.
- D. The suffix ‘q’ and the register identifiers indicate 64-bit operands, while the comparison is for $!=$, which is the same whether the arguments are signed, unsigned, or pointers. We can infer that `data_t` could be either `long`, `unsigned long`, or some form of pointer.

Solution to Problem 3.14 (page 205)

This problem is similar to Problem 3.13, except that it involves `TEST` instructions rather than `CMP` instructions.

- A. The suffix ‘q’ and the register identifiers indicate a 64-bit operand, while the comparison is for $>=$, which must be signed. We can infer that `data_t` must be `long`.
- B. The suffix ‘w’ and the register identifier indicate a 16-bit operand, while the comparison is for $==$, which is the same for signed or unsigned. We can infer that `data_t` must be either `short` or `unsigned short`.
- C. The suffix ‘b’ and the register identifier indicate an 8-bit operand, while the comparison is for unsigned $>$. We can infer that `data_t` must be `unsigned char`.
- D. The suffix ‘l’ and the register identifier indicate 32-bit operands, while the comparison is for $<$. We can infer that `data_t` must be `int`.

Solution to Problem 3.15 (page 209)

This exercise requires you to examine disassembled code in detail and reason about the encodings for jump targets. It also gives you practice in hexadecimal arithmetic.

- A. The `je` instruction has as its target $0x4003fc + 0x02$. As the original disassembled code shows, this is `0x4003fe`:

```

4003fa: 74 02          je      4003fe
4003fc: ff d0         callq  *%rax

```

- B. The `je` instruction has as its target `0x0x400431 - 12` (since `0xf4` is the 1-byte two's-complement representation of `-12`). As the original disassembled code shows, this is `0x400425`:

```
40042f: 74 f4          je      400425
400431: 5d            pop     %rbp
```

- C. According to the annotation produced by the disassembler, the jump target is at absolute address `0x400547`. According to the byte encoding, this must be at an address `0x2` bytes beyond that of the `pop` instruction. Subtracting these gives address `0x400545`. Noting that the encoding of the `ja` instruction requires 2 bytes, it must be located at address `0x400543`. These are confirmed by examining the original disassembly:

```
400543: 77 02          ja      400547
400545: 5d            pop     %rbp
```

- D. Reading the bytes in reverse order, we can see that the target offset is `0xfffff73`, or decimal `-141`. Adding this to `0x0x4005ed` (the address of the `nop` instruction) gives address `0x400560`:

```
4005e8: e9 73 ff ff    jmpq   400560
4005ed: 90            nop
```

Solution to Problem 3.16 (page 212)

Annotating assembly code and writing C code that mimics its control flow are good first steps in understanding assembly-language programs. This problem gives you practice for an example with simple control flow. It also gives you a chance to examine the implementation of logical operations.

- A. Here is the C code:

```
void goto_cond(long a, long *p) {
    if (p == 0)
        goto done;
    if (*p >= a)
        goto done;
    *p = a;
done:
    return;
}
```

- B. The first conditional branch is part of the implementation of the `&&` expression. If the test for `p` being non-null fails, the code will skip the test of `a > *p`.

Solution to Problem 3.17 (page 212)

This is an exercise to help you think about the idea of a general translation rule and how to apply it.

- A. Converting to this alternate form involves only switching around a few lines of the code:

```

long gotodiff_se_alt(long x, long y) {
    long result;
    if (x < y)
        goto x_lt_y;
    ge_cnt++;
    result = x - y;
    return result;
x_lt_y:
    lt_cnt++;
    result = y - x;
    return result;
}

```

- B. In most respects, the choice is arbitrary. But the original rule works better for the common case where there is no `else` statement. For this case, we can simply modify the translation rule to be as follows:

```

t = test-expr;
if (!t)
    goto done;
then-statement
done:

```

A translation based on the alternate rule is more cumbersome.

Solution to Problem 3.18 (page 213)

This problem requires that you work through a nested branch structure, where you will see how our rule for translating `if` statements has been applied. On the whole, the machine code is a straightforward translation of the C code.

```

long test(long x, long y, long z) {
    long val = x+y+z;
    if (x < -3) {
        if (y < z)
            val = x*y;
        else
            val = y*z;
    } else if (x > 2)
        val = x*z;
    return val;
}

```

Solution to Problem 3.19 (page 216)

This problem reinforces our method of computing the misprediction penalty.

- A. We can apply our formula directly to get $T_{MP} = 2(31 - 16) = 30$.

- B. When misprediction occurs, the function will require around $16 + 30 = 46$ cycles.

Solution to Problem 3.20 (page 219)

This problem provides a chance to study the use of conditional moves.

- A. The operator is `/`. We see this is an example of dividing by a power of 3 by right shifting (see Section 2.3.7). Before shifting by $k = 3$, we must add a bias of $2^k - 1 = 7$ when the dividend is negative.
- B. Here is an annotated version of the assembly code:

```
long arith(long x)
x in %rdi
arith:
leaq    7(%rdi), %rax    temp = x+7
testq   %rdi, %rdi      Text x
cmovns  %rdi, %rax       If x >= 0, temp = x
sarq    $3, %rax         result = temp >> 3 (= x/8)
ret
```

The program creates a temporary value equal to $x + 7$, in anticipation of x being negative and therefore requiring biasing. The `cmovns` instruction conditionally changes this number to x when $x \geq 0$, and then it is shifted by 3 to generate $x/8$.

Solution to Problem 3.21 (page 219)

This problem is similar to Problem 3.18, except that some of the conditionals have been implemented by conditional data transfers. Although it might seem daunting to fit this code into the framework of the original C code, you will find that it follows the translation rules fairly closely.

```
long test(long x, long y) {
    long val = 8*x;
    if (y > 0) {
        if (x < y)
            val = y-x;
        else
            val = x&y;
    } else if (y <= -2)
        val = x+y;
    return val;
}
```

Solution to Problem 3.22 (page 221)

- A. If we build up a table of factorials computed with data type `int`, we get the following:

n	$n!$	OK?
1	1	Y
2	2	Y
3	6	Y
4	24	Y
5	120	Y
6	720	Y
7	5,040	Y
8	40,320	Y
9	362,880	Y
10	3,628,800	Y
11	39,916,800	Y
12	479,001,600	Y
13	1,932,053,504	N

We can see that the computation of $13!$ has overflowed. As we learned in Problem 2.35, when we get value x while attempting to compute $n!$, we can test for overflow by computing x/n and seeing whether it equals $(n-1)!$ (assuming that we have already ensured that the computation of $(n-1)!$ did not overflow). In this case we get $1,932,053,504/13 = 161,004,458.667$. As a second test, we can see that any factorial beyond $10!$ must be a multiple of 100 and therefore have zeros for the last two digits. The correct value of $13!$ is 6,227,020,800.

- B. Doing the computation with data type `long` lets us go up to $20!$, yielding 2,432,902,008,176,640,000.

Solution to Problem 3.23 (page 222)

The code generated when compiling loops can be tricky to analyze, because the compiler can perform many different optimizations on loop code, and because it can be difficult to match program variables with registers. This particular example demonstrates several places where the assembly code is not just a direct translation of the C code.

- A. Although parameter x is passed to the function in register `%rdi`, we can see that the register is never referenced once the loop is entered. Instead, we can see that registers `%rax`, `%rcx`, and `%rdx` are initialized in lines 2–5 to x , $x*x$, and $x+x$. We can conclude, therefore, that these registers contain the program variables.
- B. The compiler determines that pointer p always points to x , and hence the expression $(*p)++$ simply increments x . It combines this incrementing by 1 with the increment by y , via the `leaq` instruction of line 7.
- C. The annotated code is as follows:


```

long dw_loop(long x)
x initially in %rdi
1  dw_loop:
2      movq    %rdi, %rax           Copy x to %rax
3      movq    %rdi, %rcx
4      imulq   %rdi, %rcx           Compute y = x*x
5      leaq    (%rdi,%rdi), %rdx    Compute n = 2*x
6      .L2:                          loop:
7      leaq    1(%rcx,%rax), %rax    Compute x += y + 1
8      subq    $1, %rdx             Decrement n
9      testq   %rdx, %rdx           Test n
10     jg      .L2                   If > 0, goto loop
11     rep; ret                      Return

```

Solution to Problem 3.24 (page 224)

This assembly code is a fairly straightforward translation of the loop using the jump-to-middle method. The full C code is as follows:

```

long loop_while(long a, long b)
{
    long result = 1;
    while (a < b) {
        result = result * (a+b);
        a = a+1;
    }
    return result;
}

```

Solution to Problem 3.25 (page 226)

While the generated code does not follow the exact pattern of the guarded-do translation, we can see that it is equivalent to the following C code:

```

long loop_while2(long a, long b)
{
    long result = b;
    while (b > 0) {
        result = result * a;
        b = b-a;
    }
    return result;
}

```

We will often see cases, especially when compiling with higher levels of optimization, where gcc takes some liberties in the exact form of the code it generates, while preserving the required functionality.

Solution to Problem 3.26 (page 228)

Being able to work backward from assembly code to C code is a prime example of reverse engineering.

- A. We can see that the code uses the jump-to-middle translation, using the `jmp` instruction on line 3.
- B. Here is the original C code:

```
long fun_a(unsigned long x) {
    long val = 0;
    while (x) {
        val ^= x;
        x >>= 1;
    }
    return val & 0x1;
}
```

- C. This code computes the *parity* of argument `x`. That is, it returns 1 if there is an odd number of ones in `x` and 0 if there is an even number.

Solution to Problem 3.27 (page 231)

This exercise is intended to reinforce your understanding of how loops are implemented.

```
long fact_for_gd_goto(long n)
{
    long i = 2;
    long result = 1;
    if (n <= 1)
        goto done;
loop:
    result *= i;
    i++;
    if (i <= n)
        goto loop;
done:
    return result;
}
```

Solution to Problem 3.28 (page 231)

This problem is trickier than Problem 3.26, since the code within the loop is more complex and the overall operation is less familiar.

- A. Here is the original C code:

```
long fun_b(unsigned long x) {
    long val = 0;
    long i;
```

```

    for (i = 64; i != 0; i--) {
        val = (val << 1) | (x & 0x1);
        x >>= 1;
    }
    return val;
}

```

- B. The code was generated using the guarded-do transformation, but the compiler detected that, since i is initialized to 64, it will satisfy the test $i \neq 0$, and therefore the initial test is not required.
- C. This code reverses the bits in x , creating a mirror image. It does this by shifting the bits of x from left to right, and then filling these bits in as it shifts val from right to left.

Solution to Problem 3.29 (page 232)

Our stated rule for translating a for loop into a while loop is just a bit too simplistic—this is the only aspect that requires special consideration.

- A. Applying our translation rule would yield the following code:

```

/* Naive translation of for loop into while loop */
/* WARNING: This is buggy code */
long sum = 0;
long i = 0;
while (i < 10) {
    if (i & 1)
        /* This will cause an infinite loop */
        continue;
    sum += i;
    i++;
}

```

This code has an infinite loop, since the `continue` statement would prevent index variable i from being updated.

- B. The general solution is to replace the `continue` statement with a `goto` statement that skips the rest of the loop body and goes directly to the update portion:

```

/* Correct translation of for loop into while loop */
long sum = 0;
long i = 0;
while (i < 10) {
    if (i & 1)
        goto update;
    sum += i;
update:
    i++;
}

```

Solution to Problem 3.30 (page 236)

This problem gives you a chance to reason about the control flow of a `switch` statement. Answering the questions requires you to combine information from several places in the assembly code.

- Line 2 of the assembly code adds 1 to x to set the lower range of the cases to zero. That means that the minimum case label is -1 .
- Lines 3 and 4 cause the program to jump to the default case when the adjusted case value is greater than 8. This implies that the maximum case label is $-1 + 8 = 7$.
- In the jump table, we see that the entry on lines 6 (case value 3) and 9 (case value 6) have the same destination (`.L2`) as the jump instruction on line 4, indicating the default case behavior. Thus, case labels 3 and 5 are missing in the `switch` statement body.
- In the jump table, we see that the entries on lines 3 and 10 have the same destination. These correspond to cases 0 and 7.
- In the jump table, we see that the entries on lines 5 and 7 have the same destination. These correspond to cases 2 and 4.

From this reasoning, we draw the following conclusions:

- A. The case labels in the `switch` statement body have values -1 , 0, 1, 2, 4, 5, and 7.
- B. The case with destination `.L5` has labels 0 and 7.
- C. The case with destination `.L7` has labels 2 and 4.

Solution to Problem 3.31 (page 237)

The key to reverse engineering compiled `switch` statements is to combine the information from the assembly code and the jump table to sort out the different cases. We can see from the `ja` instruction (line 3) that the code for the default case has label `.L2`. We can see that the only other repeated label in the jump table is `.L5`, and so this must be the code for the cases C and D. We can see that the code falls through at line 8, and so label `.L7` must match case A and label `.L3` must match case B. That leaves only label `.L6` to match case E.

The original C code is as follows:

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case 5:
        c = b ^ 15;
        /* Fall through */
    case 0:
        val = c + 112;
        break;
```

```

case 2:
case 7:
    val = (c + b) << 2;
    break;
case 4:
    val = a;
    break;
default:
    val = b;
}
*dest = val;
}

```

Solution to Problem 3.32 (page 244)

Tracing through the program execution at this level of detail reinforces many aspects of procedure call and return. We can see clearly how control is passed to the function when it is called, and how the calling function resumes upon return. We can also see how arguments get passed through registers `%rdi` and `%rsi`, and how results are returned via register `%rax`.

Label	Instruction		State values (at beginning)					Description
	PC	Instruction	<code>%rdi</code>	<code>%rsi</code>	<code>%rax</code>	<code>%rsp</code>	<code>*%rsp</code>	
M1	0x400560	<code>callq</code>	10	—	—	0x7fffffff820	—	Call <code>first(10)</code>
F1	0x400548	<code>lea</code>	10	—	—	0x7fffffff818	0x400565	Entry of <code>first</code>
F2	0x40054c	<code>sub</code>	10	11	—	0x7fffffff818	0x400565	
F3	0x400550	<code>callq</code>	9	11	—	0x7fffffff818	0x400565	Call <code>last(9, 11)</code>
L1	0x400540	<code>mov</code>	9	11	—	0x7fffffff810	0x400555	Entry of <code>last</code>
L2	0x400543	<code>imul</code>	9	11	9	0x7fffffff810	0x400555	
L3	0x400547	<code>retq</code>	9	11	99	0x7fffffff810	0x400555	Return 99 from <code>last</code>
F4	0x400555	<code>repz repq</code>	9	11	99	0x7fffffff818	0x400565	Return 99 from <code>first</code>
M2	0x400565	<code>mov</code>	9	11	99	0x7fffffff820	—	Resume <code>main</code>

Solution to Problem 3.33 (page 246)

This problem is a bit tricky due to the mixing of different data sizes.

Let us first describe one answer and then explain the second possibility. If we assume the first addition (line 3) implements `*u += a`, while the second (line 4) implements `v += b`, then we can see that `a` was passed as the first argument in `%edi` and converted from 4 bytes to 8 before adding it to the 8 bytes pointed to by `%rdx`. This implies that `a` must be of type `int` and `u` must be of type `long *`. We can also see that the low-order byte of argument `b` is added to the byte pointed to by `%rcx`. This implies that `v` must be of type `char *`, but the type of `b` is ambiguous—it could be 1, 2, 4, or 8 bytes long. This ambiguity is resolved by noting the return value of

6, computed as the sum of the sizes of *a* and *b*. Since we know *a* is 4 bytes long, we can deduce that *b* must be 2.

An annotated version of this function explains these details:

```

int procprob(int a, short b, long *u, char *v)
a in %edi, b in %si, u in %rdx, v in %rcx
1  procprob:
2      movslq %edi, %rdi      Convert a to long
3      addq   %rdi, (%rdx)    Add to *u (long)
4      addb   %sil, (%rcx)    Add low-order byte of b to *v
5      movl   $6, %eax        Return 4+2
6      ret

```

Alternatively, we can see that the same assembly code would be valid if the two sums were computed in the assembly code in the opposite ordering as they are in the C code. This would result in interchanging arguments *a* and *b* and arguments *u* and *v*, yielding the following prototype:

```
int procprob(int b, short a, long *v, char *u);
```

Solution to Problem 3.34 (page 252)

This example demonstrates the use of callee-saved registers as well as the stack for holding local data.

- A. We can see that lines 9–14 save local values *a0*–*a5* into callee-saved registers *%rbx*, *%r15*, *%r14*, *%r13*, *%r12*, and *%rbp*, respectively.
- B. Local values *a6* and *a7* are stored on the stack at offsets 0 and 8 relative to the stack pointer (lines 16 and 18).
- C. After storing six local variables, the program has used up the supply of callee-saved registers. It stores the remaining two local values on the stack.

Solution to Problem 3.35 (page 254)

This problem provides a chance to examine the code for a recursive function. An important lesson to learn is that recursive code has the exact same structure as the other functions we have seen. The stack and register-saving disciplines suffice to make recursive functions operate correctly.

- A. Register *%rbx* holds the value of parameter *x*, so that it can be used to compute the result expression.
- B. The assembly code was generated from the following C code:

```

long rfun(unsigned long x) {
    if (x == 0)
        return 0;
    unsigned long nx = x>>2;
    long rv = rfun(nx);
    return x + rv;
}

```

Solution to Problem 3.36 (page 256)

This exercise tests your understanding of data sizes and array indexing. Observe that a pointer of any kind is 8 bytes long. Data type `short` requires 2 bytes, while `int` requires 4.

Array	Element size	Total size	Start address	Element i
S	2	14	x_S	$x_S + 2i$
T	8	24	x_T	$x_T + 8i$
U	8	48	x_U	$x_U + 8i$
V	4	32	x_V	$x_V + 4i$
W	8	32	x_W	$x_W + 8i$

Solution to Problem 3.37 (page 258)

This problem is a variant of the one shown for integer array E. It is important to understand the difference between a pointer and the object being pointed to. Since data type `short` requires 2 bytes, all of the array indices are scaled by a factor of 2. Rather than using `movl`, as before, we now use `movw`.

Expression	Type	Value	Assembly
S+1	short *	$x_S + 2$	<code>leaq 2(%rdx), %rax</code>
S[3]	short	$M[x_S + 6]$	<code>movw 6(%rdx), %ax</code>
&S[i]	short *	$x_S + 2i$	<code>leaq (%rdx,%rcx,2), %rax</code>
S[4*i+1]	short	$M[x_S + 8i + 2]$	<code>movw 2(%rdx,%rcx,8), %ax</code>
S+i-5	short *	$x_S + 2i - 10$	<code>leaq -10(%rdx,%rcx,2), %rax</code>

Solution to Problem 3.38 (page 259)

This problem requires you to work through the scaling operations to determine the address computations, and to apply Equation 3.1 for row-major indexing. The first step is to annotate the assembly code to determine how the address references are computed:

```

long sum_element(long i, long j)
i in %rdi, j in %rsi
1  sum_element:
2    leaq    0(,%rdi,8), %rdx      Compute 8i
3    subq    %rdi, %rdx           Compute 7i
4    addq    %rsi, %rdx           Compute 7i + j
5    leaq    (%rsi,%rsi,4), %rax   Compute 5j
6    addq    %rax, %rdi           Compute i + 5j
7    movq    Q(,%rdi,8), %rax     Retrieve M[xQ + 8 (5j + i)]
8    addq    P(,%rdx,8), %rax     Add M[xP + 8 (7i + j)]
9    ret

```

We can see that the reference to matrix P is at byte offset $8 \cdot (7i + j)$, while the reference to matrix Q is at byte offset $8 \cdot (5j + i)$. From this, we can determine that P has 7 columns, while Q has 5, giving $M = 5$ and $N = 7$.

Solution to Problem 3.39 (page 262)

These computations are direct applications of Equation 3.1:

- For $L = 4$, $C = 16$, and $j = 0$, pointer A_{ptr} is computed as $x_A + 4 \cdot (16i + 0) = x_A + 64i$.
- For $L = 4$, $C = 16$, $i = 0$, and $j = k$, B_{ptr} is computed as $x_B + 4 \cdot (16 \cdot 0 + k) = x_B + 4k$.
- For $L = 4$, $C = 16$, $i = 16$, and $j = k$, B_{end} is computed as $x_B + 4 \cdot (16 \cdot 16 + k) = x_B + 1,024 + 4k$.

Solution to Problem 3.40 (page 262)

This exercise requires that you be able to study compiler-generated assembly code to understand what optimizations have been performed. In this case, the compiler was clever in its optimizations.

Let us first study the following C code, and then see how it is derived from the assembly code generated for the original function.

```
/* Set all diagonal elements to val */
void fix_set_diag_opt(fix_matrix A, int val) {
    int *Abase = &A[0][0];
    long i = 0;
    long iend = N*(N+1);
    do {
        Abase[i] = val;
        i += (N+1);
    } while (i != iend);
}
```

This function introduces a variable `Abase`, of type `int *`, pointing to the start of array `A`. This pointer designates a sequence of 4-byte integers consisting of elements of `A` in row-major order. We introduce an integer variable `index` that steps through the diagonal elements of `A`, with the property that diagonal elements i and $i + 1$ are spaced $N + 1$ elements apart in the sequence, and that once we reach diagonal element N (index value $N(N + 1)$), we have gone beyond the end.

The actual assembly code follows this general form, but now the pointer increments must be scaled by a factor of 4. We label register `%rax` as holding a value `index4` equal to `index` in our C version but scaled by a factor of 4. For $N = 16$, we can see that our stopping point for `index4` will be $4 \cdot 16(16 + 1) = 1,088$.

```
1  fix_set_diag:
   void fix_set_diag(fix_matrix A, int val)
   A in %rdi, val in %rsi
2      movl    $0, %eax                Set index4 = 0
3      .L13:                                loop:
4      movl    %esi, (%rdi,%rax)        Set Abase[index4/4] to val
5      addq    $68, %rax                Increment index4 += 4(N+1)
```

```

6      cmpq    $1088, %rax          Compare index4: 4N(N+1)
7      jne     .L13                 If !=, goto loop
8      rep; ret                     Return

```

Solution to Problem 3.41 (page 268)

This problem gets you to think about structure layout and the code used to access structure fields. The structure declaration is a variant of the example shown in the text. It shows that nested structures are allocated by embedding the inner structures within the outer ones.

A. The layout of the structure is as follows:

Offset	0	8	12	16	24
Contents	p		s.x	s.y	next

B. It uses 24 bytes.

C. As always, we start by annotating the assembly code:

```

void sp_init(struct prob *sp)
    sp in %rdi
1  sp_init:
2      movl    12(%rdi), %eax      Get sp->s.y
3      movl    %eax, 8(%rdi)      Save in sp->s.x
4      leaq    8(%rdi), %rax      Compute &(sp->s.x)
5      movq    %rax, (%rdi)       Store in sp->p
6      movq    %rdi, 16(%rdi)    Store sp in sp->next
7      ret

```

From this, we can generate C code as follows:

```

void sp_init(struct prob *sp)
{
    sp->s.x = sp->s.y;
    sp->p = &(sp->s.x);
    sp->next = sp;
}

```

Solution to Problem 3.42 (page 269)

This problem demonstrates how a very common data structure and operation on it is implemented in machine code. We solve the problem by first annotating the assembly code, recognizing that the two fields of the structure are at offsets 0 (for v) and 8 (for p).

```

long fun(struct ELE *ptr)
    ptr in %rdi
1  fun:
2      movl    $0, %eax           result = 0
3      jmp     .L2               Goto middle

```

```

4   .L3:                                loop:
5       addq    (%rdi), %rax             result += ptr->v
6       movq    8(%rdi), %rdi           ptr = ptr->p
7   .L2:                                middle:
8       testq   %rdi, %rdi              Test ptr
9       jne     .L3                     If != NULL, goto loop
10      rep; ret

```

A. Based on the annotated code, we can generate a C version:

```

long fun(struct ELE *ptr) {
    long val = 0;
    while (ptr) {
        val += ptr->v;
        ptr = ptr->p;
    }
    return val;
}

```

B. We can see that each structure is an element in a singly linked list, with field *v* being the value of the element and *p* being a pointer to the next element. Function *fun* computes the sum of the element values in the list.

Solution to Problem 3.43 (page 272)

Structures and unions involve a simple set of concepts, but it takes practice to be comfortable with the different referencing patterns and their implementations.

EXPR	TYPE	Code
<code>up->t1.u</code>	long	<code>movq (%rdi), %rax</code> <code>movq %rax, (%rsi)</code>
<code>up->t1.v</code>	short	<code>movw 8(%rdi), %ax</code> <code>movw %ax, (%rsi)</code>
<code>&up->t1.w</code>	char *	<code>addq \$10, %rdi</code> <code>movq %rdi, (%rsi)</code>
<code>up->t2.a</code>	int *	<code>movq %rdi, (%rsi)</code>
<code>up->t2.a[up->t1.u]</code>	int	<code>movq (%rdi), %rax</code> <code>movl (%rdi,%rax,4), %eax</code> <code>movl %eax, (%rsi)</code>
<code>*up->t2.p</code>	char	<code>movq 8(%rdi), %rax</code> <code>movb (%rax), %al</code> <code>movb %al, (%rsi)</code>

Solution to Problem 3.44 (page 275)

Understanding structure layout and alignment is very important for understanding how much storage different data structures require and for understanding the code generated by the compiler for accessing structures. This problem lets you work out the details of some example structures.

A. `struct P1 { int i; char c; int j; char d; };`

i	c	j	d	Total	Alignment
0	4	8	12	16	4

B. `struct P2 { int i; char c; char d; long j; };`

i	c	d	j	Total	Alignment
0	4	5	8	16	8

C. `struct P3 { short w[3]; char c[3] };`

w	c	Total	Alignment
0	6	10	2

D. `struct P4 { short w[5]; char *c[3] };`

w	c	Total	Alignment
0	16	40	8

E. `struct P5 { struct P3 a[2]; struct P2 t };`

a	t	Total	Alignment
0	24	40	8

Solution to Problem 3.45 (page 275)

This is an exercise in understanding structure layout and alignment.

A. Here are the object sizes and byte offsets:

Field	a	b	c	d	e	f	g	h
Size	8	2	8	1	4	1	8	4
Offset	0	8	16	24	28	32	40	48

B. The structure is a total of 56 bytes long. The end of the structure must be padded by 4 bytes to satisfy the 8-byte alignment requirement.

C. One strategy that works, when all data elements have a length equal to a power of 2, is to order the structure elements in descending order of size. This leads to a declaration


```

struct {
    char    *a;
    double  c;
    long    g;
    float    e;
    int      h;
    short    b;
    char     d;
    char     f;
} rec;

```

with the following offsets:

	Field							
	a	c	g	e	h	b	d	f
Size	8	8	8	4	4	2	1	1
Offset	0	8	16	24	28	32	34	35

The structure must be padded by 4 bytes to satisfy the 8-byte alignment requirement, giving a total of 40 bytes.

Solution to Problem 3.46 (page 282)

This problem covers a wide range of topics, such as stack frames, string representations, ASCII code, and byte ordering. It demonstrates the dangers of out-of-bounds memory references and the basic ideas behind buffer overflow.

A. Stack after line 3:

00 00 00 00 00 40 00 76	Return address
01 23 45 67 89 AB CD EF	Saved %rbx
	← buf = %rsp

B. Stack after line 5:

00 00 00 00 00 40 00 34	Return address
33 32 31 30 39 38 37 36	Saved %rbx
35 34 33 32 31 30 39 38	
37 36 35 34 33 32 31 30	← buf = %rsp

- C. The program is attempting to return to address 0x040034. The low-order 2 bytes were overwritten by the code for character '4' and the terminating null character.
- D. The saved value of register %rbx was set to 0x3332313039383736. This value will be loaded into the register before get_line returns.

- E. The call to `malloc` should have had `strlen(buf)+1` as its argument, and the code should also check that the returned value is not equal to `NULL`.

Solution to Problem 3.47 (page 286)

- A. This corresponds to a range of around 2^{13} addresses.
 B. A 128-byte nopsled would cover 2^7 addresses with each test, and so we would only require around $2^6 = 64$ attempts.

This example clearly shows that the degree of randomization in this version of Linux would provide only minimal deterrence against an overflow attack.

Solution to Problem 3.48 (page 288)

This problem gives you another chance to see how x86-64 code manages the stack, and to also better understand how to defend against buffer overflow attacks.

- A. For the unprotected code, we can see that lines 4 and 5 compute the positions of `v` and `buf` to be at offsets 24 and 0 relative to `%rsp`. In the protected code, the canary is stored at offset 40 (line 4), while `v` and `buf` are at offsets 8 and 16 (lines 7 and 8).
 B. In the protected code, local variable `v` is positioned closer to the top of the stack than `buf`, and so an overrun of `buf` will not corrupt the value of `v`.

Solution to Problem 3.49 (page 293)

This code combines many of the tricks we have seen for performing bit-level arithmetic. It requires careful study to make any sense of it.

- A. The `leaq` instruction of line 5 computes the value $8n + 22$, which is then rounded down to the nearest multiple of 16 by the `andq` instruction of line 6. The resulting value will be $8n + 8$ when n is odd and $8n + 16$ when n is even, and this value is subtracted from s_1 to give s_2 .
 B. The three instructions in this sequence round s_2 up to the nearest multiple of 8. They make use of the combination of biasing and shifting that we saw for dividing by a power of 2 in Section 2.3.7.
 C. These two examples can be seen as the cases that minimize and maximize the values of e_1 and e_2 .

n	s_1	s_2	p	e_1	e_2
5	2,065	2,017	2,024	1	7
6	2,064	2,000	2,000	16	0

- D. We can see that s_2 is computed in a way that preserves whatever offset s_1 has with the nearest multiple of 16. We can also see that p will be aligned on a multiple of 8, as is recommended for an array of 8-byte elements.

Solution to Problem 3.50 (page 300)

This exercise requires that you step through the code, paying careful attention to which conversion and data movement instructions are used. We can see the values being retrieved and converted as follows:

- The value at `dp` is retrieved, converted to an `int` (line 4), and then stored at `ip`. We can therefore infer that `val1` is `d`.
- The value at `ip` is retrieved, converted to a `float` (line 6), and then stored at `fp`. We can therefore infer that `val2` is `i`.
- The value of `l` is converted to a `double` (line 8) and stored at `dp`. We can therefore infer that `val3` is `l`.
- The value at `fp` is retrieved on line 3. The two instructions at lines 10–11 convert this to double precision as the value returned in register `%xmm0`. We can therefore infer that `val4` is `f`.

Solution to Problem 3.51 (page 300)

These cases can be handled by selecting the appropriate entries from the tables in Figures 3.47 and 3.48, or using one of the code sequences for converting between floating-point formats.

T_x	T_y	Instruction(s)
long	double	<code>vcvtsi2sdq %rdi, %xmm0, %xmm0</code>
double	int	<code>vcvttisd2si %xmm0, %eax</code>
float	double	<code>vunpcklpd %xmm0, %xmm0, %xmm0</code> <code>vcvtpd2ps %xmm0, %xmm0</code>
long	float	<code>vcvtsi2ssq %rdi, %xmm0, %xmm0</code>
float	long	<code>vcvttss2siq %xmm0, %rax</code>

Solution to Problem 3.52 (page 301)

The basic rules for mapping arguments to registers are fairly simple (although they become much more complex with more and other types of arguments [77]).

A. `double g1(double a, long b, float c, int d);`

Registers: `a` in `%xmm0`, `b` in `%rdi`, `c` in `%xmm1`, `d` in `%esi`

B. `double g2(int a, double *b, float *c, long d);`

Registers: `a` in `%edi`, `b` in `%rsi`, `c` in `%rdx`, `d` in `%rcx`

C. `double g3(double *a, double b, int c, float d);`

Registers: `a` in `%rdi`, `b` in `%xmm0`, `c` in `%esi`, `d` in `%xmm1`

D. `double g4(float a, int *b, float c, double d);`

Registers: `a` in `%xmm0`, `b` in `%rdi`, `c` in `%xmm1`, `d` in `%xmm2`

Solution to Problem 3.53 (page 303)

We can see from the assembly code that there are two integer arguments, passed in registers `%rdi` and `%rsi`. Let us name these `i1` and `i2`. Similarly, there are two floating-point arguments, passed in registers `%xmm0` and `%xmm1`, which we name `f1` and `f2`.

We can then annotate the assembly code:

```

Refer to arguments as i1 (%rdi), i2 (%esi)
                        f1 (%xmm0), and f2 (%xmm1)

double funct1(arg1_t p, arg2_t q, arg3_t r, arg4_t s)
1  funct1:
2      vcvtsi2ssq        %rsi, %xmm2, %xmm2    Get i2 and convert from long to float
3      vaddss            %xmm0, %xmm2, %xmm0    Add f1 (type float)
4      vcvtsi2ss        %edi, %xmm2, %xmm2    Get i1 and convert from int to float
5      vdivss            %xmm0, %xmm2, %xmm0    Compute i1 / (i2 + f1)
6      vunpcklps        %xmm0, %xmm0, %xmm0
7      vcvtps2pd         %xmm0, %xmm0         Convert to double
8      vsubsd            %xmm1, %xmm0, %xmm0    Compute i1 / (i2 + f1) - f2 (double)
9      ret

```

From this we see that the code computes the value $i1/(i2+f1)-f2$. We can also see that $i1$ has type `int`, $i2$ has type `long`, $f1$ has type `float`, and $f2$ has type `double`. The only ambiguity in matching arguments to the named values stems from the commutativity of multiplication—yielding two possible results:

```

double funct1a(int p, float q, long r, double s);
double funct1b(int p, long q, float r, double s);

```

Solution to Problem 3.54 (page 303)

This problem can readily be solved by stepping through the assembly code and determining what is computed on each step, as shown with the annotations below:

```

double funct2(double w, int x, float y, long z)
w in %xmm0, x in %edi, y in %xmm1, z in %rsi
1  funct2:
2      vcvtsi2ss        %edi, %xmm2, %xmm2    Convert x to float
3      vmulss           %xmm1, %xmm2, %xmm1    Multiply by y
4      vunpcklps       %xmm1, %xmm1, %xmm1
5      vcvtps2pd        %xmm1, %xmm2         Convert x*y to double
6      vcvtsi2sdq       %rsi, %xmm1, %xmm1    Convert z to double
7      vdivsd           %xmm1, %xmm0, %xmm0    Compute w/z
8      vsubsd           %xmm0, %xmm2, %xmm0    Subtract from x*y
9      ret                                Return

```

We can conclude from this analysis that the function computes $y * x - w/z$.

Solution to Problem 3.55 (page 305)

This problem involves the same reasoning as was required to see that numbers declared at label `.LC2` encode 1.8, but with a simpler example.

We see that the two values are 0 and 1077936128 (0x40400000). From the high-order bytes, we can extract an exponent field of 0x404 (1028), from which we subtract a bias of 1023 to get an exponent of 5. Concatenating the fraction bits of the two values, we get a fraction field of 0, but with the implied leading value giving value 1.0. The constant is therefore $1.0 \times 2^5 = 32.0$.

Solution to Problem 3.56 (page 305)

- A. We see here that the 16 bytes starting at address `.LC1` form a mask, where the low-order 8 bytes contain all ones, except for the most significant bit, which is the sign bit of a double-precision value. When we compute the AND of this mask with `%xmm0`, it will clear the sign bit of `x`, yielding the absolute value. In fact, we generated this code by defining `EXPR(x)` to be `fabs(x)`, where `fabs` is defined in `<math.h>`.
- B. We see that the `vxorpd` instruction sets the entire register to zero, and so this is a way to generate floating-point constant 0.0.
- C. We see that the 16 bytes starting at address `.LC2` form a mask with a single 1 bit, at the position of the sign bit for the low-order value in the XMM register. When we compute the EXCLUSIVE-OR of this mask with `%xmm0`, we change the sign of `x`, computing the expression `-x`.

Solution to Problem 3.57 (page 308)

Again, we annotate the code, including dealing with the conditional branch:

```
double funct3(int *ap, double b, long c, float *dp)
ap in %rdi, b in %xmm0, c in %rsi, dp in %rdx
1  funct3:
2      vmovss (%rdx), %xmm1                Get d = *dp
3      vcvtsi2sd    (%rdi), %xmm2, %xmm2    Get a = *ap and convert to double
4      vucomisd     %xmm2, %xmm0            Compare b:a
5      jbe         .L8                     If <=, goto lesseq
6      vcvtsi2ssq    %rsi, %xmm0, %xmm0     Convert c to float
7      vmulss %xmm1, %xmm0, %xmm1          Multiply by d
8      vunpcklps     %xmm1, %xmm1, %xmm1
9      vcvtps2pd     %xmm1, %xmm0          Convert to double
10     ret                                Return
11  .L8:                                lesseq:
12     vaddss %xmm1, %xmm1, %xmm1            Compute d+d = 2.0 * d
13     vcvtsi2ssq     %rsi, %xmm0, %xmm0     Convert c to float
14     vaddss %xmm1, %xmm0, %xmm0            Compute c + 2*d
15     vunpcklps     %xmm0, %xmm0, %xmm0
16     vcvtps2pd     %xmm0, %xmm0          Convert to double
17     ret                                Return
```

From this, we can write the following code for `funct3`:

```
double funct3(int *ap, double b, long c, float *dp) {
    int a = *ap;
    float d = *dp;
    if (a < b)
        return c*d;
    else
        return c+2*d;
}
```